

Improved Rate for Non-Malleable Codes and Time-Lock Puzzles

Cody Freitag*
Northeastern University and
Hebrew University of Jerusalem
codyrfreitag@gmail.com

Ilan Komargodski†
Hebrew University of Jerusalem
ilank@cs.huji.ac.il

Manu Kondapaneni
Northeastern University
kondapaneni.m@northeastern.edu

Jad Silbak‡
MIT
jadsilbak@gmail.com

Abstract

Non-malleable codes allow a sender to transmit a message to a receiver, while providing a “best-possible” integrity guarantee to ensure that no attacker—who cannot already decode the message—can meaningfully tamper the message in transit. If tampered, the received message should either be invalid or unrelated to the original message. Non-malleable time-lock puzzles (TLPs) are a special case of non-malleable codes for bounded polynomial-depth tampering with very efficient encoding.

In this work, we give generic techniques for constructing non-malleable codes and non-malleable TLPs with improved *rate*, which captures the ratio of a message’s length to its encoding length.

A key contribution of our work is identifying a security notion for non-malleability, which we term “CCA-hiding”, sufficient for our compilers. CCA-hiding is a relaxation of CCA-security for encryption or commitments to the fine-grained setting of codes, and requires that the encoded message remains hidden, even given a decoding oracle for any other codeword. Intriguingly, CCA-hiding does not imply non-malleability in the fine-grained setting, as is the case for encryption and commitments.

Using our new techniques, we give the following constructions:

- Rate-1 CCA-hiding TLPs in the plain model.
- Rate-1 non-malleable codes for bounded polynomial-depth tampering in the auxiliary-input random oracle model (AI-ROM).
- Rate-(1/2) non-malleable TLPs in the AI-ROM.

*Supported in part by a Khoury College Distinguished Postdoctoral Fellowship and the European Union (ERC, SCALE, 101162665). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

†Supported in part by the European Union (ERC, SCALE, 101162665). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

‡Part of this work was done while the author was a postdoctoral research fellow at the Simons Institute for the Theory of Computing (research supported in part by a grant from the UC Noyce Initiative to the Simons Institute for the Theory of Computing.)

1 Introduction

Non-malleable codes (NMCs), introduced by Dziembowski, Pietrzak, and Wichs [DPW10], allow a sender to encode a message while maintaining a “best-possible” integrity guarantee for the underlying message. Namely, it should be impossible to tamper—or *maul*—a codeword in transit to one that decodes to another related message. This security notion is only achievable against bounded tampering adversaries that cannot decode the message, as otherwise such an adversary could decode the message and simply output a codeword for a related message. Non-malleable codes have been widely studied and have been constructed for a variety of adversarial tampering classes including split-state, bounded space, bounded depth, and bounded size tampering (see, e.g., [DPW10, LL12, ADKO15, FMVW16, BDKM16a, CL17, FHMV17, ADL18, BDG⁺18, BDKM18, CFV19, BDK⁺19, BGW19, BCL⁺20, DKP21, BDL22, BSS24]).

Of particular interest are *non-malleable time-lock puzzles* (TLPs), which are NMCs for bounded polynomial-depth tampering, where the encoding algorithm is required to be much more efficient than decoding. In recent years, non-malleable TLPs have received significant interest due to their applications to distributed coin-flipping, auctions, and broadcast (see, e.g., [BN00, FKPS21, BDD⁺21, TCLM21, TAF⁺23, ALP⁺24]).

In this work, we are interested in the efficiency of non-malleable codes and time-lock puzzles. A particularly important efficiency measure is a code’s *rate*, which is an asymptotic notion capturing the ratio of the message length to its encoding length. The “holy grail” in terms of efficiency is building rate-1 codes, where the codeword is essentially the same length as the underlying message (up to additive terms that depend only on the security parameter). There is also great practical interest in improving the rate of non-malleable TLPs for their aforementioned applications to the blockchain setting where communication and storage costs are often prohibitively expensive (see e.g., [Eth] for example costs for Ethereum). As it stands, most known NMCs (e.g., [ADKO15, BDKM16a, CL17, ADL18, BDG⁺18, BDKM18, BDK⁺19, BGW19, BCL⁺20, BDL22]) and non-malleable time-lock puzzles (e.g., [KLX20, DKP21, FKPS21, TCLM21]) are *not* rate-1.

If we only care about hiding the underlying message m , it is easy to construct a rate-1 encoding Enc given any encoding Enc' and a pseudo-random generator PRG : simply encode a PRG seed s used to mask the message with a one-time pad,

$$\text{Enc}(m) := (\text{Enc}'(s), \text{PRG}(s) \oplus m).$$

This same general idea works in many contexts including for encryption and commitments. Note, however, that this construction is clearly malleable; adding 1 to the one-time pad results in an encoding of $m \oplus 1$. Indeed, non-malleability is notoriously difficult to compose. For example, $\text{Enc}(a, b) := (\text{Enc}'(a), \text{Enc}'(b))$ is malleable even if Enc' is non-malleable (e.g., a tampering adversary could simply swap the two encodings). So, in this paper, we ask:

*Can we generically improve the rate of any
non-malleable code or time-lock puzzle?*

1.1 Our Results

We answer the above question affirmatively by giving various transformations and constructions improving the rate of existing non-malleable codes and non-malleable TLPs in different settings. A key contribution of our work is identifying the right security notion for the underlying codes in our transformations. We call this notion *chosen-code attack* (CCA) hiding.

Non-malleability vs. CCA-hiding. We are interested in encoding schemes consisting of public, polynomial-time encoding and decoding algorithms (Enc, Dec). While conceptually similar, we compare CCA-hiding and non-malleability explicitly to clarify their differences.

- *CCA-hiding* requires that, for any pair of messages m_0, m_1 , any *bounded* distinguisher D (from a specified class of adversaries that can't compute $\text{Dec}(\cdot)$) can't predict b given a code $c \leftarrow \text{Enc}(m_b)$, even given an oracle for $\text{Dec}(\cdot)$ on any input $c' \neq c$.
- *Non-malleability* is modeled via the following “meddler-in-the-middle” (MIM) experiment for a *bounded* mauling adversary M and message m ,

$$\text{MIM}_M(m) := \text{Dec}(M(\text{Enc}(m))),$$

where $\text{Dec}(\cdot)$ returns $\perp$ if M simply copies the encoding c it receives as input. Non-malleability requires that, for any pair of messages m_0, m_1 , any polynomial-time distinguisher D (that *can* compute $\text{Dec}(\cdot)$) can't predict b given $\tilde{m} \leftarrow \text{MIM}_M(m_b)$.¹

We summarize the above two security games in Figure 1 below.

The above two definitions are similar in that they give the security experiment limited access to a decoding oracle $\text{Dec}(\cdot)$. However, they differ in that CCA-hiding ensures indistinguishability against bounded adversaries that can adaptively query $\text{Dec}(\cdot)$, whereas non-malleability ensures indistinguishability against *arbitrary polynomial-time* adversaries for the output of one call to $\text{Dec}(\cdot)$. While the classical notion of CCA-security implies non-malleability for encryption or commitments, we will see that this is not the case in the fine-grained setting of codes.² Still, we will show that CCA-hiding is a useful tool for composition, and in particular, for achieving non-malleability.

Summary of our results. In the rest of the section, we give an overview of our main results, summarized as follows:

- We give a generic transformation from any CCA-hiding encoding scheme to a rate-1 CCA-hiding encoding scheme. This transformation relies only on the existence of collision resistant hash functions. We note that this transformation also applies to non-interactive commitments.
- We next give a direct construction of a rate-1 non-malleable code for bounded polynomial-depth tampering. This construction is in the auxiliary-input random oracle model (AI-ROM) of Unruh [Unr07], where the adversary in the security game is allowed to depend arbitrarily on the random oracle.
- Lastly, we give a generic transformation from any CCA-hiding encoding scheme to a rate-1/2 *non-malleable* encoding scheme in the AI-ROM.

¹We follow the notion of non-malleability from [FKPS21] for non-malleable TLPs, which is necessary for their applications to multi-party coin-flipping and auctions. In the NMC literature, this is sometimes referred to as “strong” non-malleability where we additionally protect against attacks that maul the code c to a fresh encoding $c' \neq c$ for the same underlying message m .

²In particular, [FKPS21] give a construction that *is* malleable yet satisfies a restricted notion of CCA-hiding for one oracle call to $\text{Dec}(\cdot)$.

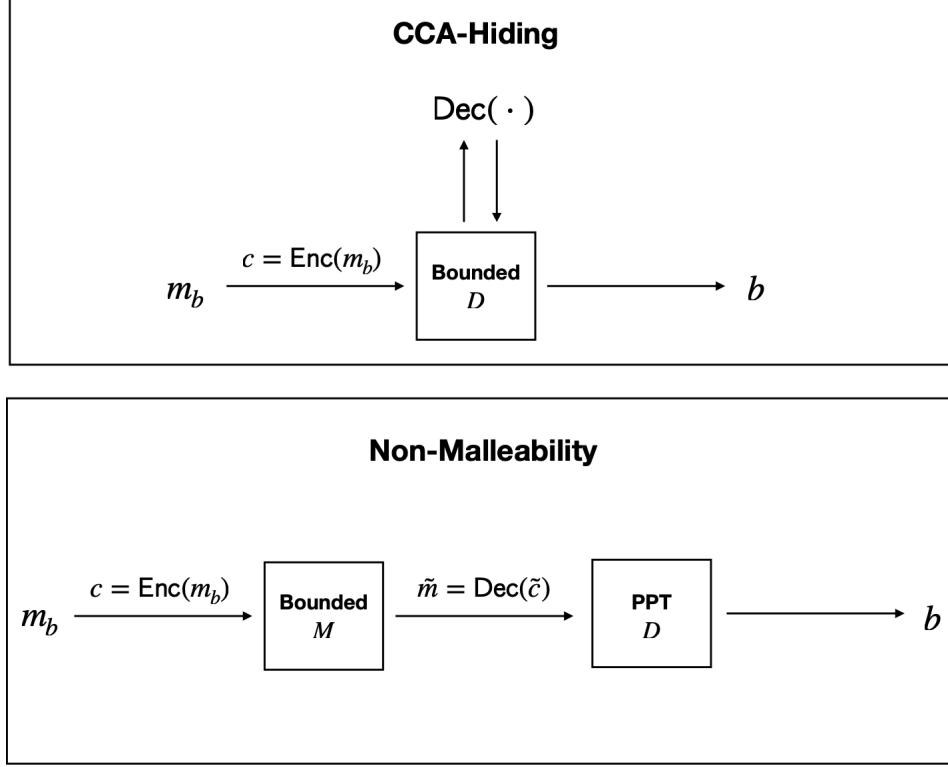


Figure 1: Comparison of CCA-hiding and non-malleability security games for polynomial-time encoding and decoding algorithms (Enc , Dec). In the games above, $\text{Dec}(\cdot)$ outputs $\perp$ on input the challenge encoding c , and we refer to an algorithm as “bounded” if it comes from a specified class that cannot compute $\text{Dec}(\cdot)$.

1.1.1 Rate-1 compiler for CCA-hiding

Our first result is a generic rate-1 compiler for CCA-hiding codes.

Theorem 1.1 (Rate-1 Compiler for CCA-Hiding; Informal, see Theorem 5.1). *Assuming collision-resistant hash functions, there is a generic compiler that given a CCA-hiding encoding scheme (with any rate), outputs a rate-1 CCA-hiding encoding scheme.*

We additionally show that the non-malleable TLPs from [DKP21, FKPS21] satisfy CCA-hiding (see Sections A and B). We prove the construction of [DKP21] satisfies CCA-hiding in the plain model from the same assumptions of TLPs, keyless multi-collision resistant hash functions (MCRH), injective one-way functions (OWF), and non-interactive witness indistinguishable (NIWI) proofs, all with sub-exponential security. The other construction of [FKPS21] is proven secure assuming the existence of sub-exponentially secure TLPs in the AI-ROM. We prove their construction satisfies CCA-hiding assuming only polynomially secure TLPs in the AI-ROM. As a result, we get the following corollaries from our generic compiler of Theorem 1.1.

Corollary 1.2 (Rate-1 CCA-Secure Time-Lock Puzzles). *There exist rate-1 CCA-hiding time-lock puzzles*

- in the plain model assuming TLPs, keyless MCRH, injective OWFs, and NIWI proofs, all with sub-exponential security, and
- in the AI-ROM assuming TLPs with polynomial security.

Applications to non-malleable non-interactive commitments. Our compilers work for arbitrary encoding schemes (Enc, Dec) with security against some class of polynomially bounded adversaries. While we usually think of Dec($\cdot$) as being relatively efficient, non-interactive commitments can be viewed as encoding schemes with inefficient decoding algorithms; you can simply guess the commitment randomness. This makes it possible to achieve notions of non-malleability against arbitrary polynomial-time tampering.

Theorem 1.1 gives a simple transformation to build rate-1 CCA-secure (and hence non-malleable) non-interactive commitments from any CCA-secure non-interactive commitment. Somewhat surprisingly, we observe in Appendix C that the recent work of [GKLW23] directly gives a rate-1 CCA-secure non-interactive commitment, even though they don’t explicitly make note of this.³ Our compiler gives a generic way to achieve rate-1 for any future CCA-secure non-interactive commitment from simpler assumptions, or for the construction of [BL18] (assuming it is also CCA-secure).

1.1.2 Rate-1 non-malleable codes in the AI-ROM

One could hope the codes based on Theorem 1.1 are already non-malleable. Unfortunately, we show an explicit attack on non-malleability for these constructions (see Section 2.1.1). As such, we require new techniques for achieving non-malleable codes or time-lock puzzles in the fine-grained setting.

For our next result, we give a direct construction of a rate-1 NMC for bounded polynomial-depth tampering. This construction relies on a random oracle and security is proven in the AI-ROM.

Theorem 1.3 (Rate-1 Non-Malleable Code in the AI-ROM; Informal, see Theorem 6.1). *There exists a rate-1 non-malleable code for bounded polynomial-depth tampering in the AI-ROM.*

In contrast to time-lock puzzles, we emphasize that encoding in the construction above requires polynomial depth which is larger than the depth-bound for the tampering class. In a similar setting, the recent work of Ball, Shaltiel, and Silbak [BSS24] gives *rate-1* NMCs for bounded polynomial-size tampering. Their constructions only tolerate computational indistinguishability up to a fixed inverse polynomial, which is inherent based on the complexity-theoretic style assumptions they use. In contrast, our construction above achieves negligible security.

We also emphasize that our construction of Theorem 1.3 requires only a random oracle and makes no additional assumptions. Mahmoody, Moran, and Vadhan [MMV11] show that this is only possible when encoding is as slow as decoding, so TLPs are impossible to achieve in this setting.

1.1.3 Rate-1/2 compiler for non-malleability from CCA-hiding in the AI-ROM

We next give a generic compiler for CCA-hiding codes that gives rate-1/2 non-malleable codes in the AI-ROM.

³Their primary motivation was to remove the non-black-box use of crypto primitives in the previous constructions of [LPS17, BL18].

Theorem 1.4 (Rate-1/2 Compiler from CCA-Hiding to Non-Malleability in the AI-ROM; Informal, see Theorem 7.2). *In the AI-ROM, there is a generic compiler that given a CCA-hiding encoding scheme, outputs a rate-1/2 non-malleable encoding scheme.*

Again, plugging in the AI-ROM TLP of [FKPS21] into this new compiler, we get the following direct improvement over their non-malleable TLP as an immediate corollary to Theorem 1.4.

Corollary 1.5 (Rate-1/2 Non-Malleable TLPs in the AI-ROM). *Assuming polynomially-secure TLPs, there exists a rate-1/2 non-malleable time-lock puzzle in the AI-ROM.*

Applications to multi-party coin-flipping and auctions. In the full Theorem 7.2, we prove that the compiler of Theorem 1.4 above results in a stronger notion of functional non-malleability introduced by [FKPS21] as a way to bypass the impossibility of concurrent non-malleability for TLPs. At a high level, functional non-malleability with respect to a multi-input function f captures the following scenario:

- A bounded tampering adversary receives $c \leftarrow \text{Enc}(m)$ and outputs a sequence of codes $\tilde{c}_1, \dots, \tilde{c}_k$ that decode to values $\tilde{m}_1, \dots, \tilde{m}_k$ (ignoring any copied codes). The adversary wins if $f(\tilde{m}_1, \dots, \tilde{m}_k)$ is non-trivially related to m .

The work of [FKPS21] show that this is useful for applications to distributed coin-flipping and auction protocols. Using our non-malleable TLPs in these protocols results in significantly less overall communication while relying on weaker assumptions. To highlight this, we briefly outline the application to fair coin-flipping, and refer to their paper for further details.

To generate ℓ bits of shared randomness, each party i uses a non-malleable TLP to “commit” to a string $r_i \in \{0, 1\}^\ell$ within a fixed window of time where the time-locked security holds. Each puzzle is then opened to reveal r_i —potentially by calling $\text{Dec}(\cdot)$ to ensure all TLPs are opened—and then all parties agree on the random string $r = \oplus_i r_i$. Now, suppose that party j is honest and commits to a random value $r_j \leftarrow \{0, 1\}^\ell$. An attacker who wants to bias the output r must come up with TLPs to values $\{r_i\}_{i \neq j}$ within the short window of time where TLPs are accepted. Functional non-malleability for the $\oplus$ function guarantees that $\oplus_{i \neq j} r_i$ doesn’t depend on the value of r_j , so it follows that the output r is indistinguishable from a uniformly random string.

On random oracle uninstantiability results for TLPs. While the compiler above for TLPs is provably secure in the AI-ROM, we show that if the underlying CCA-hiding TLP satisfies certain strong “fully homomorphic” properties, the resulting TLP is actually malleable for any concrete instantiation of the random oracle. Still, we believe that this transformation is secure for most natural underlying CCA-hiding TLPs. Moreover, constructions in the AI-ROM serve as useful heuristics for practical constructions and are important first steps towards constructions in the plain model.

This random oracle uninstantiability result is similar to other such results [CGH98] where the scheme is only computationally secure in the ROM, as is the case for the Fujisaki-Okamoto transform [FO99, FKPS21, BFM15, GKW17] and the Fiat-Shamir heuristic for arguments [FS86, Bar01, GK03, BBH⁺19, KRS25]. Namely, for certain choice of the underlying computational ingredients, there are similar attacks for any instantiation of the random oracle.

We give a formal treatment for our uninstantiability result for the resulting TLP from Theorem 1.4 in Section 7.1. As our rate-1 NMC from Theorem 1.3 is proven unconditionally secure in the AI-ROM, we are unaware of any similar attacks on this construction.

1.2 Related Work

Non-malleable encoding schemes including codes, time-lock puzzles, and non-interactive commitments have received significant interest since the seminal motivating works of [NY90, RS91, DDN91]. We provide a brief overview of the related literature in these areas.

Non-malleable codes. Non-malleable codes were first introduced by [DPW10] and have been constructed for a variety of adversarial tampering classes including split-state tampering [DPW10, LL12, CL17, ADL18, ADKO15], algebraic tampering [BCL⁺20], bounded space tampering [FHMV17, BDKM18], bounded size tampering [DPW10, FMVW16, BDKM18, BDK⁺19, BDL22, BSS24], or bounded depth tampering [BDKM16a, BDG⁺18, BGW19, DKP21]. The various works above give constructions in a variety of settings (probabilistic vs. explicit constructions, CRS vs. plain model, inverse polynomial vs. negligible security), but in this work, we are primarily interested in explicit constructions without a CRS that achieve negligible security against either bounded polynomial-size or -depth adversaries. Also, to the best of our knowledge, CCA-hiding has not been explicitly studied in the context of non-malleable codes.

Non-malleable time-lock puzzles. Non-malleable TLPs (and the related notion of non-malleable non-interactive timed commitments) have received significant attention due to their applications to distributed protocols like coin-flipping, fair auctions, and broadcast [BN00, FKPS21, BDD⁺21, TCLM21, TAF⁺23, ETY24, ALP⁺24]. While some non-malleable TLP constructions rely on adversary-independent setup [KLX20, BDD⁺21, TCLM21, TAF⁺23, CJ23, Ahr24], we are primarily interested in the fully non-interactive setting in this work following [FKPS21] and inspired by the original notion of TLPs from [RSW96]. We note that Katz, Loss, and Xu [KLX20] define a notion of CCA-hiding for non-interactive timed commitments, which was subsequently considered in the works of [TCLM21, CJ23, Ahr24]. [TCLM21] show that this notion suffices for “short-lived” applications that only require indistinguishability for a limited amount of time. Following [FKPS21], we are primarily concerned with applications that guarantee the standard notion of computational indistinguishability against arbitrary polynomial-time attackers.

Non-malleable non-interactive commitments. Non-malleable and CCA-secure (even interactive) commitments have been extremely useful for building more efficient and secure multi-party computation protocols [Bar02, Wee10, CLP10, LP12, Kiy14, GLP⁺15, LPS17, BHP17]. This has led to a long sequence of works trying to minimize the round complexity and assumptions needed for constructing such commitments [DDN91, Bar02, PR05b, PR05a, LPV08, PPV08, LP09, PW10, Wee10, Goy11, LP11, GLOV12, GPR16, COSV16, COSV17, Khu17, LPS17, KS17, BL18, KK19, GR19, GKLW21, Khu21, GKLW23]. Despite known black-box barriers for constructing non-interactive non-malleable commitments [Pas13], there has been significant progress recently in the fully non-interactive setting that we consider in this work [PPV08, LPS17, BL18, KK19, GKLW21, Khu21, GKLW23].

Rate compilers for other non-malleable primitives. Our work is not the first to explore rate-optimizing compilers for non-malleable codes (NMCs). All existing compilers, including ours, follow a similar overarching design inspired by the “hybrid” encryption paradigm: each combines a low-rate non-malleable code with a form of symmetric-key authenticated encryption. However, despite

this high-level resemblance, the specific instantiations of these components differ significantly across works, and each setting presents unique challenges.

The most closely related work to ours is [BSS24] as they consider tampering classes that may touch the entire codeword. However, as discussed before, their construction does not achieve negligible security. All other compilers are for settings where the tampering is subject to some form of locality constraint: each tampered output bit is computed by a function that has access only to partial information from the original codeword.

- **Local Tampering:** The first rate compiler by [AGM⁺15] boosted NMCs for 1-local tampering to rate 1 by combining threshold secret sharing with bit-flipping and error correction. Later, [GMW19] extended these ideas (together with ideas from [BDKM16b]) to achieving rate 1 NMCs for $c \log n$ -local functions.
- **Computational Split-State Tampering:** [AAG⁺16] showed how to obtain rate 1 NMCs for polynomial-time split-state tampering by encoding a high-rate authenticated encryption key using an augmented split-state NMC, introducing the notion of augmented non-malleability to address joint tampering across components.
- **Split-State Tampering:** A sequence of works [KOS17, KOS18, GMW17, AKO⁺22] used compilers to improve the rate of NMCs for split-state and t -state tampering, culminating in an explicit rate $1/3$ NMC for 2-state tampering—the best known to date. These compilers rely on leakage-resilient encodings combined with information-theoretic MACs and use seeded extractors and low-rate NMCs to protect the seeds and keys. The key technical challenge lies in preserving independence between tampering states.

2 Technical Overview

For the sake of this overview, we focus on the setting of TLPs, but we note that we provide results for general codes when relevant in the technical sections of the paper. We start by recalling the definition of a TLP. Throughout this overview, we consider a fixed security parameter λ and time bound t for simplicity.

A TLP consists of an encoding algorithm Enc and a decoding algorithm Dec . Correctness for TLPs stipulates that if you encode a message m resulting in a code c , decoding c should give back m . In short, $m = \text{Dec}(\text{Enc}(m))$. We recall the two security notions of CCA-hiding and non-malleability from Figure 1 specifically for the setting of TLPs:

- *CCA-hiding* requires that, for any pair of equal-length messages m_0, m_1 , any distinguishing adversary D running in depth $\ll t$ can't predict b given a code $c \leftarrow \text{Enc}(m_b)$, even given an oracle for $\text{Dec}(\cdot)$ on any input $c' \neq c$.
- *Non-malleability* is modelled via the following “meddler-in-the-middle” (MIM) experiment for a depth $\ll t$ mauling adversary M and message m ,

$$\text{MIM}_M(m) := \text{Dec}(M(\text{Enc}(m))),$$

where $\text{Dec}(\cdot)$ returns $\perp$ if M simply copies its input. Non-malleability requires that, for any pair of equal-length messages m_0, m_1 , any polynomial-time distinguisher D (that may run in depth $\geq t$) can't predict b given $\tilde{m} \leftarrow \text{MIM}_M(m_b)$.

In Section 2.1, we describe a generic compiler that converts any CCA-hiding TLP into a rate-1 CCA-hiding TLP (Theorem 1.1). We then describe why this falls short of giving a non-malleable TLP. In Section 2.2, we give a generic compiler in the AI-ROM that gives a rate-1/2 non-malleable TLP from any CCA-hiding TLP (Theorem 1.4). Finally, in Section 2.3, we give a direct construction of a rate-1 non-malleable code in the AI-ROM (Theorem 1.3).

2.1 Rate-1 CCA-Hiding Compiler

Our overall goal is to give a generic compiler that converts any “sufficiently non-malleable” TLP $(\text{Enc}', \text{Dec}')$ into a new non-malleable TLP (Enc, Dec) with improved rate. Towards this goal, we start with a simple construction that gives a rate-1 TLP. The idea is to simply encode a short seed s for a PRG G and then use $G(s)$ to mask the message m . An encoding $\text{Enc}(m)$ would consist of an inner encoding $c' \leftarrow \text{Enc}'(s)$ with a one-time pad of the message $\text{pad} = G(s) \oplus m$. While this does *hide* the message m , it is clearly malleable. Given a TLP $c = (c', \text{pad}) \leftarrow \text{Enc}(m)$, a mauling attack that outputs $(c', \text{pad} \oplus 1)$ results in a valid TLP for $m \oplus 1$.

To circumvent this simple attack, we use a collision-resistant hash to bind the underlying TLP to the message m . Namely, together with s , you also encode a hash key hk and a hash of m . This gives our first compiler,

$$\text{Enc}(m) := (c' \leftarrow \text{Enc}'(s, \text{hk}, \text{Hash}(\text{hk}, m)), \text{pad} = G(s) \oplus m).$$

To decode, you can first decode $(s, \text{hk}, \text{dig})$ from $\text{Dec}'(c')$, unmask pad to get $\hat{m} = G(s) \oplus \text{pad}$, and check that $\text{dig} = \text{Hash}(\text{hk}, \hat{m})$. In case the check fails, the decoding algorithm will simply output $\perp$.

The construction is clearly rate-1 since the overhead is simply the underlying TLP encoding with length independent of $|m|$. Proving security, on the other hand, turns out to be a bit subtle. In particular, we identify that the underlying TLP $(\text{Enc}', \text{Dec}')$ needs to be CCA-hiding, and we will show that (Enc, Dec) is then also CCA-hiding.

Security analysis. While we eventually argue CCA-hiding of this construction, it will be instructive to present and analyze the security as if it were a non-malleability experiment for a bounded depth distinguisher D . For messages m_0, m_1 , recall that the experiment consists of a bounded depth mauler M that receives a challenge TLP $c \leftarrow \text{Enc}(m_b)$ for a random bit b as input. M tries to maul c to a new TLP $\tilde{c}$, with underlying message $\tilde{m}$. Then, a distinguisher D given $\tilde{m}$ tries to predict the original bit b .

We now look at the different ways that M could potentially maul the challenge TLP c . Recall that the encoding c consists of two parts: $c' \leftarrow \text{Enc}'(s, \text{hk}, \text{dig})$ for $\text{dig} = \text{Hash}(\text{hk}, m_b)$, and $\text{pad} = G(s) \oplus m_b$. M then outputs a new TLP $\tilde{c} = (\tilde{c}', \tilde{\text{pad}})$.

- First, if $\tilde{c}' = c'$, meaning that M copied the first input, then it must be the case that $\tilde{\text{pad}} \neq \text{pad}$ (otherwise the whole TLP was copied, which isn't allowed). Since $\tilde{c}'$ was copied, it decodes to the same underlying values $(s, \text{hk}, \text{dig})$ as c' . The candidate message $\hat{m} = G(s) \oplus \tilde{\text{pad}}$ must be different from m_b since $\tilde{\text{pad}} \neq \text{pad}$. But that means that $\text{Dec}(\tilde{c})$ outputs a non- $\perp$ value (that could potentially be useful for D) only if $\text{Hash}(\text{hk}, \hat{m}) = \text{dig} = \text{Hash}(\text{hk}, m_b)$, resulting in a collision!
- If $\tilde{c}' \neq c'$, then the underlying values $(\tilde{s}, \tilde{\text{hk}}, \tilde{\text{dig}})$ should be “unrelated” to $(s, \text{hk}, \text{dig})$ —and hence m_b —if the underlying TLP is non-malleable. So, we would hope to show that this

means $\tilde{m} = G(\tilde{s}) \oplus \tilde{\text{pad}}$ is also unrelated to m_b . Formalizing this intuition is a bit tricky, and it turns out that non-malleability alone will not suffice for the underlying TLP.

To analyze the second case above, let's examine the non-malleability game in a bit more detail.

- Non-Malleability Experiment for (Enc, Dec) :

1. A code $c = (c', \text{pad}) \leftarrow \text{Enc}(m_b)$ is sampled for m_b .
 - Recall that $c' \leftarrow \text{Enc}'(s, \text{hk}, \text{dig})$ for $\text{dig} = \text{Hash}(\text{hk}, m_b)$ and $\text{pad} = G(s) \oplus m_b$.
2. The maunder outputs a tampered codeword $\tilde{c} = (\tilde{c}', \tilde{\text{pad}}) \leftarrow M(c)$.
3. The MIM experiment outputs $\tilde{m} = \text{Dec}(\tilde{c})$ as long as $\tilde{c} \neq c$.
 - Recall that $\text{Dec}(\tilde{c}', \tilde{\text{pad}})$ decodes $(\tilde{s}, \tilde{\text{hk}}, \tilde{\text{dig}}) = \text{Dec}'(\tilde{c}')$, computes a candidate message $\hat{m} = G(\tilde{s}) \oplus \tilde{\text{pad}}$, and outputs $\tilde{m} = \hat{m}$ if $\tilde{\text{dig}} = \text{Hash}(\tilde{\text{hk}}, \hat{m})$.
 - If $\tilde{c}' = c'$, we argued that this results in a collision. So, it must be the case that $\tilde{c}' \neq c'$.
4. The distinguisher predicts a bit $b' = D(\tilde{m})$ and wins if $b' = b$.

To formalize the intuition that $\tilde{m}$ should be “unrelated” to $(s, \text{hk}, \text{dig})$, we need to argue that the output of the non-malleability experiment doesn't significantly change in a hybrid experiment where we instead sample $c' \leftarrow \text{Enc}'(0^{|s|+|\text{hk}|+|\text{dig}|})$. This requires that the non-malleability experiment is “hiding” for Enc' , but this can't be the case since the experiment has to compute $\text{Dec}(\tilde{c})$! This is where CCA-hiding comes to save the day; it guarantees that hiding of Enc' still holds even if you can compute $\text{Dec}'(\tilde{c}')$ on $\tilde{c}' \neq c'$. Furthermore, we've already argued this is the only interesting case based on the collision-resistance of Hash .

Now, after making c' independent of s and m_b , we can use the pseudo-randomness of G to move to a hybrid where $\text{pad} \leftarrow \{0, 1\}^{|m_b|}$ is uniformly random and independent of m_b . The resulting experiment is independent of b , so D has no advantage for predicting b .

We note that we can extend the above argument to also capture the setting where M might make additional adaptive queries to $\text{Dec}(\cdot)$, but we still require that the distinguisher D is depth bounded. This game corresponds exactly to the CCA-hiding security game, which is why the above construction works for general CCA-hiding.

2.1.1 An Explicit Mauling Attack

One might assume that the above TLP is already non-malleable, at least under some sufficient non-malleability property of the underlying TLP. On the contrary, this is not just an issue in the analysis; there is an explicit mauling attack! The only gap in the argument above is that we assumed the distinguisher D in the non-malleability experiment ran in bounded depth $\ll t$. So, any attack on non-malleability must leverage the additional power of a polynomial-time distinguisher D that can run in depth $\geq t$. At a very high level, we will do this by having the bounded depth maunder somehow “pass useful information” to D that can be used in an attack.

Recall that $\text{Enc}(m) := (c', \text{pad})$ for $c' \leftarrow \text{Enc}'(s, \text{hk}, \text{Hash}(\text{hk}, m))$. We observe that c' is much shorter than m , but a polynomial-time D can easily distinguish c' generated using m_0 versus m_1 . Namely, D can compute $(s, \text{hk}, \text{dig}) = \text{Dec}'(c')$ and check if dig is equal to either $\text{Hash}(\text{hk}, m_0)$ or $\text{Hash}(\text{hk}, m_1)$. To turn this into an actual mauling attack, note that a maunder M on input $c = (c', \text{pad})$ can “pass” c' to D by outputting $\tilde{c} = \text{Enc}(c')$ (padding c' to the appropriate length).

By definition of the non-malleability experiment, the distinguisher D then receives c' as input, which it can easily distinguish as described above.

2.2 Non-Malleability from CCA-Hiding in the AI-ROM

We next give a compiler for any CCA-hiding TLP $(\text{Enc}', \text{Dec}')$ in the random oracle model that avoids all such mauling attacks, resulting in a non-malleable TLP (Enc, Dec) . In the random oracle model, we give all relevant parties (sender, receiver, and attackers) oracle access to a common random function $\mathcal{O}$. When proving security, we allow the mauler M and distinguisher D to depend *arbitrarily* on the random oracle, modelled via the auxiliary-input random oracle model (AI-ROM) due to Unruh [Unr07]. This captures the realistic scenario where we instantiate $\mathcal{O}$ with a fixed function like SHA-3, and we give the attacker arbitrary pre-processing time on the function to compute an advice string for the online mauler and distinguisher.

The above mauling attack used the fact that a small part of the TLP can be distinguished in polynomial-time, so could be passed to D via an encoding. Our goal now is to try to rule out all such attacks in (Enc, Dec) . First, we want to make sure everything encoded in c' itself hides m against polynomial-time D . So, we conceptually replace the digest $\text{dig} = \text{Hash}(\text{hk}, m)$ with a succinct, hiding commitment to m (using $\mathcal{O}$). The second issue is that, given s , any bit of $\text{pad} = \mathcal{G}(s) \oplus m$ can be unmasked leading to a mauling attack. We fix this by requiring “knowledge” of an additional random string r in order to unmask pad (using the extraction properties of $\mathcal{O}$). Intuitively, we will make r long enough to ensure that, information theoretically, it cannot be “passed” to D . Since the decoder will also need to know r , this will slightly increase the length of our encoding.

Construction. To encode a message m , we first encode a seed s with a hash key hk and a succinct, hiding commitment to m via $\mathcal{O}(\text{hk}||m||\rho)$ using randomness ρ . Next, we give out a random string $r \leftarrow \{0, 1\}^{|m|}$ that is the same length as m . Finally, we use $\mathcal{O}(s||r)$ to mask the message m along with the commitment randomness ρ . Altogether, our new construction is

$$\text{Enc}^{\mathcal{O}}(m) := (c' \leftarrow \text{Enc}'(s, \text{hk}, \mathcal{O}(\text{hk}||m||\rho)), r \leftarrow \{0, 1\}^{|m|+\lambda}, \text{pad} = \mathcal{O}(s||r) \oplus (m||\rho)).$$

To decode, you can first decode $(s, \text{hk}, \text{com})$ from c' , unmask pad to get a candidate message and randomness $(\hat{m}||\hat{\rho}) = \text{pad} \oplus \mathcal{O}(s||r)$, and output $m := \hat{m}$ if $\text{com} = \mathcal{O}(\text{hk}||\hat{m}||\hat{\rho})$. The construction is rate-1/2 since it includes r which is roughly the same length as the message m , and otherwise the additional overheads depend only on the security parameter λ and are independent of $|m|$.

Security analysis. Our goal is to rule out all possible mauling attacks. Let $c = (c', r, \text{pad})$ denote the challenge TLP, and $\tilde{c} = (\tilde{c}', \tilde{r}, \tilde{\text{pad}})$ denote the mauled TLP. Then every mauling attack must fall into one of the following two cases:

- First, if $(\tilde{c}', \tilde{r}) = (c', r)$, it again must be the case that $\tilde{\text{pad}} \neq \text{pad}$ if the whole TLP wasn't copied. But when c' and r are unchanged, the mauled TLP must use the same values s and r to unmask $\tilde{\text{pad}} \neq \text{pad}$. So, when decoding $\tilde{c}$, this results in different candidate message and randomness $(\hat{m}||\hat{\rho}) = \mathcal{O}(s||r) \oplus \tilde{\text{pad}}$ compared to the m and ρ used to generate the challenge TLP c . It follows that if $\tilde{c}$ is a valid encoding, this results in a collision on $\mathcal{O}(\text{hk}||\cdot)$ since $\mathcal{O}(\text{hk}||m||\rho) = \text{com} = \mathcal{O}(\text{hk}||\hat{m}||\hat{\rho})$, which is unlikely to occur.

- Otherwise, it must be the case that either $\tilde{c}' \neq c'$ or $\tilde{r} \neq r$, meaning the value $\mathcal{O}(\tilde{s}||\tilde{r})$ used to mask the mauled message is likely to be different when decoding $\tilde{c}$ compared to the challenge TLP c . Our goal in this case is to argue that the original value $\mathcal{O}(s||r)$ used to mask $(m_b||\rho)$ can be lazily sampled independent from the rest of the non-malleability experiment. If this were the case, $\text{pad} = \mathcal{O}(s||r) \oplus (m_b||\rho)$ would reveal no information about m_b or ρ , meaning that the commitment $\text{com} = \mathcal{O}(\text{hk}||m_b||\rho)$ also reveals no information about m_b , and we'd be done!

As stated above, our goal is to argue that the output of $\mathcal{O}(s||r)$ can be sampled lazily and is independent of the rest of the experiment, meaning the rest of the experiment never queries $\mathcal{O}$ on $(s||r)$. To help argue this, let's take a closer look at how the non-malleability experiment queries $\mathcal{O}$ on such values.

- Non-Malleability Experiment for $(\text{Enc}^\mathcal{O}, \text{Dec}^\mathcal{O})$:
 1. A code $c \leftarrow \text{Enc}^\mathcal{O}(m_b)$ is sampled for m_b .
 2. The mauler outputs a tampered codeword $\tilde{c} = (\tilde{c}', \tilde{r}, \tilde{\text{pad}}) \leftarrow M^\mathcal{O}(c)$.
 3. The MIM experiment outputs $\tilde{m} = \text{Dec}^\mathcal{O}(\tilde{c})$ as long as $\tilde{c} \neq c$.
 - If $\tilde{c}' \neq c'$, compute $\tilde{m}$ by querying $\mathcal{O}(\tilde{s}||\tilde{r})$ for $\tilde{s}$ output by $\text{Dec}'(\tilde{c}')$.
 - If $\tilde{c}' = c'$ but $\tilde{r} \neq r$, compute $\tilde{m}$ by querying $\mathcal{O}(s||\tilde{r})$.
 4. The distinguisher predicts a bit $b' = D^\mathcal{O}(\tilde{m})$ and wins if $b' = b$.

To argue that the experiment doesn't query $\mathcal{O}(s||r)$ computed by $\text{Enc}^\mathcal{O}(m_b)$, we separately argue that $M^\mathcal{O}$, $\text{Dec}^\mathcal{O}$, and $D^\mathcal{O}$ are unlikely to query $\mathcal{O}(s||r)$. At a very high level, this is because

- $M^\mathcal{O}$ and $\text{Dec}^\mathcal{O}$ have incomplete information on s , and
- $D^\mathcal{O}$ has incomplete information on r .

To formalize the first point, we rely on the CCA-hiding of $(\text{Enc}', \text{Dec}')$. Namely, steps 2. and 3. in the case $\tilde{c}' \neq c'$ above can be computed in bounded depth $\ll t$ given the challenge TLP c using $M^\mathcal{O}$ and given oracle access to $\text{Dec}'(\tilde{c}')$. So, if M or Dec in this case ever query $\mathcal{O}$ at some value $(s||\cdot)$ for the random s underlying c_{tlp} , this must violate CCA-hiding of $(\text{Enc}', \text{Dec}')$. For the case $\tilde{c}' = c'$, note that this step only ever queries $\mathcal{O}(s||\tilde{r})$ for $\tilde{r} \neq r$, so it also never queries $\mathcal{O}(s||r)$.

To formalize the second point, note that $D^\mathcal{O}$ only receives a string $\tilde{m}$ of length $|m_b|$ as input and makes polynomially many queries to $\mathcal{O}$. This means that the set of all possible queries for which $D^\mathcal{O}$ might have some information on is at most $2^{|m_b|} \cdot \text{poly}(\lambda)$. However, since we chose $|r| = |m_b| + \lambda$, r has $|m_b| + \lambda$ bits of entropy, implying that $D^\mathcal{O}$ is very unlikely to make any query of the form $(\cdot||r)$. Note that this argument is information theoretic, and only makes use of the fact that the input to D is bounded, limiting the amount of information that can be passed to D . Indeed, this argument holds for any polynomial-time D that may run in depth $\geq t$.

As mentioned above, it follows that $\mathcal{O}(s||r)$ can be replaced with a uniformly random string in the non-malleability experiment, so pad is a uniformly random string independent of the rest of the experiment. Then, since $\text{com} = \mathcal{O}(\text{hk}||m_b||\rho)$ hides m for random ρ , the experiment is independent of m_b , so even a polynomial-time distinguisher D has no advantage for predicting b .

Extension to functional non-malleability. In Section 7, we additionally show that the construction above satisfies *functional* non-malleability, where the mauler may output a sequence of codes $\tilde{c}_1, \dots, \tilde{c}_k$ with underlying messages $\tilde{m}_1, \dots, \tilde{m}_k$. Then, we show that the output of any function $y = f(\tilde{m}_1, \dots, \tilde{m}_k)$ doesn't depend on the input message m to the experiment as long as $|r|$ is as long as the length of the functions output $|y|$. Furthermore, the analysis above holds for *any* (potentially unbounded) function f that doesn't query $\mathcal{O}$, or for bounded depth functions f that may query $\mathcal{O}$.

Random oracle uninstantiability. In Section 7.1, we show that if the underlying TLP in the compiler satisfies certain “fully homomorphic” properties, then the resulting TLP is malleable for any concrete instantiation H of the oracle $\mathcal{O}$.⁴ At a high level, this is because our argument above relies on the extractability of $\mathcal{O}$; you have to “know” s to compute $\mathcal{O}(s||r)$. However, we show a concrete attack in case you can homomorphically evaluate $H(s||r)$ under the inner TLP for some concrete instantiation H for $\mathcal{O}$. Still, for most natural TLP candidates, the resulting TLP may still be non-malleable, and we also view this as an important first step towards a secure construction in the plain model.

2.3 Rate-1 Non-Malleable Code in the AI-ROM

We next describe a direct construction of a non-malleable code that achieves rate-1 in the AI-ROM compared to the rate-1/2 non-malleable TLP described in Section 2.2. Both constructions are non-malleable for the same class of bounded $\ll t$ polynomial-depth tampering, but the code has “slow” encoding that runs in depth $\geq t$ (recall TLPs require encoding time $\ll t$).

For starters, we observe that slow encoding alone already thwarts the mauling attack described in Section 2.1 above. In the attack, the mauling adversary M on input a challenge code c output a tampered code $\tilde{c} \leftarrow \text{Enc}(c')$ where c' was some small part of c that could be distinguished in polynomial-time. So, if the mauler can't compute $\text{Enc}(c')$ in bounded depth, this particular attack isn't possible. Indeed, the recent non-malleable code of [BSS24] for bounded polynomial-size tampering achieves rate-1 by relying on a slow encoding function. However, their construction only achieves inverse polynomial indistinguishability, so it is natural to wonder if it is possible to build a “cryptographic” rate-1 non-malleable code that achieves negligible security.

Construction. The blueprint of our construction is similar to our previous compilers and relies on a random oracle $\mathcal{O}$. Rather than encode the seed s , however, we simply give s in the clear. We then use the t -fold iteration of $\mathcal{O}$ on input s , which we denote by $\mathcal{O}^{(t)}(s)$, to mask the message m . We note that an encoding consisting of s and $\text{pad} = \mathcal{O}^{(t)}(s) \oplus m$ already suffices to hide m against bounded depth attackers, but is clearly malleable by just adding 1 to pad .

To avoid this simple attack, we again rely on a succinct, hiding commitment to bind the mauler to the seed s and message m . We also mask the commitment randomness ρ and append a commitment $\text{com} = \mathcal{O}(s||m||\rho)$ to the encoding. In sum, a code consists of the seed s , a one-time pad $\text{pad} = \mathcal{O}^{(t)}(s) \oplus (m||\rho)$, and a commitment $\text{com} = \mathcal{O}(s||m||\rho)$. Unfortunately, this construction is *still* malleable; it suffers the same mauling attack from Section 2.1. While it is hard for a depth $\ll t$ mauler M to compute $\mathcal{O}^{(t)}(s)$ for a random s , it can hardcode pairs $(s, \mathcal{O}^{(t)}(s))$ in its non-uniform

⁴Somewhat surprisingly, a TLP can satisfy both CCA-hiding while also being fully homomorphic as we define in Definition 7.13.

advice, letting it generate a valid encoding $\tilde{c} = \text{Enc}(m)$ for any message m of its choice. Combined with the fact that the seed s and first bit of pad reveals part of the message to polynomial-time distinguishers, this gives a valid mauling attack.

Our final construction modifies the above approach very slightly but significantly. We add $\mathcal{O}^{(t)}(m)$ to the commitment, so $\text{com} = \mathcal{O}(s||m||\mathcal{O}^{(t)}(m)||\rho)$. This gives our code,

$$\text{Enc}^{\mathcal{O}}(m) := (s, \text{pad} = \mathcal{O}^{(t)}(s) \oplus (m||\rho), \text{com} = \mathcal{O}(s||m||\mathcal{O}^{(t)}(m)||\rho)).$$

To decode, you use the given seed s to unmask pad and get a candidate message and randomness $(\hat{m}||\rho) = \mathcal{O}^{(t)}(s) \oplus \text{pad}$. The decoding algorithm then outputs $m := \hat{m}$ if $\text{com} = \mathcal{O}(s||\hat{m}||\mathcal{O}^{(t)}(\hat{m})||\rho)$. The construction is rate-1 since s , ρ , and com can all have length independent of $|m|$.

We note that the slight modification above is inspired by the notion of a “hard-to-sample” function from [SS24] used in the rate-1 non-malleable code of [BSS24]. At a very high level, it ensures that any maunder M can only compute $\text{Enc}(m)$ for a bounded set of messages m where the pair $(m, \mathcal{O}^{(t)}(m))$ is already hardcoded in its non-uniform advice. This limits the potential messages that can be “passed” to a distinguisher D , without relying on a long random string r as in the construction of Section 2.2.

Security analysis. Our high level goal is to argue that the output of $\mathcal{O}^{(t)}(s)$ can be lazily sampled independent to the rest of the non-malleability experiment for message m_b . If we can show this, we can switch to a hybrid experiment where pad is uniformly random and independent from m_b and ρ . But this implies that com also hides m_b , so any distinguisher D would have no advantage at guessing $\text{Enc}(m_b)$ for a random bit b . With this in mind, let’s take a closer look at the non-malleability experiment for $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$.

- Non-Malleability Experiment for $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$:
 1. A code $c = (s, \text{pad}, \text{com}) \leftarrow \text{Enc}^{\mathcal{O}}(m_b)$ is sampled for m_b .
 2. The maunder outputs a tampered codeword $\tilde{c} = (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{O}}(c)$.
 3. The MIM experiment outputs $\tilde{m} = \text{Dec}^{\mathcal{O}}(\tilde{c})$ as long as $\tilde{c} \neq c$.
 - Note that $\tilde{m}$ is derived by unmasking $\tilde{\text{pad}}$ with $\mathcal{O}^{(t)}(\tilde{s})$.
 4. The distinguisher predicts a bit $b' = D^{\mathcal{O}}(\tilde{m})$ and wins if $b' = b$.

To argue that the experiment doesn’t query the output of $\mathcal{O}^{(t)}(s)$ computed by $\text{Enc}^{\mathcal{O}}(m_b)$, we separately argue that $M^{\mathcal{O}}$, $\text{Dec}^{\mathcal{O}}$, and $D^{\mathcal{O}}$ are unlikely to compute such a query.

- First, $M^{\mathcal{O}}$ runs in depth $\ll t$, so the only way it could possibly compute the output of $\mathcal{O}^{(t)}(s)$ is if it already had hardcoded advice about s . This is very unlikely, however, since s is randomly chosen in the experiment independent of $M^{\mathcal{O}}$.
- For $\text{Dec}^{\mathcal{O}}$, we consider two separate cases:
 - If $\tilde{s} \neq s$, we claim that the path of queries for $\mathcal{O}^{(t)}(\tilde{s})$ is very unlikely to collide with the path for $\mathcal{O}^{(t)}(s)$. For starters, we can domain separate the initial query to ensure that $\tilde{s}$ is not already in the path of s . It follows that the values on the path queried by $\mathcal{O}^{(t)}(\tilde{s})$ are all random and independent of $M^{\mathcal{O}}$, so $M^{\mathcal{O}}$ is very unlikely to have any hardcoded information on $\mathcal{O}$ to come up with such a $\tilde{s}$.

- If $\tilde{s} = s$, we observe that the experiment can derive the same value for $\tilde{m}$ without computing $\mathcal{O}^{(t)}(s)$ at all! Formally, we switch to a hybrid where we use the facts that $\text{pad} = \mathcal{O}^{(t)}(s) \oplus (m_b || \rho)$ and $\tilde{\text{pad}} = \mathcal{O}^{(t)}(s) \oplus (\tilde{m} || \tilde{\rho})$ to instead derive $\tilde{m}$ from $\tilde{\text{pad}} \oplus \text{pad} \oplus m_b$ since the $\mathcal{O}^{(t)}(s)$ terms cancel out. So the experiment doesn't need to make any such queries to $\mathcal{O}$ in this case.
- Finally, for $D^\mathcal{O}$, we leverage the fact that pairs $(m, \mathcal{O}^{(t)}(m))$ are “hard-to-sample” by any $M^\mathcal{O}$ running in depth $\ll t$. Intuitively, this means that $M^\mathcal{O}$ can only compute such pairs for a bounded, polynomial set of messages $M^\mathcal{O}$. Since $M^\mathcal{O}$ must compute $\text{com} = \mathcal{O}(\tilde{s} || \tilde{m} || \mathcal{O}^{(t)}(\tilde{m}) || \tilde{\rho})$ for any code $\tilde{c}$ it outputs, it must already “know” the underlying message $\tilde{m}$ (using the extractability of $\mathcal{O}$). It follows that $M^\mathcal{O}$ can only “pass” a polynomial-size set of messages to $D^\mathcal{O}$, who runs in polynomial time. This means there is only a polynomial set of values that $D^\mathcal{O}$ might have information on, so $D^\mathcal{O}$ is very unlikely to query the output of $\mathcal{O}^{(t)}(s)$ for a random s .

As such, the output of $\mathcal{O}^{(t)}(s)$ can be lazily sampled and independent of the rest of the experiment. We argued above that this suffices to guarantee that $D^\mathcal{O}$ has no advantage at predicting the challenge bit b , ruling out all possible mauling attacks.

On “hard-to-sample” functions. The new random oracle-based techniques in the construction and analysis above are inspired by the notion of a “hard-to-sample” function from [SS24] and used by [BSS24]. In their work, they construct and use hard-to-sample functions in a complexity-theoretic setting that only achieves inverse-polynomial security. Here, we implicitly use such functions in our construction based on random oracles. We think it is a fascinating direction for future work to build cryptographically secure hard-to-sample functions in the plain model, and is one possible avenue towards constructing rate-1 non-malleable codes in the plain model with negligible security. However, we note that our proof above additionally relies on the extractability property of the random oracle when computing com .

3 Preliminaries

We let $\mathbb{N} = \{1, 2, 3, \dots\}$ denote the set of natural numbers, and for any $n \in \mathbb{N}$, we write $[n]$ to denote the set $[n] = \{1, \dots, n\}$. For integers $a, b \in \mathbb{Z}$ with $a \leq b$, we write $[a, b]$ to denote the set $\{a, a+1, \dots, b\}$. For a set Σ , referred to as the *alphabet*, we denote Σ^* the set of strings consisting of 0 or more elements from Σ . We let Σ^n denote the set of n -character strings from Σ and $\Sigma^{\leq n}$ the set of string of length at most n . For a string $s \in \Sigma^*$, we let $|s|$ denote the length of s . For any string $s \in \Sigma^*$ and $i \in [|s|]$, let $s[i]$ denote the i th character of s . For $i \notin [|s|]$, we assume that $s[i]$ always returns a special character $\perp$. Unless specified otherwise, we assume that a string s is defined over the binary alphabet $\{0, 1\}$.

We write $\lambda \in \mathbb{N}$ to denote the security parameter. We say that a function $p: \mathbb{N} \rightarrow \mathbb{N}$ is in the set $\text{poly}(\lambda)$ and is *polynomially-bounded* if there exists a constant c and an index $i \in \mathbb{N}$ such that $p(\lambda) \leq \lambda^c$ for all $\lambda \geq i$. For notational convenience when referring to multiple inputs, we define $\text{poly}(a, b, \dots)$ as $\text{poly}(a + b + \dots)$. We say that a function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}$ is negligible if for every constant $c > 0$ there exists $i \in \mathbb{N}$ such that $\text{negl}(\lambda) \leq \lambda^{-c}$ for all $\lambda \geq i$. We similarly define $\text{negl}(a, b, \dots)$ as $\text{negl}(a + b + \dots)$.

We use PPT to denote the acronym *probabilistic, polynomial time*. A uniform algorithm A is a RAM program with a fixed (constant-size) description length. A non-uniform algorithm A consists of a sequence of algorithms $\{A_\lambda\}_{\lambda \in \mathbb{N}}$, one for each security parameter λ ; we assume for simplicity that A_λ always receives 1^λ as its first input. When the security parameter is clear from context, we may write $A(\cdot)$ instead of $A_\lambda(\cdot)$ for simplicity. A non-uniform PPT algorithm is one where the description size of A_λ as a function of λ is in $\text{poly}(\lambda)$. We use $\text{size}(A_\lambda)$ to denote that running-time or size of the algorithm A_λ , and $\text{depth}(A_\lambda)$ to denote the *parallel* running time of depth of the algorithm A_λ .

For a distribution X , we write $x \leftarrow X$ to denote the process of sampling a value x from the distribution X . For a set $\mathcal{X}$, we use $x \leftarrow \mathcal{X}$ to denote the process of sampling a value x from the uniform distribution over $\mathcal{X}$. We use $x = A(\cdot)$ to denote the output of a deterministic algorithm and $x \leftarrow A(\cdot)$ to denote the output of a randomized algorithm. For a randomized algorithm, we write $x = A(\cdot; r)$ to denote the deterministic output given sequential access to the random coins r . We write $x := y$ to denote the assignment of value y to x . For a distribution D , we define $\text{Supp}(D)$ to denote the support of the distribution D .

3.1 Psuedo-Random Generators

We give the notion of a pseudo-random generator (PRG) that we use in this work.

Definition 3.1 (Pseudo-Random Generator). Let $\beta: \mathbb{N} \rightarrow \mathbb{N}$. A β -secure *pseudo-random generator* is given by an algorithm G with the following syntax:

- $r = G(1^\ell, s)$: A deterministic algorithm that on input a length 1^ℓ and a seed $s \in \{0, 1\}^\lambda$ outputs a value $r \in \{0, 1\}^\ell$.

We require that G runs in polynomial-time and satisfies the following security property:

- **β -Pseudo-randomness**: For all non-uniform $\text{poly}(\lambda, \beta(\lambda))$ -time distinguishers D and functions $\ell \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} s \leftarrow \{0, 1\}^\lambda \\ r_0 \leftarrow G(1^{\ell(\lambda)}, s) \\ r_1 \leftarrow \{0, 1\}^{\ell(\lambda)} : b' = b \\ b \leftarrow \{0, 1\} \\ b' \leftarrow D(r_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda, \beta(\lambda)).$$

When a PRG G is β -secure for $\beta(\lambda) = \lambda$, we simply say it is a (polynomially-secure) pseudo-random generator.

3.2 Collision-Resistant Hash Functions

We give the notion of a (keyed) collision-resistant hash functions (CRH) that we use in this work. We emphasize that our definition below allows for arbitrary length input and produces a fixed size digest, but it is well known that this is implied by any compressing CRH with fixed input length [Mer87, Mer89, Dam89].

Definition 3.2 (Collision-Resistant Hash Function). Let $\beta: \mathbb{N} \rightarrow \mathbb{N}$. A β -secure *collision-resistant hash function* is given by an algorithm Hash with the following syntax:

- $\text{dig} = \text{Hash}(\text{hk}, x)$: A deterministic algorithm that on input a hash key $\text{hk} \in \{0, 1\}^\lambda$ and string $x \in \{0, 1\}^*$, outputs a digest $\text{dig} \in \{0, 1\}^\lambda$.

We require that Hash runs in polynomial-time and satisfies the following security property:

- **β -Collision Resistance:** For all non-uniform $\text{poly}(\lambda, \beta(\lambda))$ -time adversaries A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} \text{hk} \leftarrow \{0, 1\}^\lambda \\ (x, x') \leftarrow A(\text{hk}) \end{array} : \begin{array}{l} x \neq x' \\ \text{Hash}(\text{hk}, x) = \text{Hash}(\text{hk}, x') \end{array} \right] \leq \text{negl}(\lambda, \beta(\lambda)).$$

When a CRH Hash is β -secure for $\beta(\lambda) = \lambda$, we simply say it is a (polynomially-secure) collision-resistant hash function.

3.3 Auxiliary-Input Random Oracle Model

Compared to the “standard” random oracle model of Bellare and Rogaway [BR93], the AI-ROM of Unruh [Unr07] more closely captures how “random oracles” are used in practice. Namely, all parties in a protocol may make use of a fixed cryptographic hash function (like SHA-3), and we require the *de facto* notion of security against non-uniform polynomial-time attackers. In the AI-ROM, we model the hash function as if it were a truly random function that all parties have oracle access to, and we first give the adversary an *unbounded* pre-processing phase on the function that outputs a polynomial-size non-uniform advice string for the online attacker. In this sense, the AI-ROM captures a random oracle in the setting we consider where there is no adversary-independent trusted setup.

More formally, to model protocols in the ROM or AI-ROM, we give all relevant parties oracle access to a randomly sampled function $\mathcal{O}: \{0, 1\}^* \rightarrow \{0, 1\}^*$. Formally, we represent $\mathcal{O}$ with a collection of functions $f_a: \{0, 1\}^a \rightarrow \{0, 1\}^{2^a}$ for each input length $a \in \mathbb{N}$ and write $\mathcal{O}(1^\ell, x)$ to represent the first ℓ bits of $f_{|x|}(x)$. For inputs of length a , 2^a output bits suffices for any exponentially efficient application, but we note that this choice is without loss of generality.⁵ We denote RF to be the set of all sequences of functions $\{f_a: \{0, 1\}^a \rightarrow \{0, 1\}^{2^a}\}$, so $\mathcal{O} \leftarrow \text{RF}$ indicates a randomly sampled function. A partial assignment F is a subset of $\{0, 1\}^a \times \{0, 1\}^b$ for any $a \in \mathbb{N}$ and $b \leq 2^a$ representing partial input/ output pairs. We denote by $\text{RF}[F]$ the collections of functions that are consistent with F , so $f \in \text{RF}[F]$ if for all $(x, y) \in F$, $f(x)$ is consistent with y .

One of the main benefits of the plain ROM is that you can argue security in the setting where the outputs for $\mathcal{O}$ are sampled “lazily.” The following lemma, adapted from Unruh [Unr07], shows how we can use similar lazy sampling techniques in AI-ROM without significant loss in security. At a high level, it allows us to “switch” a random $\mathcal{O}$ (that the attacker may depend arbitrarily on) with another oracle $\mathcal{P}$ that is lazily sampled on almost all points. In other words, the adversary cannot distinguish the output of $\mathcal{O}$ from $\mathcal{P}$ on a random query.

Lemma 3.3 (Pre-Sampling in the AI-ROM [Unr07]). *For any $q, f: \mathbb{N} \rightarrow \mathbb{N}$ and non-uniform algorithms $Z = \{Z_\lambda\}_{\lambda \in \mathbb{N}}$ that output oracle machines that make at most $q(\lambda)$ oracle queries, there*

⁵For any bound β on the output length, such an $\mathcal{O}$ can be constructed from a sequence of 1-bit functions $\{f'_a: [\beta] \times \{0, 1\}^a \rightarrow \{0, 1\} : a \in \mathbb{N}\}$ by defining $\mathcal{O}(1^\ell, x) = f'_{\log \beta + a}(1, x) || \dots || f'_{\log \beta + a}(\ell, x)$ to get $\ell \leq \beta$ bits of output.

exists a non-uniform algorithm $\text{Sam} = \{\text{Sam}_\lambda\}_{\lambda \in \mathbb{N}}$ that outputs partial assignments of size $f(\lambda)$ such that for any distinguisher D and $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} b \leftarrow \{0, 1\} \\ \mathcal{O}_0 \leftarrow \text{RF} \\ A = Z_\lambda(\mathcal{O}_0) \\ F = \text{Sam}_\lambda(A) \quad : \quad b' = b \\ \mathcal{O}_1 \leftarrow \text{RF}[F] \\ y \leftarrow A^{\mathcal{O}_b} \\ b' \leftarrow D(y) \end{array} \right] \leq \frac{1}{2} + \left(\frac{(q(\lambda))^2}{2 \cdot f(\lambda)} \right)^{1/2}$$

4 Encoding Schemes

In this section, we define a general framework for encoding schemes. In Section 4.1, we define the various security properties we consider for these schemes. Then we show how this framework captures time-lock puzzles and bounded polynomial-depth tampering in Section 4.2.

Definition 4.1 (Encoding Schemes). An encoding scheme consists of a pair of algorithms (Enc, Dec) with the following syntax:

- $c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m)$: A randomized algorithm that on input a security parameter 1^λ , auxiliary information $\text{aux} \in \{0, 1\}^*$, and a message m , outputs an encoding $c \in \{0, 1\}^*$. We implicitly assume that c specifies 1^λ , aux , and $|m|$.
- $m = \text{Dec}(c)$: A deterministic algorithm that on input a (possible invalid) encoding $c \in \{0, 1\}^*$ outputs a message $m \in \{0, 1\}^*$.

We require that (Enc, Dec) additionally satisfies the following property:

- **Correctness:** For any $\lambda \in \mathbb{N}$, $\text{aux}, m \in \{0, 1\}^*$, it holds that $m = \text{Dec}(\text{Enc}(1^\lambda, \text{aux}, m))$.

One measure for the efficiency of an encoding scheme is its rate, which captures the length of an encoding relative to the underlying message.

Definition 4.2 (Rate of an Encoding). Let $\alpha: \mathbb{N}^3 \rightarrow \mathbb{N}$. We say that an encoding scheme (Enc, Dec) has rate α if for all $\lambda \in \mathbb{N}$, $\text{aux} \in \{0, 1\}^*$, it holds that

$$\lim_{n \rightarrow \infty} \left(\max_{m: |m|=n} \frac{n}{|\text{Enc}(1^\lambda, \text{aux}, m)|} \right) = \alpha(\lambda, |\text{aux}|, n).$$

4.1 Security of Encoding Schemes

In terms of security, we require that an encoding scheme “hides” the message with respect to some class of adversaries. We consider two-stage, non-uniform adversaries (Z, D) indexed by the security parameter. For each security parameter $\lambda \in \mathbb{N}$, the first stage of the adversary Z_λ specifies a pair of messages m_0, m_1 and auxiliary information aux for the encoding. The second stage adversary D_λ receives an encoding for either m_0 or m_1 under aux and has to guess the encoded message. A class of adversaries consists of a set of such non-uniform adversaries that satisfies some additional constraints.

Definition 4.3 (Hiding). We say that an encoding scheme (Enc, Dec) satisfies *hiding* for a class of adversaries $\mathcal{A}$ if for all $\{(Z_\lambda, D_\lambda)\}_{\lambda \in \mathbb{N}} \in \mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (m_0, m_1, \text{aux}) \leftarrow Z_\lambda \\ b \leftarrow \{0, 1\} \\ c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m_b) \\ b' \leftarrow D_\lambda(c) \end{array} : \begin{array}{l} |m_0| = |m_1| \\ \wedge b = b' \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

CCA-hiding. We next consider a special case of hiding with respect to a class of adversaries that have access to an oracle that decodes any non-challenge encodings for messages of the same length message and parameters λ, aux . We refer to this as CCA-hiding for *chosen-code attack*.⁶ For convenience, we first define a special decoding function.

Definition 4.4 (Special Decoding). Let (Enc, Dec) be an encoding scheme and $c \in \{0, 1\}^*$. We define

$$\text{Dec}_c^*(\tilde{c}) := \begin{cases} \perp & \text{if } \tilde{c} = c \text{ or } \tilde{c} \text{ is not for same } 1^\lambda, \text{aux}, |m| \text{ as } c, \text{ or} \\ \text{Dec}(\tilde{c}) & \text{otherwise.} \end{cases}$$

Definition 4.5 (CCA-Hiding). We say that an encoding scheme (Enc, Dec) satisfies *CCA-hiding* for a class of adversaries $\mathcal{A}$ if for all $\{(Z_\lambda, D_\lambda)\}_{\lambda \in \mathbb{N}} \in \mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (m_0, m_1, \text{aux}) \leftarrow Z_\lambda \\ b \leftarrow \{0, 1\} \\ c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m_b) \\ b' \leftarrow D_\lambda^{\text{Dec}_c^*(\cdot)}(c) \end{array} : \begin{array}{l} |m_0| = |m_1| \\ \wedge b = b' \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Functional non-malleability. Non-malleability [DDN91, DPW10] informally guarantees that a “meddler-in-the-middle” (MIM) adversary that receives an encoding of a message m cannot produce an encoding of a messages $\tilde{m}$ non-trivially related to m . Functional non-malleability [FKPS21] for a class of multi-input functions $\mathcal{F}$ extends this by requiring that a MIM adversary that receives an encoding of a message m cannot produce a sequence of encoded messages $\tilde{m}_1, \dots, \tilde{m}_k$ of the same length such that $f(\tilde{m}_1, \dots, \tilde{m}_k)$ is non-trivially related to m .

To formalize the above, we define the functional MIM experiment for the MIM adversary M and a particular function f .

Definition 4.6 (Functional MIM experiment). Let M be an algorithm and f be a multi-input function. For any $\lambda \in \mathbb{N}$ and $\text{aux}, m \in \{0, 1\}^*$, we define the functional MIM experiment as follows:

$$\text{mim}_{M,f}(1^\lambda, \text{aux}, m) = \left\{ \begin{array}{l} c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m) \\ (\tilde{c}_1, \dots, \tilde{c}_k) \leftarrow M(c) \\ \forall i \in [k], \tilde{m}_i := \text{Dec}_c^*(\tilde{c}_i) \end{array} : f(\tilde{m}_1, \dots, \tilde{m}_k) \right\}.$$

⁶CCA historically stands for chosen-ciphertext attack, introduced to model particular attacks against encryption schemes [NY90, RS91], and later adapted to model chosen-commitment attacks by [CLP10].

To define functional non-malleability, we consider a three-stage, non-uniform adversary (Z, M, D) . For each $\lambda \in \mathbb{N}$, the first stage of the adversary Z_λ specifies the messages m_0, m_1 , auxiliary information aux , and a function f per security parameter. The second stage is the MIM adversary M_λ that runs the functional MIM experiment above. The third stage adversary D_λ receives the output of the MIM experiment and guesses if it corresponds to m_0 or m_1 .

Definition 4.7 (Functional Non-Malleability). We say that an encoding scheme (Enc, Dec) satisfies *functional non-malleability* for a class of adversaries $\mathcal{A}$ and functions $\mathcal{F}$ if for all $\{(Z_\lambda, M_\lambda, D_\lambda)\}_{\lambda \in \mathbb{N}} \in \mathcal{A}$ there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (m_0, m_1, \text{aux}, f) \leftarrow Z_\lambda \\ b \leftarrow \{0, 1\} \\ \text{out}_b \leftarrow \text{mim}_{M_\lambda, f}(1^\lambda, \text{aux}, m_b) \\ b' \leftarrow D_\lambda(\text{out}_b) \end{array} : \begin{array}{l} b = b' \\ \wedge |m_0| = |m_1| \\ \wedge f \in \mathcal{F} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

In the above definition, the MIM adversary takes in a single encoding c “on the left” and outputs an arbitrary number of encodings $\tilde{c}_1, \dots, \tilde{c}_k$ “on the right.” Taking a single encoding rather than many encodings as input on the left is without loss of generality by a simple hybrid argument (see e.g., [LPV08] and [FKPS21] for formal statements in regards to commitments and time-lock puzzles, respectively). Allowing for an arbitrary number of encodings on the right is typically referred to as the concurrent setting, which is implicit in this functional non-malleable definition. In the $k(\cdot)$ -bounded concurrent setting, the number of encodings on the right is required to be bounded by a fixed function $k(\cdot)$. In the special case where $k = 1$, we refer to this as plain or one-to-one (functional) non-malleability.

We note that when the function class $\mathcal{F}$ is simply the identity function, this corresponds to concurrent non-malleability. Further, if one-to-one functional non-malleability holds for the identity function, this captures plain non-malleability.

4.2 Time-Lock Puzzles and Bounded Polynomial-Depth Tampering

Below, we give the definition of a time-lock puzzle in terms of encoding schemes with additional properties. To do so, we first define the class of bounded depth adversaries following the time-lock puzzle definition in [FKPS21].

Definition 4.8 (α -Gap β -Size Bounded Polynomial Depth Adversaries). A pair of non-uniform adversaries $(Z, M) = \{(Z_\lambda, M_\lambda)\}_{\lambda \in \mathbb{N}}$ is a valid α -gap β -size bounded depth adversary if Z is a non-uniform PPT algorithm that outputs a time bound t and there exists a polynomial q such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (z, t) \leftarrow Z_\lambda : \\ \alpha(\lambda, |z|) \leq t \leq q(\lambda, \beta(\lambda)) \\ \wedge \text{size}(M_\lambda) \leq q(\lambda, \beta(\lambda)) \\ \wedge \text{depth}(M_\lambda) \leq t/\alpha(\lambda, |z|) \end{array} \right] = 1.$$

We can similarly define bounded size adversaries where $\text{size}(M_\lambda)$ (and hence also $\text{depth}(M_\lambda)$) is required to be at most $t/\alpha(\lambda, |z|)$ in the expression above. Bounded depth adversaries are more expressive, so this is the primary notion we focus on in this work.

Definition 4.9 (Time-Lock Puzzles). Let $\beta: \mathbb{N} \rightarrow \mathbb{N}$. An encoding scheme (Enc, Dec) is a β -secure time-lock puzzle (TLP) if it satisfies the following additional properties:

- **Efficient Encoding:** $\text{Enc}(1^\lambda, t, m)$ takes as auxiliary input a time bound $\text{aux} = t \in \mathbb{N}$ and runs in time polynomial in the length of its inputs.
- **Timed Decoding:** There exists a polynomial q such that for all encodings $c \in \{0, 1\}^*$ specifying the time bound t , $\text{Dec}(c)$ runs in time $t \cdot q(|c|)$.
- **β -Size Bounded Polynomial Depth Hiding:** There exists a positive, monotonic polynomial α such that the hiding (Definition 4.3) holds for all α -gap β -size bounded depth adversaries (Definition 4.8).

If a TLP is β -secure for $\beta = \lambda$ we simply say that it is a time-lock puzzle and satisfies bounded depth hiding. If a TLP (Enc, Dec) satisfies bounded depth hiding with respect to a CCA adversary with oracle access to Dec_c^* as in Definition 4.5, we say that it is a CCA-hiding TLP. A functional non-malleable TLP is one that additionally satisfies the following.

Definition 4.10 (β -Size Bounded Polynomial Depth Functional Non-Malleability). Let $\beta: \mathbb{N} \rightarrow \mathbb{N}$. An encoding scheme (Enc, Dec) satisfies β -size bounded depth functional non-malleability for a class of functions $\mathcal{F}$ if there exists a positive, monotonic polynomial α such that functional non-malleability for $\mathcal{F}$ (Definition 4.7) holds for all non-uniform adversaries (Z, M, D) such that (Z, M) is an α -gap β -size bounded depth adversary (Definition 4.8) and D is a non-uniform probabilistic $\text{poly}(\lambda, \beta(\lambda))$ time algorithm.

We note that bounded polynomial depth (CCA-)hiding and functional non-malleability are defined to be preserved by reductions with fixed polynomial-depth overhead. In particular, the gap parameter α absorbs the associated overhead in depth for the depth bounded adversary. We formalize depth-preserving reductions and the associated claim in Appendix D.

5 Rate-1 Compiler for CCA-Security

In this section, we construct a rate-1 CCA-secure encoding scheme from one with inverse-polynomial rate, resulting in the following theorem.

Theorem 5.1 (Rate-1 Compiler for CCA-Secure Encoding Schemes). *Let $(\text{Enc}', \text{Dec}')$ be any encoding scheme that satisfies CCA-hiding for a subset of non-uniform PPT adversaries. For any $\text{aux} \in \{0, 1\}^*$, let $\beta(\lambda)$ be a bound on the size of $\text{Dec}'(c)$ for any $c \leftarrow \text{Enc}'(1^\lambda, \text{aux}, m)$ for $|m| \leq 3\lambda$. Assume the existence of β -secure collision-resistant hash functions. There exists a rate-1 encoding scheme (Enc, Dec) with a size-preserving (Definition D.1) straight-line, black-box reduction from the CCA-hiding of (Enc, Dec) to the CCA-hiding of $(\text{Enc}', \text{Dec}')$.*

Since the reduction in the above theorem is size-preserving, it preserves CCA-hiding for polynomial-size, bounded polynomial-depth, and bounded polynomial-size tampering classes (e.g., see Lemma D.2).

We note that our construction relies on a CRH and a PRG, which are both known to be implied by CRH. At a high level, our construction uses the inner encoding Enc' to encode a PRG seed s and a collision-resistant hash of the message m . It then expands the seed s to mask the message m . We formalize this in Construction 5.2.

Construction 5.2. Rate-1 CCA-Hiding Compiler

Ingredients: Let G be pseudo-random generator (Definition 3.1), Hash be a collision-resistant hash function (Definition 3.2), and $(\text{Enc}', \text{Dec}')$ be any encoding scheme that satisfies CCA-hiding for some class of polynomially bounded adversaries (Definitions 4.1, 4.5).

Construction: We define encoding and decoding functions (Enc, Dec) as follows:

- $c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m)$:
 1. Sample $s, \text{hk} \leftarrow \{0, 1\}^\lambda$.
 2. Compute $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{dig}))$ for $\text{dig} = \text{Hash}(\text{hk}, m)$.
 3. Set $\text{pad} = G(1^{|m|}, s) \oplus m$.
 4. Output $c := (c', \text{pad})$.
- $m = \text{Dec}(c)$:
 1. Parse $c := (c', \text{pad})$.
 2. Compute $(s, \text{hk}, \text{dig}) = \text{Dec}'(c')$.
 3. Set $\hat{m} = G(1^{|\text{pad}|}, s) \oplus \text{pad}$.
 4. Output $m := \hat{m}$ if $\text{Hash}(\text{hk}, \hat{m}) = \text{dig}$ and $m := \perp$ otherwise.

In the following lemmas, we prove that this construction satisfies correctness, rate-1, and CCA-hiding as required for the theorem.

Lemma 5.3 (Correctness). *Assuming correctness of $(\text{Enc}', \text{Dec}')$, (Enc, Dec) satisfies correctness.*

Proof. Fix $\lambda \in \mathbb{N}$ and $\text{aux}, m \in \{0, 1\}^*$. Let $c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m)$ and consider the output of $\text{Dec}(c)$. $\text{Dec}(c)$ parse c as (c', pad) and computes $(s, \text{hk}, \text{dig}) = \text{Dec}'(c')$. By correctness of $(\text{Enc}', \text{Dec}')$, it follows that $(s, \text{hk}, \text{dig})$ are the associated values from running Enc . By commutativity of $\oplus$ and since $|m| = |\text{pad}|$ by construction, it follows that $m = G(1^{|\text{pad}|}, s) \oplus \text{pad}$. Since $\text{dig} = \text{Hash}(\text{hk}, m)$ by construction in Enc , it follows that $\text{Dec}(c)$ outputs m , as required. $\square$

Lemma 5.4 (Rate-1). *(Enc, Dec) has rate 1.*

Proof. Fix any $\lambda \in \mathbb{N}$ and $\text{aux} \in \{0, 1\}^*$. An encoding of any length n message m consists of an encoding $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{dig}))$ where $|s| = |\text{hk}| = |\text{dig}| = \lambda$, a value pad where $|\text{pad}| = |m| = n$, and implicitly includes 1^λ , aux , and $|m|$, which has length $\lambda + |\text{aux}| + \log(n)$. Observe that the inputs to Enc' are all independent of n , so $|c'|$ is independent of n . It follows that, as $n \rightarrow \infty$, $(|c'|/n) \rightarrow 0$, $(|\text{pad}|/n) \rightarrow 1$, and $(\lambda + |\text{aux}| + \log(n))/n \rightarrow 0$, so

$$\lim_{n \rightarrow \infty} \left(\max_{m: |m|=n} \frac{n}{|c'| + |\text{pad}| + \lambda + |\text{aux}| + \log(n)} \right) = 1,$$

as required. $\square$

Lemma 5.5 (CCA-Hiding). *Assuming Hash and G are β -secure, there exists a polynomial-time, straight-line, black-box reduction from the CCA-hiding of (Enc, Dec) to the CCA-hiding of $(\text{Enc}', \text{Dec}')$.*

Proof. Suppose by way of contradiction that (Enc, Dec) does not satisfy CCA-hiding for a subset of the class of non-uniform PPT adversaries. Namely, there exist non-uniform PPT algorithms $(Z, D) = \{(Z_\lambda, D_\lambda)\}_{\lambda \in \mathbb{N}}$ and a polynomial p such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} b \leftarrow \{0, 1\} \\ (m_0, m_1, \text{aux}) \leftarrow Z_\lambda \\ c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m_b) \\ b' \leftarrow D_\lambda^{\text{Dec}_c^*}(\cdot)(c) \end{array} : \begin{array}{l} |m_0| = |m_1| \\ \wedge b = b' \end{array} \right] > \frac{1}{2} + \frac{1}{p(\lambda)}.$$

We assume without loss of generality that $|m_0| = |m_1|$, otherwise we could consider an adversary Z' that simply aborts when that is not the case and succeeds with the same probability. We use k to denote the number of queries that D_λ makes to Dec_c^* in the above experiment. For fixed adversaries as in the above experiment, we define the following sequence of hybrid experiments for every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$:

- $H_{b,0}(\lambda)$: In the first hybrid, we simply fix a value of b in the above CCA-hiding experiment.
 1. Compute $(m_0, m_1, \text{aux}) \leftarrow Z_\lambda$.
 2. Compute $c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m_b)$.
 3. For $i \in [k]$, run $D_\lambda(c)$ until its i th query $\tilde{c}_i$ for Dec_c^* . Compute the response $\tilde{m}_i$ as follows:
 - If $\tilde{c}_i = c$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - Otherwise, set $\tilde{m}_i := \text{Dec}(\tilde{c}_i)$.
 4. Let b' be the final output of $D_\lambda(c)$.
 5. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,1}(\lambda)$: In this hybrid, we compute the $\tilde{m}_i$ values differently depending on whether the Enc' encodings were copied or not.
 1. Compute $(m_0, m_1, \text{aux}) \leftarrow Z_\lambda$.
 2. Compute $c := (c', \text{pad}) = \text{Enc}(1^\lambda, \text{aux}, m_b)$. Namely, sample $s, \text{hk} \leftarrow \{0, 1\}^\lambda$, compute $\text{dig} = \text{Hash}(\text{hk}, m_b)$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{dig}))$, and set $\text{pad} = \text{G}(1^{|m_b|}, s) \oplus m_b$.
 3. For $i \in [k]$, run $D_\lambda(c)$ until its i th query $\tilde{c}_i = (c'_i, \text{pad}_i)$ for Dec_c^* . Compute the response $\tilde{m}_i$ as follows:
 - If $(c'_i, \text{pad}_i) = (c', \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $\text{pad}_i \neq \text{pad}$, use $(s_i, \text{hk}_i, \text{dig}_i) := (s, \text{hk}, \text{dig})$ and compute $m'_i = \text{G}(1^{|m_b|}, s) \oplus \text{pad}_i$. Set $\tilde{m}_i := m'_i$ if $\text{Hash}(\text{hk}, m'_i) = \text{dig}$ and $\perp$ otherwise.
 - Otherwise, if $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{dig}_i) = \text{Dec}'(c'_i)$ and $m_i = \text{G}(1^{|m_b|}, s_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := m_i$ if $\text{Hash}(\text{hk}_i, m_i) = \text{dig}_i$ and $\perp$ otherwise.
 4. Let b' be the final output of $D_\lambda(c)$.
 5. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,2}(\lambda)$: In this hybrid, we set $\tilde{m}_i$ to be $\perp$ whenever the inner encoding c'_i is copied but the pad value pad_i is not.

1. Compute $(m_0, m_1, \text{aux}) \leftarrow Z_\lambda$.
 2. Compute $c := (c', \text{pad}) = \text{Enc}(1^\lambda, \text{aux}, m_b)$. Namely, sample $s, \text{hk} \leftarrow \{0, 1\}^\lambda$, compute $\text{dig} = \text{Hash}(\text{hk}, m_b)$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{dig}))$, and set $\text{pad} = G(1^{|m_b|}, s) \oplus m_b$.
 3. For $i \in [k]$, run $D_\lambda(c)$ until its i th query $\tilde{c}_i = (c'_i, \text{pad}_i)$ for Dec_c^* . Compute the response $\tilde{m}_i$ as follows:
 - If $(c'_i, \text{pad}_i) = (c', \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $\text{pad}_i \neq \text{pad}$, set $\tilde{m}_i := \perp$.
 - Otherwise, if $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{dig}_i) = \text{Dec}'(c'_i)$ and $m_i = G(1^{|m_b|}, s_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := m_i$ if $\text{Hash}(\text{hk}_i, m_i) = \text{dig}_i$ and $\perp$ otherwise.
 4. Let b' be the final output of $D_\lambda(c)$.
 5. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,3}(\lambda)$: In this hybrid, we make the inner encoding independent of m_b , replacing the real seed s with 0^λ and the hash of m_b with a hash of $0^{|m_b|}$.
 1. Compute $(m_0, m_1, \text{aux}) \leftarrow Z_\lambda$.
 2. Compute $c := (c', \text{pad})$ as follows. Sample $s, \text{hk} \leftarrow \{0, 1\}^\lambda$, compute $\text{dig} = \text{Hash}(\text{hk}, 0^{|m_b|})$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (0^\lambda, \text{hk}, \text{dig}))$, and set $\text{pad} = G(1^{|m_b|}, s) \oplus m_b$.
 3. For $i \in [k]$, run $D_\lambda(c)$ until its i th query $\tilde{c}_i = (c'_i, \text{pad}_i)$ for Dec_c^* . Compute the response $\tilde{m}_i$ as follows:
 - If $(c'_i, \text{pad}_i) = (c', \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $\text{pad}_i \neq \text{pad}$, set $\tilde{m}_i := \perp$.
 - Otherwise, if $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{dig}_i) = \text{Dec}'(c'_i)$ and $m_i = G(1^{|m_b|}, s_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := m_i$ if $\text{Hash}(\text{hk}_i, m_i) = \text{dig}_i$ and $\perp$ otherwise.
 4. Let b' be the final output of $D_\lambda(c)$.
 5. Output 1 if $b' = b$ and 0 otherwise.
 - $H_{b,4}(\lambda)$: In this hybrid, we replace the value pad for the challenge encoding with a uniformly random string.
 1. Compute $(m_0, m_1, \text{aux}) \leftarrow Z_\lambda$.
 2. Compute $c := (c', \text{pad})$ as follows. Sample $\text{hk} \leftarrow \{0, 1\}^\lambda$, compute $\text{dig} = \text{Hash}(\text{hk}, 0^{|m_b|})$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (0^\lambda, \text{hk}, \text{dig}))$, and set $\text{pad} \leftarrow \{0, 1\}^{|m_b|}$.
 3. For $i \in [k]$, run $D_\lambda(c)$ until its i th query $\tilde{c}_i = (c'_i, \text{pad}_i)$ for Dec_c^* . Compute the response $\tilde{m}_i$ as follows:
 - If $(c'_i, \text{pad}_i) = (c', \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $\text{pad}_i \neq \text{pad}$, set $\tilde{m}_i := \perp$.
 - Otherwise, if $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{dig}_i) = \text{Dec}'(c'_i)$ and $m_i = G(1^{|m_b|}, s_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := m_i$ if $\text{Hash}(\text{hk}_i, m_i) = \text{dig}_i$ and $\perp$ otherwise.
 4. Let b' be the final output of $D_\lambda(c)$.
 5. Output 1 if $b' = b$ and 0 otherwise.

By assumption, it holds that for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{p(\lambda)} &< \frac{1}{2} \cdot \Pr[H_{0,0}(\lambda)] + \frac{1}{2} \cdot \Pr[H_{1,0}(\lambda)] \\ &= \frac{1}{2} \cdot \left(\Pr[H_{0,4}(\lambda)] + \sum_{i=1}^4 (\Pr[H_{0,i-1}(\lambda)] - \Pr[H_{0,i}(\lambda)]) \right) \\ &\quad + \frac{1}{2} \cdot \left(\Pr[H_{1,4}(\lambda)] + \sum_{i=1}^4 (\Pr[H_{1,i-1}(\lambda)] - \Pr[H_{1,i}(\lambda)]) \right) \end{aligned}$$

However, observe that $H_{0,4}$ and $H_{1,4}$ are identically distributed, so $\Pr[H_{0,4}(\lambda)] + \Pr[H_{1,4}(\lambda)] = 1$ for all $\lambda \in \mathbb{N}$. It follows that for infinitely many $\lambda \in \mathbb{N}$,

$$\frac{1}{p(\lambda)} < \frac{1}{2} \cdot \sum_{b \in \{0,1\}, i \in [4]} (\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)]).$$

so there exists a pair of consecutive hybrids such that for some $b \in \{0,1\}$ and $i \in [4]$,

$$\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)] > \frac{1}{4 \cdot p(\lambda)}.$$

In the claims below, we show that this contradicts security of **Hash** or **G**, or otherwise gives a reduction to the CCA-hiding of $(\text{Enc}', \text{Dec}')$. We give the below claims for $b = 0$ without loss of generality.

Claim 5.6. *For all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)]| = 0$.*

Proof. Hybrids $H_{0,0}(\lambda)$ and $H_{0,1}(\lambda)$ only differ when computing $\tilde{m}_i$ in the case when $c'_i = c'$ but $\text{pad}_i \neq \text{pad}$. In $H_{0,0}(\lambda)$ $(s_i, \text{hk}_i, \text{dig}_i)$ is computed from $\text{Dec}'(\tilde{c}_i)$. In $H_{0,1}$ it is computed by setting $(s_i, \text{hk}_i, \text{dig}_i) := (s, \text{hk}, \text{dig})$. Since $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{dig}))$ and $c_i = c'$, the two hybrids are identically distributed by the correctness of $(\text{Enc}', \text{Dec}')$. It follows that $|\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)]| = 0$. $\square$

Claim 5.7. *If $\Pr[H_{0,1}(\lambda)] - \Pr[H_{0,2}(\lambda)] > 1/(4p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then **Hash** is not β -secure.*

Proof. We construct an adversary $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$ against the collision resistance of **Hash**.

- On input $\text{hk} \leftarrow \{0,1\}^\lambda$, $A_\lambda(\text{hk})$ runs the experiment $H_{0,1}(\lambda)$ given adversaries Z_λ, D_λ and uses hk to generate $\text{dig} = \text{Hash}(\text{hk}, m_0)$ for the challenge encoding. A_λ continues running $H_{0,1}(\lambda)$ to compute query responses $\tilde{m}_i$ for each $i \in [k]$. If, at any point, the experiment computes $\tilde{m}_i \neq m_0$ such that $\text{Hash}(\text{hk}, \tilde{m}_i) = \text{dig}$, A_λ outputs the collision $(m_0, \tilde{m}_i)$. Otherwise, A_λ aborts and outputs $\perp$.

In terms of running time, note that the experiment—and hence A_λ —runs in polynomial time in the complexity of $Z_\lambda, D_\lambda, \text{Enc}'$, and Dec' . Specifically, it computes $\text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{dig}))$ and $\text{Dec}'(\tilde{c}_i)$ for parameters $1^\lambda, \text{aux}$ and message length 3λ , which takes time $\beta(\lambda)$ by assumption. It follows that A_λ runs in time $\text{poly}(\lambda, \beta(\lambda))$ as required for efficiency of the attack.

In terms of correctness, observe that hybrids $H_{0,1}(\lambda)$ and $H_{0,2}(\lambda)$ only differ when computing $\tilde{m}_i$ in the case when $c'_i = c'$ but $\text{pad}_i \neq \text{pad}$. Hybrid $H_{0,2}(\lambda)$ simply outputs $\perp$ in this case, whereas $H_{0,1}(\lambda)$ outputs $\tilde{m}_i \neq \perp$ as the output of $G(1^{|m_0|}, s) \oplus \text{pad}_i$ if $\text{Hash}(\text{hk}, \tilde{m}_i) = \text{dig}$. However, note that $\tilde{m}_i \neq m_0$ since $\tilde{m}_i = G(1^{|m_0|}, s) \oplus \text{pad}_i$, $m_0 = G(1^{|m_0|}, s) \oplus \text{pad}$, and $\text{pad}_i \neq \text{pad}$ in this case. Thus, m_b and $\tilde{m}_i$ form a valid collision under hk whenever the experiments differ. Since the experiments differ with probability at least $1/(4p(\lambda))$ by assumption, it follows that A_λ succeeds with at least this probability, contradicting collision-resistance of Hash . $\square$

Claim 5.8. *If $\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)] > 1/(4p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}', \text{Dec}')$ does not satisfy CCA-hiding based on a polynomial-time, straight-line, black-box reduction.*

Proof. We construct an adversary $\{(Z'_\lambda, D'_\lambda)\}_{\lambda \in \mathbb{N}}$ against the CCA-hiding of $(\text{Enc}', \text{Dec}')$. Observe that $\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)] > 1/(4p(\lambda))$ over the randomness of the experiments, so in particular, there must also exist a fixed string s sampled during the experiments such that the difference in probabilities is at least $1/(4p(\lambda))$. We hardcode such a value s into our construction of (Z'_λ, D'_λ) below.

- Z'_λ emulates the first part of hybrids $H_{0,2}(\lambda)$ and $H_{0,3}(\lambda)$ corresponding to generating the inputs for the inner code c' generated as a part of the challenge code c . Specifically, Z'_λ first computes $(m_0, m_1, \text{aux}) \leftarrow Z_\lambda$ and samples the common value $\text{hk} \leftarrow \{0, 1\}^\lambda$. For the first challenge message, it sets $m'_0 := (s, \text{hk}, \text{dig}_0)$ for $\text{dig}_0 = \text{Hash}(\text{hk}, m_0)$. For the second challenge message, it sets $m'_1 := (0^\lambda, \text{hk}, \text{dig}_1)$ for $\text{dig}_1 = \text{Hash}(\text{hk}, 0^{|m_0|})$. Z'_λ outputs values (m'_0, m'_1, aux) .
- On input a code $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, m'_b)$, D'_λ computes its output by emulating the remainder of the experiment $H_{0,2}(\lambda)$ using oracle access to $\text{Dec}'(\cdot)$ for inputs not equal to c' . Specifically, D'_λ first finishes computing the challenge encoding $c := (c', \text{pad})$ by setting $\text{pad} = G(1^{|m_0|}, s) \oplus m_0$. It then runs $D_\lambda(c)$ and computes the associated responses $\tilde{m}_i$ corresponding to each query $\tilde{c}_i = (c'_i, \text{pad}_i)$ that it makes to Dec'_c . Notably, it simply set $\tilde{m}_i := \perp$ unless $c'_i \neq c'$. When $c'_i \neq c'$, note that D'_λ can compute $\tilde{m}_i$ as in the experiment only using the oracle Dec' on inputs $c'_i \neq c'$. It then computes and output the bit b' output by $D_\lambda(c)$ (rather than the output of the experiment).

In terms of the running time of the reduction, note that Z'_λ runs in polynomial-time in addition to making a single black-box call to Z_λ . D'_λ also emulates a single run of D_λ via black-box access while responding to the oracle calls that D_λ makes to $\text{Dec}'_c(\cdot)$. For each such oracle call, D'_λ does some fixed polynomial amount of work (independent of the complexity of D_λ) and potentially also makes an oracle call to $(\text{Dec}')^*_{c'}(\cdot)$. It follows that the constructions of Z'_λ and D'_λ are polynomial-time and only makes straight-line and black-box use of Z_λ and D_λ , respectively.

In terms of correctness, we note that the output of D'_λ corresponds to hybrid $H_{0,2}(\lambda)$ when $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, m'_0)$, and corresponds to $H_{0,3}(\lambda)$ when $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, m'_1)$. Recall that b is the challenge bit in the overall experiment, and $b = 0$ is fixed for hybrids $H_{0,2}$ and $H_{0,3}$. Let b^* denote the challenge bit for the CCA-hiding experiment with adversary (Z'_λ, D'_λ) . So, if $b^* = 0$, the adversary wins when the experiment outputs 0 so $b' = b$, and if $b^* = 1$, the adversary wins when

the experiments output 1 so $b' \neq b$. It follows that

$$\begin{aligned}
& \Pr \left[\begin{array}{l} b^* \leftarrow \{0, 1\} \\ (m'_0, m'_1, \mathbf{aux}) \leftarrow Z'_\lambda \\ c' \leftarrow \mathbf{Enc}'(1^\lambda, \mathbf{aux}, m'_{b^*}) \\ b' \leftarrow (D'_\lambda)^{(\mathbf{Dec}')^*(\cdot)}(c') \end{array} : b^* = b' \right] \\
&= \Pr[b^* = 0] \cdot \Pr[b' = 0 \mid b^* = 0] + \Pr[b^* = 1] \cdot \Pr[b' = 1 \mid b^* = 1] \\
&= \frac{1}{2} \cdot \Pr[H_{0,2}(\lambda)] + \frac{1}{2} \cdot (1 - \Pr[H_{0,3}(\lambda)]) \\
&= \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)]) \\
&> \frac{1}{2} + \frac{1}{8 \cdot p(\lambda)},
\end{aligned}$$

for infinitely many $\lambda \in \mathbb{N}$, which contradicts the CCA-hiding of $(\mathbf{Enc}', \mathbf{Dec}')$ as required by the reduction. $\square$

Claim 5.9. *If $\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)] > 1/(4 \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $\mathbf{G}$ is not β -secure.*

Proof. We construct an adversary D'_λ against the pseudo-randomness of $\mathbf{G}$.

- D'_λ takes as input a string $r \in \{0, 1\}^{|m_0|}$ which is either uniformly random or the output of $\mathbf{G}(1^{|m_0|}, s)$ for $s \leftarrow \{0, 1\}^\lambda$. D'_λ emulates the hybrid $H_{0,3}(\lambda)$ except uses the string r to mask the message m_0 when computing $\mathbf{pad}$. Namely, it computes the challenge message $c := (c', \mathbf{pad})$ using $\mathbf{pad} = r \oplus m_0$. D'_λ then emulates the rest of $H_{0,3}(\lambda)$ and outputs the bit b' eventually output by $D_\lambda(c)$.

In terms of running time, note that the experiment—and hence D'_λ —runs in polynomial time in the complexity of Z_λ , D_λ , $\mathbf{Enc}'$, and $\mathbf{Dec}'$, which takes time $\text{poly}(\lambda, \beta(\lambda))$ by assumption. It follows that D'_λ has the required efficiency for the attack.

In terms of correctness, observe that the output of D'_λ corresponds to hybrid $H_{0,3}(\lambda)$ when $r = \mathbf{G}(1^{|m_0|}, s)$ for $s \leftarrow \{0, 1\}^\lambda$, and the output of the experiment is identically distributed to $H_{0,4}(\lambda)$ when $r \leftarrow \{0, 1\}^{|m_0|}$. Recall that b is the challenge bit in the overall experiment, and $b = 0$ is fixed for hybrids $H_{0,3}$ and $H_{0,4}$. Let b^* denote the challenge bit of the pseudo-randomness experiment with adversary D'_λ . Namely, $b^* = 0$ corresponds to a pseudo-random r value, and D'_λ wins when $b' = 0$ so $b' = b$. $b^* = 1$ corresponds to a fully random r value, and D'_λ wins when $b' = 1$

so $b' \neq b$. It follows that

$$\begin{aligned}
& \Pr \left[\begin{array}{l} s \leftarrow \{0, 1\}^\lambda \\ r_0 \leftarrow \mathbf{G}(1^{|m_0|}, s) \\ r_1 \leftarrow \{0, 1\}^{|m_0|} \\ b^* \leftarrow \{0, 1\} \\ b' \leftarrow D'_\lambda(r_{b^*}) \end{array} : b^* = b' \right] \\
&= \Pr[b^* = 0] \cdot \Pr[b' = 0 \mid b^* = 0] + \Pr[b^* = 1] \cdot \Pr[b' = 1 \mid b^* = 1] \\
&= \frac{1}{2} \cdot \Pr[H_{0,2}(\lambda)] + \frac{1}{2} \cdot (1 - \Pr[H_{0,3}(\lambda)]) \\
&= \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)]) \\
&> \frac{1}{2} + \frac{1}{8 \cdot p(\lambda)},
\end{aligned}$$

for infinitely many $\lambda \in \mathbb{N}$, which contradicts the pseudo-randomness of $\mathbf{G}$ as required. $\square$

This completes the proof of Lemma 5.5. $\square$

6 Rate-1 Non-Malleable Code in the AI-ROM

In this section, we construct a rate-1 non-malleable code for bounded polynomial-depth adversaries in the AI-ROM, from no additional assumptions.

Theorem 6.1 (Non-Malleable Code for Bounded Polynomial-Depth Tampering in the AI-ROM). *In the AI-ROM, there exists a rate-1 non-malleable code that satisfied bounded polynomial-depth one-to-one non-malleability.*

Our construction for Theorem 6.1 makes use of a common random oracle $\mathcal{O}$. We'll leverage the fact that iterating a random oracle is a inherently sequential and “hard-to-sample” function. For any $t, n \in \mathbb{N}$, we define $\mathcal{O}^{(t)}(1^n, x)$ to be the t -fold composition of the function $\mathcal{O}(1^n, \cdot)$ starting with input x , so

$$\mathcal{O}^{(t)}(1^n, x) = \underbrace{\mathcal{O}(1^n, \cdot) \circ \dots \circ \mathcal{O}(1^n, \cdot)}_{t \text{ times}}(x).$$

At a high level, for a given time bound t , our encoding algorithm gives out a seed s in the clear and uses $\mathcal{O}^{(t)}(s)$ to mask the message m with randomness ρ . It then uses $\mathcal{O}$ to produce a commitment for the relevant values s , ρ , and m as well as $\mathcal{O}^{(t)}(m)$. We formalize this in Construction 6.2.

Construction 6.2. Non-Malleable Code in the AI-ROM

Ingredients: Let $\mathcal{O}$ be a common random function (formalized in Section 3.3).

Construction: We construct encoding and decoding algorithms ($\text{Enc}^\mathcal{O}, \text{Dec}^\mathcal{O}$) as follows:

- $c \leftarrow \text{Enc}^\mathcal{O}(1^\lambda, t, m)$:
 1. Sample $s, \rho \leftarrow \{0, 1\}^\lambda$.
 2. Set $\text{pad} = y_s \oplus (m \parallel \rho)$ for $y_s = \mathcal{O}^{(t)}(1^{|m|+\lambda}, s)$.
 3. Compute $\text{com} = \mathcal{O}(1^\lambda, s \parallel \rho \parallel m \parallel y_m)$ for $y_m = \mathcal{O}^{(t)}(1^\lambda, m)$.

4. Output $c = (s, \text{pad}, \text{com})$.
 - $m = \text{Dec}^{\mathcal{O}}(c)$:
 1. Parse $c = (s, \text{pad}, \text{com})$ for parameters λ, t .
 2. Set $(\hat{m}||\rho) = y_s \oplus \text{pad}$ for $y_s = \mathcal{O}^{(t)}(1^{|\text{pad}|}, s)$.
 3. Compute $y_{\hat{m}} = \mathcal{O}^{(t)}(1^\lambda, \hat{m})$.
 4. Output $m := \hat{m}$ if $\text{com} = \mathcal{O}(1^\lambda, s||\rho||\hat{m}||y_{\hat{m}})$ and $m := \perp$ otherwise.

In the following lemmas, we prove that this construction satisfies correctness, rate-1, and bounded polynomial-depth non-malleability as required for Theorem 6.1.

Lemma 6.3 (Correctness). $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ satisfies correctness with probability 1 over the choice of $\mathcal{O}$.

Proof. Fix $\lambda, t \in \mathbb{N}$, $m \in \{0, 1\}^*$, and any oracle $\mathcal{O}$. Let $c := (s, \text{pad}, \text{com}) \leftarrow \text{Enc}^{\mathcal{O}}(1^\lambda, t, m)$ and consider the output of $\text{Dec}^{\mathcal{O}}(c)$. $\text{Dec}^{\mathcal{O}}(c)$ first computes $(m||\rho) = y_s \oplus \text{pad}$ for $y_s = \mathcal{O}^{(t)}(1^{|\text{pad}|}, s)$, which results in corresponding values m, ρ by construction of Enc . It follows that $\text{com} = \mathcal{O}(1^\lambda, s||\rho||m||y_{\hat{m}})$ for $y_{\hat{m}} = \mathcal{O}^{(t)}(1^\lambda, \hat{m})$, so $\text{Dec}^{\mathcal{O}}(c)$ outputs m , as required. $\square$

Lemma 6.4 (Rate). $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ has rate 1 for any choice of $\mathcal{O}$.

Proof. Fix any $\lambda, t \in \mathbb{N}$. An encoding of any length n message m consists of a seed s of length λ , a pad pad of length $n + \lambda$, a commitment com of length λ and implicitly includes $1^\lambda, t$, and $|m|$, which has length $\lambda + \log(t) + \log(n)$. It follows that, as $n \rightarrow \infty$, $(|s|/n) \rightarrow 0$, $|\text{pad}|/n \rightarrow 1$, $|\text{com}|/n \rightarrow 0$, and $(\lambda + \log(t) + \log(n))/n \rightarrow 0$, so

$$\lim_{n \rightarrow \infty} \left(\max_{m: |m|=n} \frac{n}{|s| + |\text{pad}| + |\text{com}| + \lambda + |\text{aux}| + \log(n)} \right) = 1,$$

as required. $\square$

Lemma 6.5 (Non-Malleability). $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ satisfies bounded polynomial-depth non-malleability. Security is proven in the AI-ROM.

Proof. Let $\text{mim}^{\mathcal{O}}$ denote the associated (plain) MIM experiment for $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ for the identity function on one-message where the relevant algorithms have oracle access to $\mathcal{O}$. Suppose by way of contradiction that $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ is not bounded polynomial-depth non-malleable in the AI-ROM. Namely, for every positive, monotonic polynomial α , there exists a non-uniform algorithm $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$ that outputs adversaries with gap α for the bounded polynomial depth non-malleability game and polynomials p, q such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} \mathcal{O} \leftarrow \text{RF} \\ (Z, M, D) \leftarrow A_\lambda(\mathcal{O}) \\ b \leftarrow \{0, 1\} \\ (m_0, m_1, t) \leftarrow Z^{\mathcal{O}} \\ \tilde{m} \leftarrow \text{mim}_M^{\mathcal{O}}(1^\lambda, t, m_b) \\ b' \leftarrow D^{\mathcal{O}}(\text{out}_b) \end{array} : \begin{array}{l} b = b' \\ \wedge |m_0| = |m_1| \\ \wedge \alpha(\lambda) \leq t \leq q(\lambda) \\ \wedge \text{size}(Z), \text{size}(M), \text{size}(D) \leq q(\lambda) \\ \wedge \text{depth}(M) \leq t/\alpha(\lambda, |m_0, m_1|) \end{array} \right] > \frac{1}{2} + \frac{1}{p(\lambda)}.$$

For given polynomial α and fixed adversary A , we assume for without loss of generality that $|m_0| = |m_1|$ and t, Z, M, D have the correct efficiency required by the experiment, otherwise we could consider an adversary A' that aborts when this is not the case and succeeds with the same probability. We define the following sequence of hybrid experiments for every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$.

- $H_{b,0}(\lambda)$: The first hybrid is simply the experiment in the probability statement above with fixed challenge bit b .
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{O}}$.
 3. Sample $c \leftarrow \text{Enc}^{\mathcal{O}}(1^\lambda, t, m_b)$. Namely, compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} = \mathcal{O}^{(t)}(1^{|m_b|+\lambda}, s) \oplus (m_b || \rho)$, and $\text{com} = \mathcal{O}(1^\lambda, s || \rho || m_b || \mathcal{O}^{(t)}(1^\lambda, m_b))$.
 4. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{O}}(c)$.
 5. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - Otherwise, set $\tilde{m} := \text{Dec}^{\mathcal{O}}(\tilde{c})$.
 6. Compute $b' \leftarrow D^{\mathcal{O}}(\tilde{m})$.
 7. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,1}(\lambda)$: In this hybrid, we swap the random oracle $\mathcal{O}$ with a pre-sampled oracle $\mathcal{P}$. In more detail, let $\text{Exp}^{\mathcal{O}}$ denote the remainder of the experiment consisting of (Z, M, D) output by $A_\lambda(\mathcal{O})$ and oracle machines Enc and Dec . Let $q'(\lambda)$ be a bound on the description size and the number of oracle queries made by $\text{Exp}^{\mathcal{O}}$, which is polynomially bounded by assumption and the efficiency of Enc and Dec . We set $f'(\lambda) = (7 \cdot q'(\lambda) \cdot p(\lambda))^2$, and let Sam be the algorithm from Lemma 3.3 that on input $\text{Exp}^{\mathcal{O}}$ that makes at most $q'(\lambda)$ queries outputs a partial assignment F of $f'(\lambda)$ points.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. Sample $c \leftarrow \text{Enc}^{\mathcal{P}}(1^\lambda, t, m_b)$. Namely, compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} = \mathcal{P}^{(t)}(1^{|m_b|+\lambda}, s) \oplus (m_b || \rho)$, and $\text{com} = \mathcal{P}(1^\lambda, s || \rho || m_b || \mathcal{P}^{(t)}(1^\lambda, m_b))$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - Otherwise, set $\tilde{m} := \text{Dec}^{\mathcal{P}}(\tilde{c})$.
 7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 8. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,2}(\lambda)$: In this hybrid, we set $\tilde{m}$ to be $\perp$ whenever the seed s and pad pad are copied, but the commitment com is modified.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.

2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. Sample $c \leftarrow \text{Enc}^{\mathcal{P}}(1^\lambda, t, m_b)$. Namely, compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} = \mathcal{P}^{(t)}(1^{|m_b|+\lambda}, s) \oplus (m_b || \rho)$, and $\text{com} = \mathcal{P}(1^\lambda, s || \rho || m_b || \mathcal{P}^{(t)}(1^\lambda, m_b))$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - If $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$, set $\tilde{m} := \perp$.
 - Otherwise, set $\tilde{m} := \text{Dec}^{\mathcal{P}}(\tilde{c})$.
 7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 8. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,3}(\lambda)$: In this hybrid, we set $\tilde{m}$ to be $\perp$ whenever the candidate output $\hat{m}$ computed by $\text{Dec}^{\mathcal{P}}(\tilde{c})$ is not in the pre-sampled set F .
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. Sample $c \leftarrow \text{Enc}^{\mathcal{P}}(1^\lambda, t, m_b)$. Namely, compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} = \mathcal{P}^{(t)}(1^{|m_b|+\lambda}, s) \oplus (m_b || \rho)$, and $\text{com} = \mathcal{P}(1^\lambda, s || \rho || m_b || \mathcal{P}^{(t)}(1^\lambda, m_b))$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - If $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$, set $\tilde{m} := \perp$.
 - Otherwise, compute $\hat{m} = \text{Dec}^{\mathcal{P}}(\tilde{c})$. If $\hat{m} \notin F$, set $\tilde{m} := \perp$, otherwise set $\tilde{m} := \hat{m}$.
 7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 8. Output 1 if $b' = b$ and 0 otherwise.
 - $H_{b,4}(\lambda)$: In this hybrid, we compute the candidate $\hat{m}$ in the case where $\tilde{s} = s$ without having to compute $\mathcal{P}^{(t)}(1^\lambda, \tilde{s})$. Namely, instead we use knowledge of pad , m_b , and ρ computed earlier in the experiment to determine $\hat{m}$.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. Sample $c \leftarrow \text{Enc}^{\mathcal{P}}(1^\lambda, t, m_b)$. Namely, compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} = \mathcal{P}^{(t)}(1^{|m_b|+\lambda}, s) \oplus (m_b || \rho)$, and $\text{com} = \mathcal{P}(1^\lambda, s || \rho || m_b || \mathcal{P}^{(t)}(1^\lambda, m_b))$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.

- If $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$, set $\tilde{m} := \perp$.
 - If $\tilde{s} = s$ and $\tilde{\text{pad}} \neq \text{pad}$, compute $(\hat{m} || \tilde{\rho}) = \text{pad} \oplus \tilde{\text{pad}} \oplus (m_b || \rho)$. Set $\tilde{m} := \hat{m}$ if $\tilde{\text{com}} = \mathcal{P}(1^\lambda, s || \tilde{\rho} || \hat{m} || \mathcal{P}^{(t)}(1^\lambda, \hat{m}))$ and $\hat{m} \in F$, and $\tilde{m} := \perp$ otherwise.
 - Otherwise when $\tilde{s} \neq s$, compute $\hat{m} = \text{Dec}^{\mathcal{P}}(\tilde{c})$. If $\hat{m} \notin F$, set $\tilde{m} := \perp$, otherwise set $\tilde{m} := \hat{m}$.
 - 7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 - 8. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,5}(\lambda)$: In this hybrid, we instead sample the value pad for the challenge encoding as a uniformly random string.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. Compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} \leftarrow \{0, 1\}^{|m_b| + \lambda}$, and $\text{com} = \mathcal{P}(1^\lambda, s || \rho || m_b || \mathcal{P}^{(t)}(1^\lambda, m_b))$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - If $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$, set $\tilde{m} := \perp$.
 - If $\tilde{s} = s$ and $\tilde{\text{pad}} \neq \text{pad}$, compute $(\hat{m} || \tilde{\rho}) = \text{pad} \oplus \tilde{\text{pad}} \oplus (m_b || \rho)$. Set $\tilde{m} := \hat{m}$ if $\tilde{\text{com}} = \mathcal{P}(1^\lambda, s || \tilde{\rho} || \hat{m} || \mathcal{P}^{(t)}(1^\lambda, \hat{m}))$ and $\hat{m} \in F$, and $\tilde{m} := \perp$ otherwise.
 - Otherwise when $\tilde{s} \neq s$, compute $\hat{m} = \text{Dec}^{\mathcal{P}}(\tilde{c})$. If $\hat{m} \notin F$, set $\tilde{m} := \perp$, otherwise set $\tilde{m} := \hat{m}$.
 7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 8. Output 1 if $b' = b$ and 0 otherwise.
 - $H_{b,6}(\lambda)$: In this hybrid, we sample the commitment com for the challenge encoding as a uniformly random string.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. Compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} \leftarrow \{0, 1\}^{|m_b| + \lambda}$, and $\text{com} \leftarrow \{0, 1\}^\lambda$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - If $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$, set $\tilde{m} := \perp$.
 - If $\tilde{s} = s$ and $\tilde{\text{pad}} \neq \text{pad}$, compute $(\hat{m} || \tilde{\rho}) = \text{pad} \oplus \tilde{\text{pad}} \oplus (m_b || \rho)$. Set $\tilde{m} := \hat{m}$ if $\tilde{\text{com}} = \mathcal{P}(1^\lambda, s || \tilde{\rho} || \hat{m} || \mathcal{P}^{(t)}(1^\lambda, \hat{m}))$ and $\hat{m} \in F$, and $\tilde{m} := \perp$ otherwise.
 - Otherwise when $\tilde{s} \neq s$, compute $\hat{m} = \text{Dec}^{\mathcal{P}}(\tilde{c})$. If $\hat{m} \notin F$, set $\tilde{m} := \perp$, otherwise set $\tilde{m} := \hat{m}$.

7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 8. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,7}(\lambda)$: In this hybrid, we simply set $\tilde{m}$ to be $\perp$ in the case where $\tilde{s} = s$.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z^{\mathcal{P}}$.
 4. compute $c = (s, \text{pad}, \text{com})$ where $s, \rho \leftarrow \{0, 1\}^\lambda$, $\text{pad} \leftarrow \{0, 1\}^{|m_b|+\lambda}$, and $\text{com} \leftarrow \{0, 1\}^\lambda$.
 5. Compute $\tilde{c} := (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) \leftarrow M^{\mathcal{P}}(c)$.
 6. Compute $\tilde{m}$ as follows:
 - If $(\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}}) = (s, \text{pad}, \text{com})$, set $\tilde{m} := \perp$.
 - If $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$, set $\tilde{m} := \perp$.
 - If $\tilde{s} = s$ and $\tilde{\text{pad}} \neq \text{pad}$, set $\tilde{m} := \perp$.
 - Otherwise when $\tilde{s} \neq s$, compute $\hat{m} = \text{Dec}^{\mathcal{P}}(\tilde{c})$. If $\hat{m} \notin F$, set $\tilde{m} := \perp$, otherwise set $\tilde{m} := \hat{m}$.
 7. Compute $b' \leftarrow D^{\mathcal{P}}(\tilde{m})$.
 8. Output 1 if $b' = b$ and 0 otherwise.

By assumption, it holds that for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned}
 \frac{1}{2} + \frac{1}{p(\lambda)} &< \frac{1}{2} \cdot \Pr[H_{0,0}(\lambda)] + \frac{1}{2} \cdot \Pr[H_{1,0}(\lambda)] \\
 &= \frac{1}{2} \cdot \left(\Pr[H_{0,7}(\lambda)] + \sum_{i=1}^7 (\Pr[H_{0,i-1}(\lambda)] - \Pr[H_{0,i}(\lambda)]) \right) \\
 &\quad + \frac{1}{2} \cdot \left(\Pr[H_{1,7}(\lambda)] + \sum_{i=1}^7 (\Pr[H_{1,i-1}(\lambda)] - \Pr[H_{1,i}(\lambda)]) \right)
 \end{aligned}$$

However, observe that $H_{0,7}$ and $H_{1,7}$ are identically distributed since $|m_0| = |m_1|$, so $\Pr[H_{0,7}(\lambda)] + \Pr[H_{1,7}(\lambda)] = 1$ for all $\lambda \in \mathbb{N}$. It follows that for infinitely many $\lambda \in \mathbb{N}$,

$$\frac{1}{p(\lambda)} < \frac{1}{2} \cdot \sum_{b \in \{0,1\}, i \in [7]} (\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)]).$$

So, there exists a pair of consecutive hybrids such that for some $b \in \{0, 1\}$ and $i \in [7]$,

$$\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)] > \frac{1}{7 \cdot p(\lambda)}.$$

In the claims below, we show that this leads to a contradiction, as required for the lemma. We give the below claims for $b = 0$ without loss of generality.

Claim 6.6. *For all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)]| \leq 1/(7 \cdot p(\lambda))$.*

Proof. This follows from the AI-ROM pre-sampling lemma of Lemma 3.3 similar to Claim 7.8. Hybrid $H_{0,0}(\lambda)$ corresponds to the case where $\mathcal{O}$ is a random function and $A_\lambda(\mathcal{O})$ outputs a description of the rest of the experiment $\text{Exp}^\mathcal{O}$. Hybrid $H_{0,1}(\lambda)$ corresponds to the case where instead the experiment receives oracle access to $\mathcal{P}$ pre-sampled at F determined by **Sam** given the description of Exp . F is fixed at $f'(\lambda) = (7 \cdot q'(\lambda) \cdot p(\lambda))^2$ points, where $q'(\lambda)$ is a bound on the description size and number of oracle queries made by Exp . It follows from Lemma 3.3 that the statistical distance between the output bits in both experiments is at most

$$\left(\frac{(q'(\lambda))^2}{2 \cdot f'(\lambda)} \right)^{1/2} = \left(\frac{(q'(\lambda))^2}{2 \cdot (7 \cdot q'(\lambda) \cdot p(\lambda))^2} \right)^{1/2} \leq \frac{1}{7 \cdot p(\lambda)},$$

as required. $\square$

Claim 6.7. *For all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,1}(\lambda)] - \Pr[H_{0,2}(\lambda)]| = 0$.*

Proof. Hybrid $H_{0,1}(\lambda)$ and $H_{0,2}(\lambda)$ only differ when computing $\tilde{m}$ in the case that $(\tilde{s}, \tilde{\text{pad}}) = (s, \text{pad})$ but $\tilde{\text{com}} \neq \text{com}$. Specifically, $H_{0,1}(\lambda)$ outputs $\tilde{m} = \text{Dec}^\mathcal{P}(\tilde{c})$ and $H_{0,2}(\lambda)$ outputs $\perp$, so the experiments differ if $\tilde{m} \neq \perp$. By correctness of **Enc**, we know that $\tilde{\text{pad}} = \text{pad} = \mathcal{P}^{(t)}(1^{|m_0|+\lambda}, s) \oplus (m_0 || \rho)$, so $\text{Dec}^\mathcal{P}(\tilde{c})$ will unmask the original message m_0 with randomness ρ . But $\mathcal{P}(1^\lambda, s || \rho || m_0 || \mathcal{P}^{(t)}(1^\lambda, m_0)) = \text{com}$ by correctness of **Enc**, so it cannot also be equal to $\tilde{\text{com}}$ since $\tilde{\text{com}} \neq \text{com}$. It follows that, in this case, $\text{Dec}^\mathcal{P}(\tilde{c})$ always outputs $\tilde{m} = \perp$, as required. $\square$

Claim 6.8. *There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, $|\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)]| \leq \mu(\lambda)$.*

Proof. Hybrids $H_{0,2}(\lambda)$ and $H_{0,3}(\lambda)$ potentially differ when computing $\tilde{m}$. Namely, $H_{0,2}(\lambda)$ may output a candidate message $\hat{m} = \text{Dec}^\mathcal{P}(\tilde{c})$ corresponding to $\hat{m} \notin F$, whereas $H_{0,3}(\lambda)$ always outputs $\perp$ in this case. So, it suffices to argue that the probability that $M^\mathcal{P}$ outputs a value $\tilde{c} = (\tilde{s}, \tilde{\text{pad}}, \tilde{\text{com}})$ with underlying value $\tilde{m} = \text{Dec}^\mathcal{P}(\tilde{c})$ such that $\tilde{m} \neq \perp$ and $\tilde{m} \notin F$ is negligible.

Let $y_{\tilde{m}} = \mathcal{P}^{(t)}(1^\lambda, \tilde{m})$. Given that $\tilde{m} \neq \perp$, it means that $\tilde{\text{com}} = \mathcal{P}(1^\lambda, s || \tilde{\rho} || \tilde{m} || y_{\tilde{m}})$ for some $\tilde{\rho}$ computed during $\text{Dec}^\mathcal{P}(\tilde{c})$. Since $s \leftarrow \{0, 1\}^\lambda$ is uniform and sampled independently of F , this query is in F with probability at most $|F|/2^\lambda = f'(\lambda)/2^\lambda$. As such, the output com can be lazily sampled during the experiment. If M did not already query $\mathcal{P}$ at this value, the probability that this check passes is at most $1/2^\lambda$. It follows that $M^\mathcal{P}$ must have queried $\mathcal{P}$ at $(s || \tilde{\rho} || \tilde{m} || y_{\tilde{m}})$ with probability at least $1 - (f'(\lambda) + 1)/2^\lambda$, so in other words, it likely must have “known” the value $y_{\tilde{m}} = \mathcal{P}^{(t)}(1^\lambda, \tilde{m})$ before outputting the value $\tilde{c}$.

We proceed to bound the probability that M makes a query to $\mathcal{P}$ of the form $(\cdot || \tilde{m} || y_{\tilde{m}})$ for $\tilde{m} \notin F$. We define the query path used to compute $y_{\tilde{m}} = \mathcal{O}^{(t)}(1^n, \tilde{m})$ as follows. Let $x_0 := \tilde{m}$ denote the initial input, and define $x_i := \mathcal{O}(1^\lambda, x_{i-1})$ for each $i \in [t]$. Now, let $Q_{\leq i}$ denote the set of queries that M makes to $\mathcal{P}$ at or before depth i . We want to argue that the penultimate value x_{t-1} is unlikely to be in F or queried by any M running in depth less than t . This means that $x_t := y_{\tilde{m}}$ can be lazily sampled, bounding the probability that M could have possibly queried $(\cdot || \tilde{m} || y_{\tilde{m}})$.

We argue this inductively and show that for all $i = 0, \dots, t-1$, it holds that

$$\Pr[x_i \in Q_{\leq i} \cup F \mid \tilde{m} \notin F] \leq i \cdot \frac{q(\lambda) + f'(\lambda)}{2^\lambda}.$$

In the base case of $i = 0$, this is vacuously true since $Q_{\leq 0} = \emptyset$ and $x_0 := \tilde{m} \notin F$ by assumption. It follows that

$$\Pr[x_0 \in Q_{\leq 0} \cup F] = 0 \leq 0 \cdot \frac{q(\lambda) + f'(\lambda)}{2^\lambda}.$$

Now inductively assume the claim holds for $i - 1 \geq 0$, and we will show that it must also hold for i . To analyze $\Pr[x_i \in Q_{\leq i} \cup F]$, we consider two cases: either $x_{i-1} \in Q_{\leq i-1} \cup F$ or not. By induction, the first case holds with probability at most $(i - 1) \cdot (q(\lambda) + f'(\lambda))/2^\lambda$. In the second case, since x_{i-1} has not been queried before depth i and isn't in F , we can lazily sample the output x_i at depth i while emulating $M^\mathcal{P}$. Since x_i is a uniformly random λ -bit value, the probability that it is in either $Q_{\leq i}$ or F is at most $(|Q_{\leq i}| + |F|)/2^\lambda \leq (q(\lambda) + f'(\lambda))/2^\lambda$. Combining the two disjoint cases, this implies that

$$\Pr[x_i \in Q_{\leq i} \cup F] \leq i \cdot \frac{q(\lambda) + f'(\lambda)}{2^\lambda},$$

which completes the inductive claim.

The proof of the claim follows by taking a union over the two bad events above. Namely, if the hybrids differ, then $\tilde{m} \notin F$ and either M queries $\mathcal{P}$ at $(s||\tilde{\rho}||\tilde{m}||y_{\tilde{m}})$, or this query is in F or happens to hit $\text{c}\tilde{\text{om}}$. Together, these events happen with probability at most

$$\mu(\lambda) = (t + 1) \cdot \frac{q(\lambda) + f'(\lambda)}{2^\lambda},$$

which is negligible in λ as t , q , and f' are all polynomially bounded. $\square$

Claim 6.9. *For all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)]| = 0$.*

Proof. Hybrids $H_{0,3}(\lambda)$ and $H_{0,4}(\lambda)$ potentially differ in the way $\tilde{m}$ is computed in the case that $\tilde{s} = s$ and $\tilde{\text{pad}} \neq \text{pad}$. $H_{0,3}(\lambda)$ computes $\tilde{m} = \text{Dec}^\mathcal{P}(\tilde{c})$, so it computes candidate message and randomness pair $(\hat{m}||\tilde{\rho}) = \mathcal{O}^{(t)}(1^{|\tilde{\text{pad}}|}, s) \oplus \tilde{\text{pad}}$. $H_{0,4}(\lambda)$ computes its candidate message and randomness pair as

$$\begin{aligned} & \text{pad} \oplus \tilde{\text{pad}} \oplus (m_0||\rho) \\ &= \left(\mathcal{O}^{(t)}(1^{|\tilde{\text{pad}}|}, s) \oplus (m_0||\rho) \right) \oplus \left(\mathcal{O}^{(t)}(1^{|\tilde{\text{pad}}|}, s) \oplus (\hat{m}||\tilde{\rho}) \right) \oplus (m_0||\rho) \\ &= (\hat{m}||\tilde{\rho}), \end{aligned}$$

so it follows that both hybrids compute the same candidate message and randomness pair $(\hat{m}||\tilde{\rho})$. Both hybrids then make the same checks that $\text{c}\tilde{\text{om}} = \mathcal{P}(1^\lambda, s||\tilde{\rho}||\hat{m}||\mathcal{P}^{(t)}(1^\lambda, \hat{m}))$ and $\hat{m} \in F$, so their outputs are identically distributed in this case. $\square$

Claim 6.10. *There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,4}(\lambda)] - \Pr[H_{0,5}(\lambda)]| \leq \mu(\lambda)$.*

Proof. Hybrids $H_{0,4}(\lambda)$ and $H_{0,5}(\lambda)$ differ in the way pad is computed for the challenge code c . Namely, $H_{0,4}(\lambda)$ computes $\text{pad} = \mathcal{P}^{(t)}(1^{m_0+\lambda}, s) \oplus (m_0||\rho)$ and $H_{0,5}(\lambda)$ simply sets $\text{pad} \leftarrow \{0, 1\}^{m_0+\lambda}$. We define the query path for s as follows. Let $s_0 := s$ and define $s_i = \mathcal{P}(1^{m_0+\lambda}, s_{i-1})$ for $i \in [t]$. It follows that these hybrids are identically distributed whenever the final output $s_t = \mathcal{P}(1^{m_0+\lambda}, s_{t-1})$ can be lazily sampled and the rest of the experiment doesn't query $\mathcal{P}$ at s_{t-1} .

So, it suffices to bound $\Pr[s_{t-1} \in F]$ or the probability that the rest of the experiment queries $\mathcal{P}$ at s_{t-1} .

Note that $s \leftarrow \{0,1\}^\lambda$ is sampled after and hence independently of F . It follows that $s_0 \in F$ with probability at most $|F|/2^\lambda = f'(\lambda)/2^\lambda$. Assuming that $s_0 \notin F$, then it follows that $s_{t-1} \notin F$ and M doesn't query $\mathcal{P}$ with probability at most $t \cdot (q(\lambda) + f'(\lambda))/2^\lambda$ by an identical inductive argument as used in Claim 6.8 above. It remains to bound the probability that the experiment queries $\mathcal{P}$ at s_{t-1} when computing $\tilde{m}$ or when computing $D^{\mathcal{P}}(\tilde{m})$, in the event that $s_0 \notin F$ and M did not already query s_{t-1} .

When computing $\tilde{m}$, the experiments $H_{0,4}$ or $H_{0,5}$ only possibly queries $\mathcal{P}$ at s_{t-1} in the case when $\tilde{s} \neq s$. In this case, it computes a candidate message $\hat{m} = \text{Dec}^{\mathcal{P}}(\tilde{c})$. The only possible values of length $|m_0| + \lambda$ queried by $\text{Dec}^{\mathcal{P}}(\tilde{c})$ are from the query path for $\tilde{s}$ when computing $\mathcal{P}^{(t)}(1^{|m_0|+\lambda}, \tilde{s})$. So, let $\tilde{s}_0 := \tilde{s}$ and define $\tilde{s}_i = \mathcal{P}(1^{|m_0|+\lambda}, \tilde{s}_{i-1})$ for $i \in [t]$. It suffices to bound the probability that $s_{t-1} = \tilde{s}_i$ for any $i \in [t-1]$. Since we are already assuming $s_{t-1} \notin F$, then this can hold only for values $\tilde{s}_i \notin F$. Each such value, however, can be lazily sampled as the output from the previous value in the query path. Taking a union bound, the probability that any of these values are equal to s_{t-1} is at most $t/2^\lambda$.

When computing $D^{\mathcal{P}}(\tilde{m})$, note that $\tilde{m}$ must be in F by construction. That means that $D^{\mathcal{P}}(\cdot)$ can ever possibly make $|F| \cdot q(\lambda)$ queries across all possible inputs. So the probability that s_{t-1} , which was lazily sampled when $s_{t-1} \notin F$, is in this set is at most $f'(\lambda) \cdot q(\lambda)/2^\lambda$.

Taking a union over all of the bad events above, we have that these two hybrids differ with probability at most

$$\mu(\lambda) = \frac{f'(\lambda)}{2^\lambda} + t \cdot \frac{q(\lambda) + f'(\lambda)}{2^\lambda} + \frac{t}{2^\lambda} + \frac{f'(\lambda) \cdot q(\lambda)}{2^\lambda},$$

which is negligible in λ as f' , t , and q are all polynomially bounded. $\square$

Claim 6.11. *There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,5}(\lambda)] - \Pr[H_{0,6}(\lambda)]| \leq \mu(\lambda)$.*

Proof. Hybrids $H_{0,5}(\lambda)$ and $H_{0,6}(\lambda)$ differ in the way **com** is computed for the challenge code. $H_{0,5}(\lambda)$ computes $\text{com} = \mathcal{P}(1^\lambda, s || \rho || m || y_m)$ for $y_m = \mathcal{O}^{(t)}(1^\lambda, m)$ and $H_{0,6}(\lambda)$ computes $\text{com} \leftarrow \{0,1\}^\lambda$. These experiments are identically distributed as long as $(s || \rho || m || y_m) \notin F$ and the rest of the experiment never queries $(s || \rho || m || y_m)$. Since $\rho \leftarrow \{0,1\}^\lambda$ is uniformly random and is now completely independent of F and the rest of the experiment, it follows that $\Pr[(s || \rho || m || y_m) \in F] \leq f'(\lambda)/2^\lambda$ and the rest of the experiment makes a query including ρ with probability at most $q'(\lambda)/2^\lambda$, where recall q' is a bound on the number of queries made by the experiment. Thus, the experiments differ with probability at most

$$\mu(\lambda) = \frac{f'(\lambda) + q'(\lambda)}{2^\lambda},$$

which is at most negligible in λ since f' and q' are polynomially bounded. $\square$

Claim 6.12. *There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,6}(\lambda)] - \Pr[H_{0,7}(\lambda)]| \leq \mu(\lambda)$.*

Proof. Hybrids $H_{0,6}(\lambda)$ and $H_{0,7}(\lambda)$ potentially differ in how they compute $\tilde{m}$ in the case that $\tilde{s} = s$ and $\tilde{\text{pad}} \neq \text{pad}$. Hybrid $H_{0,7}(\lambda)$ simply sets $\tilde{m} := \perp$ in this case. Hybrid $H_{0,6}(\lambda)$ computes candidate message and randomness $(\hat{m}||\tilde{\rho}) = \text{pad} \oplus \tilde{\text{pad}} \oplus (m_0||\rho)$ and outputs $\perp$ whenever $\text{c\ddot{o}m} \neq \mathcal{P}(1^\lambda, s||\tilde{\rho}||\hat{m}||y_{\hat{m}})$ for $y_{\hat{m}} = \mathcal{P}^{(t)}(1^\lambda, \hat{m})$. Note that ρ is a uniformly random value that is now independent of pad and $\tilde{\text{pad}}$, so $\tilde{\rho}$ is also a uniformly random value. It follows that any query $(\cdot||\tilde{\rho}||\cdot)$ is in F with probability at most $f'(\lambda)/2^\lambda$ and otherwise its output can be lazily sampled. If lazily sampled, the probability that $\text{c\ddot{o}m} = \mathcal{P}(1^\lambda, s||\tilde{\rho}||\hat{m}||y_{\hat{m}})$ for any fixed value of $\text{c\ddot{o}m}$ is at most $1/2^\lambda$. It follows that $H_{0,6}(\lambda)$ outputs $\perp$ in this case, and hence the two hybrids are identically distributed, except with probability

$$\mu(\lambda) = \frac{f'(\lambda) + 1}{2^\lambda},$$

which is negligible in λ as f' is polynomially bounded. □

This completes the proof of Lemma 6.5. □

7 Functional Non-Malleability from CCA-Hiding in the AI-ROM

In this section, we construct a functional non-malleable encoding scheme from a CCA-hiding one in the AI-ROM. We consider the following function class that allows for arbitrary functions with bounded output length.

Definition 7.1 (Function Class $\mathcal{F}_{k,\ell}$ with Bounded Output Length). Let $k: \mathbb{N} \rightarrow \mathbb{N}$ and $\ell: \mathbb{N}^3 \rightarrow \mathbb{N}$ be a function that may depend on k . Let $\mathcal{F}_{k,\ell}$ be a class of multi-input functions $f_{\lambda,t}$ parameterized by a security parameter $\lambda \in \mathbb{N}$ and auxiliary input $\text{aux} \in \{0,1\}^*$. We say that $f_{\lambda,t} \in \mathcal{F}_{k,\ell}$ if for any $k' \leq k(\lambda)$, $n \in \mathbb{N}$, and messages $m_1, \dots, m_{k'} \in \{0,1\}^n$, it holds that $|f_{\lambda,t}(m_1, \dots, m_{k'})| \leq \ell(\lambda, |\text{aux}|, n)$.

We prove the following theorem in the rest of this section.

Theorem 7.2 (Functional Non-Malleability Compiler for CCA-Hiding Encoding Schemes). *Let $(\text{Enc}', \text{Dec}')$ be any encoding scheme that satisfies CCA-hiding for a subset of non-uniform PPT adversaries. For any efficiently computable $k: \mathbb{N} \rightarrow \mathbb{N}$ and $\ell: \mathbb{N}^3 \rightarrow \mathbb{N}$, there exists an encoding scheme (Enc, Dec) in the AI-ROM with rate $\alpha(\lambda, |\text{aux}|, n) = 1/(1 + \ell(\lambda, |\text{aux}|, n)/n)$ with a depth-preserving reduction from the functional non-malleability of (Enc, Dec) for $\mathcal{F}_{k,\ell}$ to the CCA-hiding of $(\text{Enc}', \text{Dec}')$.*

Before giving the construction, we remark on a few immediate corollaries to Theorem 7.2 that follow from instantiating the theorem with particular function classes.

- For plain (one-to-one) non-malleability, $k(\lambda) = 1$ and $\ell(\lambda, |\text{aux}|, n) = n$, so the resulting construction has rate $1/2$.
- For k -bounded concurrent non-malleability, $\ell(\lambda, |\text{aux}|, n) = k(\lambda) \cdot n$, so the resulting construction has rate $1/(1 + k)$. For the case of time-lock puzzles, [FKPS21] prove that any k -bounded concurrent non-malleable time-lock puzzle has rate at most $1/k$, so our compiler nearly matches this bound.

- Finally, the application to multi-party coin-flipping requires functional non-malleability for the “ $\oplus$ ” function, which has $\ell(\lambda, |\text{aux}|, n) = n$ for any (potentially unbounded) k . For this function class, our resulting construction has rate $1/2$. In comparison, the corresponding functional non-malleable time-lock puzzle of [FKPS21] in the AI-ROM only achieves inverse polynomial-rate.

Remark 7.3 (On allowing functions that query $\mathcal{O}$). We note that the Theorem 7.2 holds for all (potentially unbounded) functions $f \in \mathcal{F}_{k,\ell}$ as long as the output length is bounded. However, we do not allow functions f that query $\mathcal{O}$. This is still meaningful as the function class $\mathcal{F}_{k,\ell}$ itself is not adversarially chosen and still captures many of the above relevant application scenarios. With a slight modification to the proof below, we could allow f to query $\mathcal{O}$, but we would then require f to be restricted in the same way as the MIM adversary in the functional non-malleability game.

Our construction for Theorem 7.2 makes use of an underlying encoding scheme $(\text{Enc}', \text{Dec}')$ and a common random oracle $\mathcal{O}$. At a high level, our construction uses the inner encoding Enc' to encode a small seed s and a succinct commitment to m using randomness ρ (based on $\mathcal{O}$). It additionally outputs a sufficiently long random string r . It then uses $\mathcal{O}$ to expand the seed s together with r to mask the message m and ρ . We formalize this in Construction 7.4.

Construction 7.4. Functional NM Codes from CCA-Hiding

Ingredients: Let $(\text{Enc}', \text{Dec}')$ be any encoding scheme that satisfies CCA-hiding for some class of polynomially bounded adversaries (Definitions 4.1, 4.5), and let $\mathcal{O}$ be a common random function (formalized in Section 3.3).

Construction: For each efficiently computable k, ℓ , we construct encoding and decoding algorithms $(\text{Enc}_{k,\ell}^{\mathcal{O}}, \text{Dec}_{k,\ell}^{\mathcal{O}})$ as follows:

- $c \leftarrow \text{Enc}_{k,\ell}^{\mathcal{O}}(1^\lambda, \text{aux}, m)$:
 1. Sample $s, \text{hk}, \rho \leftarrow \{0, 1\}^\lambda$ and $r \leftarrow \{0, 1\}^{\ell(\lambda, |\text{aux}|, |m|)}$.
 2. Compute $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{com}))$ for $\text{com} = \mathcal{O}(1^\lambda, \text{hk}||m||\rho)$.
 3. Set $\text{pad} = \mathcal{O}(1^{|m|+\lambda}, s||r) \oplus (m||\rho)$.
 4. Output $c := (c', r, \text{pad})$.
- $m = \text{Dec}_{k,\ell}^{\mathcal{O}}(c)$:
 1. Parse $c := (c', r, \text{pad})$.
 2. Compute $(s, \text{hk}, \text{com}) = \text{Dec}'(c')$.
 3. Set $(\hat{m}||\rho) = \mathcal{O}(1^{|\text{pad}|}, s||r) \oplus \text{pad}$.
 4. Output $m := \hat{m}$ if $\text{com} = \mathcal{O}(1^\lambda, \text{hk}||\hat{m}||\rho)$ and $m := \perp$ otherwise.

In the following lemmas, we prove that this construction satisfies correctness, has the claimed rate, and satisfies functional non-malleability as required for Theorem 7.2.

Lemma 7.5 (Correctness). *Assuming correctness of $(\text{Enc}', \text{Dec}')$, $(\text{Enc}_{k,\ell}^{\mathcal{O}}, \text{Dec}_{k,\ell}^{\mathcal{O}})$ satisfies correctness with probability 1 over the choice of $\mathcal{O}$.*

Proof. Fix $\lambda \in \mathbb{N}$, $\text{aux}, m \in \{0, 1\}^*$, and any oracle $\mathcal{O}$. Let $c \leftarrow \text{Enc}_{k,\ell}^{\mathcal{O}}(1^\lambda, \text{aux}, m)$ and consider the output of $\text{Dec}_{k,\ell}^{\mathcal{O}}(c)$. Recall that $c = (c', r, \text{pad})$. $\text{Dec}_{k,\ell}^{\mathcal{O}}(c)$ first computes $(s, \text{hk}, \text{com}) = \text{Dec}'(c')$, which correspond to the correct associated values computed in $\text{Enc}_{k,\ell}^{\mathcal{O}}$ by correctness of $(\text{Enc}', \text{Dec}')$. $\text{Dec}_{k,\ell}^{\mathcal{O}}(c)$ next computes $(\hat{m}||\rho) = \mathcal{O}(1^{|\text{pad}|}, s||r) \oplus \text{pad}$, and by construction of Enc , it follows that $\hat{m} = m$ and $\text{com} = \mathcal{O}(1^\lambda, \text{hk}||\hat{m}||\rho)$ for any choice of $\mathcal{O}$. It follows that $\text{Dec}_{k,\ell}^{\mathcal{O}}(c)$ outputs m as required. $\square$

Lemma 7.6 (Rate). $(\text{Enc}_{k,\ell}^{\mathcal{O}}, \text{Dec}_{k,\ell}^{\mathcal{O}})$ has rate $1/(1 + \ell(\lambda, |\text{aux}|, n)/n)$ for any choice of $\mathcal{O}$.

Proof. Fix any $\lambda \in \mathbb{N}$ and $\text{aux} \in \{0, 1\}^*$. An encoding of any length n message m consists of an inner encoding c' , a random string r of length $\ell(\lambda, |\text{aux}|, n)$, a pad pad of length $n + \lambda$, and implicitly includes 1^λ , aux , and $|m|$, which has length $\lambda + |\text{aux}| + \log(n)$. The encoding c' is sampled from $\text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{com}))$ where $|s| = |\text{hk}| = |\text{com}| = \lambda$, so the inputs to Enc' and hence $|c'|$ are independent of n . It follows that, as $n \rightarrow \infty$, $(|c'|/n) \rightarrow 0$, $(|r|/n) \rightarrow \ell(\lambda, |\text{aux}|, n)/n$, $(|\text{pad}|/n) \rightarrow 1$, and $(\lambda + |\text{aux}| + \log(n))/n \rightarrow 0$, so

$$\lim_{n \rightarrow \infty} \left(\max_{m: |m|=n} \frac{n}{|c'| + |r| + |\text{pad}| + \lambda + |\text{aux}| + \log(n)} \right) = \frac{1}{1 + \ell(\lambda, |\text{aux}|, n)/n},$$

as required. $\square$

Lemma 7.7 (Functional Non-Malleability). *There exists a depth-preserving reduction from the functional non-malleability of $(\text{Enc}_{k,\ell}^{\mathcal{O}}, \text{Dec}_{k,\ell}^{\mathcal{O}})$ for $\mathcal{F}_{k,\ell}$ to the CCA-hiding of $(\text{Enc}', \text{Dec}')$. Security is proven in the AI-ROM.*

Proof. Let $\text{mim}^{\mathcal{O}}$ denote the associated functional MIM experiment for $(\text{Enc}_{k,\ell}^{\mathcal{O}}, \text{Dec}_{k,\ell}^{\mathcal{O}})$ as in Definition 4.6 where the relevant algorithms are given oracle access to $\mathcal{O}$. Suppose by way of contradiction that $(\text{Enc}_{k,\ell}^{\mathcal{O}}, \text{Dec}_{k,\ell}^{\mathcal{O}})$ is not functional non-malleable for $\mathcal{F}_{k,\ell}$ in the AI-ROM. Namely, there exists a non-uniform algorithm $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$ that outputs adversaries for the functional non-malleability game for $\mathcal{F}_{k,\ell}$ and a polynomial p such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} \mathcal{O} \leftarrow \text{RF} \\ (Z, M, D) \leftarrow A_\lambda(\mathcal{O}) \\ b \leftarrow \{0, 1\} \\ (m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{O}} \\ \text{out}_b \leftarrow \text{mim}_{M,f}^{\mathcal{O}}(1^\lambda, \text{aux}, m_b) \\ b' \leftarrow D^{\mathcal{O}}(\text{out}_b) \end{array} : \begin{array}{l} b = b' \\ \wedge |m_0| = |m_1| \\ \wedge f \in \mathcal{F}_{k,\ell} \end{array} \right] > \frac{1}{2} + \frac{1}{p(\lambda)}.$$

For fixed adversary A , we assume without loss of generality that $|m_0| = |m_1|$ and $f \in \mathcal{F}_{k,\ell}$, otherwise we could consider an adversary A' that simply aborts when this is not the case and succeeds with the same probability.

We define the following sequence of hybrid experiments for every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$. For simplicity, we write k to denote $k(\lambda)$ and ℓ to denote $\ell(\lambda, |\text{aux}|, |m_b|)$.

- $H_{b,0}(\lambda)$: In the first hybrid, we simply fix a value of b in the above functional non-malleability experiment.

1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.

2. Compute $(m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{O}}$.
 3. Compute $c \leftarrow \text{Enc}_{k,\ell}^{\mathcal{O}}(1^\lambda, \text{aux}, m_b)$.
 4. Compute $(\tilde{c}_1, \dots, \tilde{c}_k) \leftarrow M^{\mathcal{O}}(c)$.
 5. For each $i \in [k]$, set $\tilde{m}_i = (\text{Dec}_{k,\ell}^{\mathcal{O}})^*(\tilde{c}_i)$.
 6. Compute $\text{out}_b = f(\tilde{m}_1, \dots, \tilde{m}_k)$ and $b' \leftarrow D^{\mathcal{O}}(\text{out}_b)$.
 7. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,1}(\lambda)$: In this hybrid, we swap the random oracle $\mathcal{O}$ with a pre-sampled oracle $\mathcal{P}$. In more detail, let $\text{Exp}^{\mathcal{O}}$ denote the remainder of the experiment consisting of (Z, M, D) output by $A_\lambda(\mathcal{O})$ and oracle machines Enc and Dec . Let $q'(\lambda)$ be a bound on the description size and the number of oracle queries made by $\text{Exp}^{\mathcal{O}}$, which is polynomially bounded by assumption and the efficiency of Enc and Dec . We set $f'(\lambda) = (10 \cdot q'(\lambda) \cdot p(\lambda))^2$, and let Sam be the algorithm from Lemma 3.3 that on input $\text{Exp}^{\mathcal{O}}$ that makes at most $q'(\lambda)$ queries outputs a partial assignment F of $f'(\lambda)$ points.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$, $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{P}}$.
 4. Compute $c \leftarrow \text{Enc}_{k,\ell}^{\mathcal{P}}(1^\lambda, \text{aux}, m_b)$.
 5. Compute $(\tilde{c}_1, \dots, \tilde{c}_k) \leftarrow M^{\mathcal{P}}(c)$.
 6. For each $i \in [k]$, set $\tilde{m}_i = (\text{Dec}^{\mathcal{P}})^*(\tilde{c}_i)$.
 7. Compute $\text{out}_b = f(\tilde{m}_1, \dots, \tilde{m}_k)$ and $b' \leftarrow D^{\mathcal{P}}(\text{out}_b)$.
 8. Output 1 if $b' = b$ and 0 otherwise.
 - $H_{b,2}(\lambda)$: In this hybrid, we compute the $\tilde{m}_i$ values differently depending on whether the underlying encodings were copied or not.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$, $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{P}}$.
 4. Compute $c \leftarrow \text{Enc}_{k,\ell}^{\mathcal{P}}(1^\lambda, \text{aux}, m_b)$. Namely, sample $s, \text{hk}, \rho \leftarrow \{0, 1\}^\lambda$, $r \leftarrow \{0, 1\}^\ell$, compute $\text{com} = \mathcal{P}(1^\lambda, \text{hk} || m_b || \rho)$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{com}))$, $\text{pad} = \mathcal{P}(1^{|m_b|+\lambda}, s || r) \oplus (m_b || \rho)$, and set $c = (c', r, \text{pad})$.
 5. Compute $(\tilde{c}_1 = (c'_1, r_1, \text{pad}_1), \dots, \tilde{c}_k = (c'_k, r_k, \text{pad}_k)) \leftarrow M^{\mathcal{P}}(c = (c', r, \text{pad}))$.
 6. For each $i \in [k]$, we compute $\tilde{m}_i$ as follows:
 - If $(c'_i, r_i, \text{pad}_i) = (c', r, \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $(r_i, \text{pad}_i) \neq (r, \text{pad})$, use $(s_i, \text{hk}_i, \text{com}_i) = (s, \text{hk}, \text{com})$ and compute $(\hat{m}_i || \rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s || r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com} = \mathcal{P}(1^\lambda, \text{hk} || \hat{m}_i || \rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 - If $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{com}_i) = \text{Dec}'(c'_i)$ and $(\hat{m}_i || \rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s_i || r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com}_i = \mathcal{P}(1^\lambda, \text{hk}_i || \hat{m}_i || \rho_i)$ and $\tilde{m}_i := \perp$ otherwise.

7. Compute $\text{out}_b = f(\tilde{m}_1, \dots, \tilde{m}_k)$ and $b' \leftarrow D^{\mathcal{P}}(\text{out}_b)$.
 8. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,3}(\lambda)$: In this hybrid, we always set $\tilde{m}_i$ to $\perp$ in the case that $(c'_i, r_i) = (c', r)$ but $\text{pad}_i \neq \text{pad}$.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$, $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{P}}$.
 4. Compute $c \leftarrow \text{Enc}_{k,\ell}^{\mathcal{P}}(1^\lambda, \text{aux}, m_b)$. Namely, sample $s, \text{hk}, \rho \leftarrow \{0, 1\}^\lambda$, $r \leftarrow \{0, 1\}^\ell$, compute $\text{com} = \mathcal{P}(1^\lambda, \text{hk}||m_b||\rho)$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{com}))$, $\text{pad} = \mathcal{P}(1^{|m_b|+\lambda}, s||r) \oplus (m_b||\rho)$, and set $c = (c', r, \text{pad})$.
 5. Compute $(\tilde{c}_1 = (c'_1, r_1, \text{pad}_1), \dots, \tilde{c}_k = (c'_k, r_k, \text{pad}_k)) \leftarrow M^{\mathcal{P}}(c = (c', r, \text{pad}))$.
 6. For each $i \in [k]$, we compute $\tilde{m}_i$ as follows:
 - If $(c'_i, r_i, \text{pad}_i) = (c', r, \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ and $r_i = r$ but $\text{pad}_i \neq \text{pad}$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $r_i \neq r$, use $(s_i, \text{hk}_i, \text{com}_i) = (s, \text{hk}, \text{com})$ and compute $(\hat{m}_i||\rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s||r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com} = \mathcal{P}(1^\lambda, \text{hk}||\hat{m}_i||\rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 - If $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{com}_i) = \text{Dec}'(c'_i)$ and $(\hat{m}_i||\rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s_i||r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com}_i = \mathcal{P}(1^\lambda, \text{hk}_i||\hat{m}_i||\rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 7. Compute $\text{out}_b = f(\tilde{m}_1, \dots, \tilde{m}_k)$ and $b' \leftarrow D^{\mathcal{P}}(\text{out}_b)$.
 8. Output 1 if $b' = b$ and 0 otherwise.
 - $H_{b,4}(\lambda)$: In this hybrid, we sample the value pad randomly and independent of m_b .
 1. Sample $\mathcal{O} \leftarrow \text{RF}$, $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{P}}$.
 4. Compute c as follows. Sample $s, \text{hk}, \rho \leftarrow \{0, 1\}^\lambda$, $r \leftarrow \{0, 1\}^\ell$, compute $\text{com} = \mathcal{P}(1^\lambda, \text{hk}||m_b||\rho)$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{com}))$, $\text{pad} \leftarrow \{0, 1\}^{|m_b|+\lambda}$, and set $c = (c', r, \text{pad})$.
 5. Compute $(\tilde{c}_1 = (c'_1, r_1, \text{pad}_1), \dots, \tilde{c}_k = (c'_k, r_k, \text{pad}_k)) \leftarrow M^{\mathcal{P}}(c = (c', r, \text{pad}))$.
 6. For each $i \in [k]$, we compute $\tilde{m}_i$ as follows:
 - If $(c'_i, r_i, \text{pad}_i) = (c', r, \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ and $r_i = r$ but $\text{pad}_i \neq \text{pad}$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $r_i \neq r$, use $(s_i, \text{hk}_i, \text{com}_i) = (s, \text{hk}, \text{com})$ and compute $(\hat{m}_i||\rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s||r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com} = \mathcal{P}(1^\lambda, \text{hk}||\hat{m}_i||\rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 - If $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{com}_i) = \text{Dec}'(c'_i)$ and $(\hat{m}_i||\rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s_i||r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com}_i = \mathcal{P}(1^\lambda, \text{hk}_i||\hat{m}_i||\rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 7. Compute $\text{out}_b = f(\tilde{m}_1, \dots, \tilde{m}_k)$ and $b' \leftarrow D^{\mathcal{P}}(\text{out}_b)$.

8. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,5}(\lambda)$: In this hybrid, we compute the value com to be independent of the underlying message m_b .
 1. Sample $\mathcal{O} \leftarrow \text{RF}$, $(Z, M, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, \text{aux}, f) \leftarrow Z^{\mathcal{P}}$.
 4. Compute c as follows. Sample $s, \text{hk}, \rho \leftarrow \{0, 1\}^\lambda$, $r \leftarrow \{0, 1\}^\ell$, compute $\text{com} = \mathcal{P}(1^\lambda, \text{hk} || 0^{|m_b|} || \rho)$, $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s, \text{hk}, \text{com}))$, $\text{pad} \leftarrow \{0, 1\}^{|m_b|+\lambda}$, and set $c = (c', r, \text{pad})$.
 5. Compute $(\tilde{c}_1 = (c'_1, r_1, \text{pad}_1), \dots, \tilde{c}_k = (c'_k, r_k, \text{pad}_k)) \leftarrow M^{\mathcal{P}}(c = (c', r, \text{pad}))$.
 6. For each $i \in [k]$, we compute $\tilde{m}_i$ as follows:
 - If $(c'_i, r_i, \text{pad}_i) = (c', r, \text{pad})$ or $\tilde{c}_i$ doesn't specify parameters $1^\lambda, \text{aux}, |m_b|$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ and $r_i = r$ but $\text{pad}_i \neq \text{pad}$, set $\tilde{m}_i := \perp$.
 - If $c'_i = c'$ but $r_i \neq r$, use $(s_i, \text{hk}_i, \text{com}_i) = (s, \text{hk}, \text{com})$ and compute $(\hat{m}_i || \rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s || r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com} = \mathcal{P}(1^\lambda, \text{hk} || \hat{m}_i || \rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 - If $c'_i \neq c'$, compute $(s_i, \text{hk}_i, \text{com}_i) = \text{Dec}'(c'_i)$ and $(\hat{m}_i || \rho_i) = \mathcal{P}(1^{|m_b|+\lambda}, s_i || r_i) \oplus \text{pad}_i$. Set $\tilde{m}_i := \hat{m}_i$ if $\text{com}_i = \mathcal{P}(1^\lambda, \text{hk}_i || \hat{m}_i || \rho_i)$ and $\tilde{m}_i := \perp$ otherwise.
 7. Compute $\text{out}_b = f(\tilde{m}_1, \dots, \tilde{m}_k)$ and $b' \leftarrow D^{\mathcal{P}}(\text{out}_b)$.
 8. Output 1 if $b' = b$ and 0 otherwise.

By assumption, it holds that for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned}
\frac{1}{2} + \frac{1}{p(\lambda)} &< \frac{1}{2} \cdot \Pr[H_{0,0}(\lambda)] + \frac{1}{2} \cdot \Pr[H_{1,0}(\lambda)] \\
&= \frac{1}{2} \cdot \left(\Pr[H_{0,5}(\lambda)] + \sum_{i=1}^5 (\Pr[H_{0,i-1}(\lambda)] - \Pr[H_{0,i}(\lambda)]) \right) \\
&\quad + \frac{1}{2} \cdot \left(\Pr[H_{1,5}(\lambda)] + \sum_{i=1}^5 (\Pr[H_{1,i-1}(\lambda)] - \Pr[H_{1,i}(\lambda)]) \right)
\end{aligned}$$

However, observe that $H_{0,5}$ and $H_{1,5}$ are identically distributed since $|m_0| = |m_1|$, so $\Pr[H_{0,5}(\lambda)] + \Pr[H_{1,5}(\lambda)] = 1$ for all $\lambda \in \mathbb{N}$. It follows that for infinitely many $\lambda \in \mathbb{N}$,

$$\frac{1}{p(\lambda)} < \frac{1}{2} \cdot \sum_{b \in \{0,1\}, i \in [5]} (\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)]).$$

So, there exists a pair of consecutive hybrids such that for some $b \in \{0, 1\}$ and $i \in [5]$,

$$\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)] > \frac{1}{5 \cdot p(\lambda)}.$$

In the claims below, we show that this gives a reduction to the CCA-hiding of $(\text{Enc}', \text{Dec}')$. We give the below claims for $b = 0$ without loss of generality.

Claim 7.8. For all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)]| \leq 1/(5 \cdot p(\lambda))$.

Proof. This follows from the AI-ROM pre-sampling lemma of Lemma 3.3. Namely, hybrid $H_{0,0}(\lambda)$ corresponds to the case where $\mathcal{O}$ is sampled uniformly at random, an unbounded algorithm $A_\lambda(\mathcal{O})$ outputs a description of an oracle algorithm $\text{Exp}^\mathcal{O}$ corresponding to the rest of the hybrid that outputs a single bit. Hybrid $H_{0,1}(\lambda)$ corresponds to the case where instead the experiment receives oracle access to $\mathcal{P}$ pre-sampled at F determined by **Sam** given the description of the algorithm Exp . By construction, F is fixed at $f'(\lambda) = (5 \cdot q'(\lambda) \cdot p(\lambda))^2$ points, where $q'(\lambda)$ is a bound on the description size and number of oracle queries made by Exp . Lemma 3.3 therefore guarantees that the statistical distance between the output bits in both experiments is at most

$$\left(\frac{(q'(\lambda))^2}{2 \cdot f'(\lambda)} \right)^{1/2} = \left(\frac{(q'(\lambda))^2}{2 \cdot (5 \cdot q'(\lambda) \cdot p(\lambda))^2} \right)^{1/2} \leq \frac{1}{5 \cdot p(\lambda)},$$

as required. $\square$

Claim 7.9. Assuming $(\text{Enc}', \text{Dec}')$ satisfies correctness, for all $\lambda \in \mathbb{N}$, it holds that $|\Pr[H_{0,1}(\lambda)] - \Pr[H_{0,2}(\lambda)]| = 0$.

Proof. The only change between $H_{0,1}(\lambda)$ and $H_{0,2}(\lambda)$ is in the cases where some $\tilde{c}_i = (c'_i, r_i, \text{pad}_i)$ satisfies $c'_i = c'$ but $(r_i, \text{pad}_i) \neq (r, \text{pad})$. In $H_{0,1}(\lambda)$, $(s_i, \text{hk}_i, \text{com}_i) = \text{Dec}'(c'_i)$, whereas in $H_{0,2}(\lambda)$, $(s_i, \text{hk}_i, \text{com}_i) := (s, \text{hk}, \text{com})$. As $c' \leftarrow \text{Enc}'(1^\lambda, t, (s, \text{hk}, \text{com}))$ by construction and $c'_i = c'$ in this case, correctness of $(\text{Enc}', \text{Dec}')$ implies that $(s, \text{hk}, \text{com}) = \text{Dec}'(c'_i)$. It follows that the two hybrids are actually identically distributed, so $|\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)]| = 0$, as required. $\square$

Claim 7.10. There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, $|\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)]| \leq \mu(\lambda)$.

Proof. At a high level, this follows from collision-resistance of the function $\mathcal{P}(1^\lambda, \text{hk}||\cdot)$ for $\text{hk} \leftarrow \{0, 1\}^\lambda$ sampled independently of $\mathcal{P}$ in the AI-ROM.

The only change between $H_{0,2}(\lambda)$ and $H_{0,3}(\lambda)$ is in the case where some $\tilde{c}_i$ satisfies $(c'_i, r_i) = (c', r)$ but $\text{pad}_i \neq \text{pad}$. In this case, $\tilde{m}_i$ is always set to $\perp$ in $H_{0,3}(\lambda)$ whereas it is set to $\hat{m}_i \neq \perp$ in $H_{0,2}(\lambda)$ in the event that $\text{com} = \mathcal{P}(1^\lambda, \text{hk}||\hat{m}_i||\rho_i)$. If these experiments differ, we claim that one of the following two cases must occur and then bound the probabilities of each event.

- We say that (E1) holds if F includes any input of the form $(\text{hk}||\cdot)$. Given that (E1) doesn't hold, all queries $\mathcal{P}(1^\lambda, \text{hk}||\cdot)$ can be sampled lazily for the experiment.
- We say that (E2) holds if (E1) doesn't hold and after the experiment queries $\text{com} = \mathcal{P}(1^\lambda, \text{hk}||m_b||\rho)$, it makes another distinct query of the form $\mathcal{P}(1^\lambda, \text{hk}||\cdot)$ that outputs com .

We first argue that if $H_{0,2}(\lambda)$ and $H_{0,3}(\lambda)$ differ, one of (E1) or (E2) must hold. This is because when $(c'_i, r_i) = (c', r)$ but $\text{pad}_i \neq \text{pad}$, it means that $(\hat{m}_i||\rho_i) \neq (m_0||\rho)$ since otherwise $\text{pad}_i = (\hat{m}_i||\rho_i) \oplus \mathcal{P}(1^{|\text{mol}|+\lambda}, s||r) = (m_0||\rho) \oplus \mathcal{P}(1^{|\text{mol}|+\lambda}, s||r) = \text{pad}$, in contradiction. So if (E1) does not hold, both $(\text{hk}||m_0||\rho)$ and $(\text{hk}||\hat{m}_i||\rho_i)$ are distinct, queried by the experiment, can be lazily sampled, and result in a collision on com .

Next, we bound the probability of either (E1) or (E2) occurring. For (E1), since hk is sampled uniformly from $\{0, 1\}^\lambda$ after F is fixed, the probability that any input of the form $(\text{hk}||\cdot)$ is in F is at most $|F|/2^\lambda$. To bound (E2), observe that after $\text{com} \in \{0, 1\}^\lambda$ is fixed, all remaining lazily sampled

queries have a $1/2^\lambda$ probability of returning the value `com`. There are at most $q'(\lambda)$ such queries, so the probability of (E2) given (E1) doesn't hold is at most $q'(\lambda)/2^\lambda$. It follows that either (E1) or (E2) occur with probability at most $\mu(\lambda) = (|F| + q'(\lambda))/2^\lambda$. As $|F| = f'(\lambda) = (5 \cdot q'(\lambda) \cdot p(\lambda))^2$ and q', p are polynomially bounded, it follows that $\mu(\lambda)$ is negligible, as required. $\square$

Claim 7.11. *If $\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)] > 1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}', \text{Dec}')$ does not satisfy CCA-hiding based on a depth-preserving reduction relative to the bounded depth adversary (Z, M) .*

Proof. At a high level, we show that, with overwhelming probability, the output of $\mathcal{P}(1^{|m_0|+\lambda}, s||r)$ can be lazily sampled when used in $\text{Enc}_{k,\ell}^{\mathcal{P}}$ and the remainder of the experiment doesn't query $(s||r)$.

The only difference between $H_{0,3}(\lambda)$ and $H_{0,4}(\lambda)$ is how `pad` is sampled. In $H_{0,3}(\lambda)$, it is sampled as $\mathcal{P}(1^{|m_0|+\lambda}, s||r) \oplus (m_0||\rho)$, and in $H_{0,4}(\lambda)$, it is sampled uniformly at random. If these experiments differ, we claim that one of the following three cases must occur and then bound the probabilities of each event.

- We say that (E1) holds if F includes an input of the form $(s||r)$. Given that (E1) doesn't hold, the query $\mathcal{P}(1^{|m_0|+\lambda}, s||r)$ can be sampled lazily.
- We say that (E2) holds if $D^{\mathcal{P}}$ makes a query to $(s||r)$.
- We say that (E3) holds if (E1) doesn't hold and either $M^{\mathcal{P}}$ makes a query to $(s||r)$ or the experiment needs to make a query to $(s||r)$ when computing $\tilde{m}_i$ for $i \in [k]$ such that $c'_i \neq c'$.

We argue that if $H_{0,3}(\lambda)$ and $H_{0,4}(\lambda)$ differ, one of (E1), (E2), or (E3) must hold. If (E1) doesn't hold, then the output for $(s||r)$ can be lazily sampled and is uniformly random. (E2) and (E3) jointly capture the event that the rest of the experiment ever queries $(s||r)$. This is because the only other queries to $\mathcal{P}$ come from the case where $c'_i = c'$ but $r_i \neq r$, which only queries $\mathcal{P}$ on input $(s||r_i) \neq (s||r)$. So, if neither (E2) or (E3) hold, then `pad` is uniformly random and independent of the rest of the experiment, so $H_{0,3}(\lambda)$ and $H_{0,4}(\lambda)$ are identically distributed.

Next, we bound the probability of either (E1) or (E2) occurring. For (E1), since s, r are sampled randomly and independent of F , it follows that (E1) holds with probability at most $|F|/2^{\lambda+\ell} \leq |F|/2^\lambda$. For (E2), observe that $D^{\mathcal{P}}$ takes inputs y of length at most ℓ and makes at most $q(\lambda)$ oracle queries by assumption. For any fixed oracle algorithm $\mathcal{P}$, the set of all possible queries that $D^{\mathcal{P}}(y)$ might make on any input $y \in \{0,1\}^\lambda$ is at most $2^{\ell+1} \cdot q(\lambda)$. Since s, r are sampled randomly and independent of F and hence $\mathcal{P}$, it follows that (E2) holds with probability at most $2^{\ell+1} \cdot q(\lambda)/2^{\lambda+\ell} \leq 2q(\lambda)/2^\lambda$. Thus, either (E1) or (E2) hold with probability at most $(|F| + 2q(\lambda))/2^\lambda$, which is negligible in λ . In particular, this implies that $\Pr[(E1) \vee (E2)] \leq 1/(10p(\lambda))$ for sufficiently large $\lambda \in \mathbb{N}$, so for infinitely many $\lambda \in \mathbb{N}$, it follows that

$$\Pr[H_{0,3}(\lambda) \wedge (E3)] - \Pr[H_{0,4}(\lambda) \wedge (E3)] > 1/(10p(\lambda)).$$

Given the above, we construct an adversary that breaks the CCA-hiding of $(\text{Enc}', \text{Dec}')$. We do this via the following two steps.

1. We first construct a particular CCA-style adversary that given an encoding $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s^*, \text{hk}, \text{com}))$ for $s^* \leftarrow \{0,1\}^\lambda$ and known `hk`, `com` values as above, outputs s^* with inverse polynomial probability.

2. Second, we show that such an adversary can be used to break CCA-hiding of $(\text{Enc}', \text{Dec}')$.

We start by constructing a non-uniform CCA-style adversary $\{B_\lambda\}_{\lambda \in \mathbb{N}}$ such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} s^* \leftarrow \{0, 1\}^\lambda \\ c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s^*, \text{hk}, \text{com})) : s' = s^* \\ s' \leftarrow B_\lambda^{(\text{Dec}')^*_{c'}(\cdot)}(c') \end{array} \right] > 1/(10p(\lambda) \cdot q'(\lambda)).$$

For each $\lambda \in \mathbb{N}$, we use all associated values from hybrid $H_{0,3}(\lambda)$. The main difference is we will replace the value of s sampled in $\text{Enc}'_{k,\ell}$ with the randomly sampled challenge value s^* .

- B_λ receives $c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s^*, \text{hk}, \text{com}))$ as input and has oracle access to $\text{Dec}'(\cdot)$ for inputs other than c' . B_λ computes $(\tilde{c}_1, \dots, \tilde{c}_k) \leftarrow M^{\mathcal{P}}(c = (c', r, \text{pad}))$ and then computes $\tilde{m}_i$ for each $i \in [k]$ where $c'_i \neq c'$ as in hybrid $H_{0,3}(\lambda)$ using oracle access to $(\text{Dec}')^*_{c'}(\cdot)$. To simulate queries to $\mathcal{P}$, B_λ lazily samples $\mathcal{P}$ given F . B_λ picks a random input of the form $(s' || r)$ that was queried to $\mathcal{P}$ and outputs s' .

Since the experiment makes at most $q'(\lambda)$ queries for some polynomial q' , it follows that B_λ succeeds in guessing s^* with probability at least $1/q'(\lambda)$ times the probability that (E3) occurs. We also established that when there is a difference between hybrids $H_{0,3}(\lambda)$ and $H_{0,4}(\lambda)$, (E3) occurs with at least $1/(10p(\lambda))$ probability. Thus, B_λ succeeds with probability $1/(10p(\lambda) \cdot q'(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, as required.

We now use $\{B_\lambda\}_{\lambda \in \mathbb{N}}$ as constructed above to break CCA-hiding of $(\text{Enc}', \text{Dec}')$. Let $(Z_\lambda, M_\lambda, D_\lambda)$ be the specific adversaries output by $A_\lambda(\mathcal{O})$ on security parameter λ for some fixed random oracle $\mathcal{O}$ such that our initial assumption holds that for infinitely many $\lambda \in \mathbb{N}$, $\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)] > 1/(5p(\lambda))$. We additionally assume for simplicity that, other than the value of $s \leftarrow \{0, 1\}^\lambda$, randomness used by Enc' , and lazily sampled points in $\mathcal{P}$, all other randomness in the hybrid $H_{0,3}(\lambda)$ is fixed such that this assumption holds. We construct a non-uniform adversary $\{(Z'_\lambda, D'_\lambda)\}_{\lambda \in \mathbb{N}}$ against the CCA-hiding of $(\text{Enc}', \text{Dec}')$ as follows:

- Z'_λ samples $s_0, s_1 \leftarrow \{0, 1\}^\lambda$. It then computes $(m_0, m_1, \text{aux}, f) \leftarrow Z_\lambda^{\mathcal{P}}$, simulating $\mathcal{P}$ via lazily sampling given F . Z'_λ uses the fixed value for hk and computes com as in $H_{0,3}(\lambda)$. It outputs (m'_0, m'_1, aux) for $m'_0 = (s_0, \text{hk}, \text{com})$ and $m'_1 = (s_1, \text{hk}, \text{com})$.
- D'_λ receives as input $c' \leftarrow \text{Enc}'(1^\lambda \text{aux}, m'_{b^*})$ for challenge bit $b^* \leftarrow \{0, 1\}$ and has oracle access to Dec' on any input other than c' . D'_λ computes $s' \leftarrow B_\lambda(c')$ while simulating calls to $\text{Dec}'(\cdot)$ with its own oracle. It outputs $b' \in \{0, 1\}$ if $s' = s_{b'}$ and outputs a random bit $b' \leftarrow \{0, 1\}$ otherwise.

In terms of running time, note that Z'_λ runs in polynomial-time in addition to running Z_λ once. D'_λ primarily runs B_λ and otherwise does simple polynomial-time operations. B_λ first runs M while simulating the associated oracle calls to $\mathcal{P}$ by lazy sampling. The oracle calls to $\mathcal{P}$ can be simulated by either returning a consistent response for queries in F or previously queried values, or otherwise lazily sampling the response. This requires size $\text{poly}(\lambda, |F|)$ and linear depth $O(\lambda)$ to maintain the set F and all previously queried values. Next, B_λ computes the necessary $\tilde{m}_i$ values for $i \in [k]$ such that $c'_i \neq c'$. This requires oracle access to $\text{Dec}'(\cdot)$ on inputs different from c' , which D'_λ can simulate for B_λ . B_λ then outputs a value at random from one of the queries made to $\mathcal{P}$.

Thus, D'_λ runs in size $\text{poly}(\lambda, |F|) = \text{poly}(\lambda, q'(\lambda), p(\lambda))$, which is in particular is polynomial in the size of the adversary $(Z_\lambda, M_\lambda, D_\lambda)$ and the success probability $p(\lambda)$. For depth, D'_λ only makes one call to M_λ , can simulate all parallel rounds of oracle queries simultaneously, and otherwise only runs in fixed polynomial-time. So, D'_λ has only fixed polynomial overhead in depth relative to M_λ , so $\text{depth}(D'_\lambda) \leq \text{depth}(M_\lambda) \cdot \text{poly}(\lambda)$. Since it is also the case that $|m'_0| \leq 3\lambda$, it follows that the reduction is depth-preserving as required.

For correctness, we need to analyze

$$\Pr \left[\begin{array}{l} b \leftarrow \{0, 1\} \\ ((s_0, \text{hk}, \text{com}), (s_1, \text{hk}, \text{com}), t) \leftarrow Z'_\lambda \\ c' \leftarrow \text{Enc}'(1^\lambda, \text{aux}, (s_b, \text{hk}, \text{com})) \\ b' \leftarrow (D'_\lambda)^{(\text{Dec}')^*_{c'}(\cdot)}(c') \end{array} : b' = b \right].$$

Fixing the probability event space, we can rewrite the event that $b' = b$ in the following way by construction of D'_λ :

$$\begin{aligned} \Pr[b' = b] &= \Pr[s' = s_b] - \Pr[s' = s_{1-b}] + \frac{1}{2} \cdot \Pr[s' \notin \{s_0, s_1\}] \\ &= \Pr[s' = s_b] - \Pr[s' = s_{1-b}] + \frac{1}{2} \cdot (1 - \Pr[s' = s_b] - \Pr[s' = s_{1-b}]) \\ &= \frac{1}{2} + \frac{1}{2} \cdot \Pr[s' = s_b] - \frac{1}{2} \cdot \Pr[s' = s_{1-b}]. \end{aligned}$$

Since the inputs to B_λ are appropriately distributed, it follows by assumption that $\Pr[s' = s_b] > 1/(10p(\lambda) \cdot q'(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. Additionally, note that $\Pr[s' = s_{1-b}] = 1/2^\lambda$ since s' is independent of s_{1-b} . It follows that, for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned} \Pr[b' = b] &> \frac{1}{2} + \frac{1}{20p(\lambda) \cdot q'(\lambda)} - \frac{1}{2^{\lambda+1}} \\ &\geq \frac{1}{2} + \frac{1}{40p(\lambda) \cdot q'(\lambda)}. \end{aligned}$$

□

Claim 7.12. *There exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, $|\Pr[H_{0,4}(\lambda)] - \Pr[H_{0,5}(\lambda)]| \leq \mu(\lambda)$.*

Proof. At a high level, this follows from the hiding of the function $\mathcal{P}(1^\lambda, \cdot || \rho)$ for $\rho \leftarrow \{0, 1\}^\lambda$ sampled independently of $\mathcal{P}$ and the rest of the experiment in the AI-ROM.

The only change between $H_{0,4}(\lambda)$ and $H_{0,5}(\lambda)$ is how com is computed. In $H_{0,4}(\lambda)$, $\text{com} = \mathcal{P}(1^\lambda, \text{hk} || m_0 || \rho)$, and in $H_{0,5}(\lambda)$, $\text{com} = \mathcal{P}(1^\lambda, \text{hk} || 0^{|m_0|} || \rho)$. We observe that, in these hybrids, ρ is only used to compute com and appears nowhere else. If these experiments differ, we claim that one of the following two cases must occur and then bound the probability of each event.

- We say that (E1) holds if F includes any input of the form $(\cdot || \rho)$. Given that (E1) doesn't hold, all queries $\mathcal{P}(1^\lambda, \cdot || \rho)$ can be sampled lazily for the rest of the experiment.
- We say that (E2) holds if (E1) doesn't hold and the rest of the experiment ever makes another query for an input of the form $(\cdot || \rho)$.

We first argue that if $H_{0,4}(\lambda)$ and $H_{0,5}(\lambda)$ differ, one of (E1) or (E2) must hold. If (E1) doesn't hold, then the experiment must make a query to $\mathcal{P}$ to get the output for a $(\cdot||\rho)$ query. If only one such query is made, its output can be lazily sampled and is uniformly random and identically distributed in both $H_{0,4}$ and $H_{0,5}$. So, at least two such queries to $(\cdot||\rho)$ must be made if the two hybrid experiments differ.

To bound (E1), we again observe that since $\rho \leftarrow \{0,1\}^\lambda$ is sampled independent of F , the probability that any such input is in F is at most $|F|/2^\lambda$. To bound (E2), we again recall that the first output `com` can be lazily sampled and is independent of ρ . The rest of the experiment is therefore independent of the particular value ρ sampled. So, the probability that any of the at most $q'(\lambda)$ queries are of the form $(\cdot||\rho)$ is at most $q'(\lambda)/2^\lambda$. It follows that either (E1) or (E2) occur with probability at most $\mu(\lambda) = (|F| + q'(\lambda))/2^\lambda$, which is negligible as required. $\square$

This completes the proof of Lemma 7.7 $\square$

7.1 Random Oracle Uninstantiability for Fully Homomorphic TLPs

In this section, we exhibit a random oracle uninstantiability result for our transformation of Theorem 7.2. Specifically, we show sufficient conditions for the underlying CCA-hiding time-lock puzzle such that the resulting transformation is malleable for any fixed polynomial-size instantiation H of the random oracle $\mathcal{O}$.

We first define the additional properties needed for the underlying time-lock puzzle. At a high level, we require that the underlying time-lock puzzle is fully homomorphic, similar to the notion introduced by [MT19]. Specifically, we require that there is an associated evaluation algorithm `Eval` that mauls an encoding c for a message m into new (invalid) encodings c^* corresponding to some function $f(m)$ that can be recovered in time roughly t . Since c^* is allowed to be an invalid encoding, this notion is not in conflict with the notion of CCA hiding. We model this via an alternative decoding algorithm `Decalt` for evaluated encodings.

Definition 7.13 (Fully Homomorphic TLPs). Let $\mathcal{C}$ be a class of circuits. We say that a time-lock puzzle (Enc, Dec) is *homomorphic for $\mathcal{C}$* if there exist algorithms $(\text{Eval}, \text{Dec}_{\text{alt}})$ with the following syntax:

- $c^* = \text{Eval}(f, c)$: A deterministic algorithm that on input a circuit $f \in \mathcal{C}$ and encoding $c \in \{0,1\}^*$ outputs a new encoding c^* .
- $m^* = \text{Dec}_{\text{alt}}(c^*)$: A deterministic algorithm that on input an encoding $c^* \in \{0,1\}^*$ outputs a message m^* .

We require that $(\text{Eval}, \text{Dec}_{\text{alt}})$ additionally satisfy the following properties:

- **Evaluation Efficiency:** there exists a polynomial q such that $\text{Eval}(f, c)$ runs in time $q(|f|, |c|)$ and $\text{Dec}_{\text{alt}}(c^*)$ runs in time $t \cdot \text{poly}(|c|)$ for encodings c^* that specify the time bound t .
- **Correctness of Evaluation:** For any $\lambda, t \in \mathbb{N}$, $m \in \{0,1\}^*$, and $f \in \mathcal{C}$, it holds that

$$\Pr \left[\begin{array}{l} c \leftarrow \text{Enc}(1^\lambda, t, m) \\ c^* = \text{Eval}(f, c) \\ m^* = \text{Dec}_{\text{alt}}(c^*) \end{array} : m^* = f(m) \right] = 1.$$

If (Enc, Dec) is homomorphic for the class of all polynomial-size circuits, we say that it is *fully homomorphic*.

We give our formal random oracle uninstantiability result in the following theorem.

Theorem 7.14. *Let $(\text{Enc}', \text{Dec}')$ be a fully homomorphic TLP with associated algorithms $(\text{Eval}, \text{Dec}_{\text{alt}})$. Let H be any polynomial-size hash function. Then the corresponding TLP construction $(\text{Enc}_{1,|m|}^H, \text{Dec}_{1,|m|}^H)$ from Construction 7.4 does not satisfy non-malleability.*

Proof. We construct a mauling attacker $(Z_\lambda, M_\lambda, D_\lambda)$ for each $\lambda \in \mathbb{N}$ and any sufficiently large message length $n_\lambda \in \mathbb{N}$ specified below:

- Z_λ outputs $(m_0, m_1, t_\lambda, f_{\text{id}})$ where $m_0 = 0^{n_\lambda}$, $m_1 = 1^{n_\lambda}$, $t_\lambda \in \mathbb{N}$ is larger than the description of M_λ below, and f_{id} is the identity function of n_λ -bit strings.
- M_λ takes as input an encoding $c = (c', r, \text{pad})$ and does the following:
 - Compute $c^* = \text{Eval}(f_{r, \text{pad}}, c')$ for function $f_{r, \text{pad}}(s, \text{hk}, \text{com})$ that outputs the first bit of $H(1^{n_\lambda + \lambda}, s || r) \oplus \text{pad}$.
 - Output $\tilde{c} \leftarrow \text{Enc}(c^* || 0^{n_\lambda - |c^*|})$. We require that $n_\lambda \geq |c^*|$.
- D_λ receives as input a string $\tilde{m} = c^* || 0^{n_\lambda - |c^*|}$ and outputs $\text{Dec}_{\text{alt}}(c^*)$.

It remains to argue that $(Z_\lambda, M_\lambda, D_\lambda)$ is a valid mauling attacker.

First, in terms of efficiency, we note that $n_\lambda \in \text{poly}(\lambda)$ and independent of t_λ by the efficiency of Eval as $|f_{r, \text{pad}}|$ and $|c'|$ are independent of t_λ . Thus, it also holds that $t_\lambda \in \text{poly}(\lambda)$ by the efficiency of Eval and Enc , and furthermore $|M_\lambda| < t_\lambda$ by construction. Lastly, D_λ runs in time $t_\lambda \cdot \text{poly}(\lambda)$ by the efficiency of Dec_{alt} . It follows that $(Z_\lambda, M_\lambda, D_\lambda)$ has the required efficiency.

In terms of correctness, suppose that $c \leftarrow \text{Enc}(1^\lambda, t, m_b)$ for some bit $b \in \{0, 1\}$. We observe that the value $m^* = \text{Dec}_{\text{alt}}(c^*)$ output by D_λ is equal to $f_{r, \text{pad}}(s, \text{hk}, \text{com})$ by construction. Note that $f_{r, \text{pad}}$ simply unmask the first bit of m_b , which is simply equal to b by construction. It follows that D_λ always outputs $b' = b$ in the non-malleability experiment, so it is a valid attack. $\square$

Discussion of uninstantiability result. We note that Theorem 7.14 gives sufficient conditions for which the transformation of Theorem 7.2 is malleable for any concrete instantiation of the random oracle. As a result, one could assume that the transformation is secure for “natural” TLP candidates that don’t clearly support homomorphic properties. Indeed, existing candidates of fully homomorphic time-lock puzzles seem to rely on strong tools like fully homomorphic encryption or obfuscation [MT19] and are only proven secure with setup in the CRS model, so it is potentially a reasonable assumption that the compiler is secure for non-contrived CCA-hiding TLPs. This is similar to real-world applications of the Fiat-Shamir heuristic for arguments [FS86] or the Fujisaki-Okamoto transform for CCA-secure encryption [FO99], where we know there are concrete counterexamples [Bar01, GK03, BBH⁺19, KRS25, BFM15, GKW17], yet it is believed to still be secure for many practical applications.

Concretely, we observe that the proof of Theorem 7.2 avoids this attack by the extractability property of a random oracle $\mathcal{O}$. Namely, the only way to compute $\mathcal{O}(s || r)$ is by “knowing” s . However, the attack from Theorem 7.14 shows this is not true if you can homomorphically evaluate $H(s || r)$ without knowing s for any polynomial-size instantiation H for $\mathcal{O}$.

To the best of our knowledge, a similar attack does not apply to our non-malleable code of Section 6 as it is proven unconditionally secure in the AI-ROM without any additional assumptions. Furthermore, the implicit underlying code that computes $y_s = \mathcal{O}^{(t)}(s)$ lacks structure and is very unlikely to satisfy any homomorphic properties.

Overall, we view the compiler in this section as a first step towards a more efficient non-malleable TLP construction in the plain model, highlighting many of the technical and definitional challenges involved in using and composing time-lock puzzles.

Acknowledgments

We thank Jesko Dujmovic for helpful discussion on the paper and for first pointing out the uninstantiability result for our construction in the random oracle model.

References

- [AAG⁺16] Divesh Aggarwal, Shashank Agrawal, Divya Gupta, Hemanta K. Maji, Omkant Pandey, and Manoj Prabhakaran. Optimal computational split-state non-malleable codes. In *Theory of Cryptography - 13th International Conference, TCC*, pages 393–417, 2016.
- [ADKO15] Divesh Aggarwal, Yevgeniy Dodis, Tomasz Kazana, and Maciej Obremski. Non-malleable reductions and applications. In *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC*, pages 459–468, 2015.
- [ADL18] Divesh Aggarwal, Yevgeniy Dodis, and Shachar Lovett. Non-malleable codes from additive combinatorics. *SIAM J. Comput.*, 47(2):524–546, 2018.
- [AGM⁺15] Shashank Agrawal, Divya Gupta, Hemanta K. Maji, Omkant Pandey, and Manoj Prabhakaran. A rate-optimizing compiler for non-malleable codes against bit-wise tampering and permutations. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *Theory of Cryptography - 12th Theory of Cryptography Conference, TCC 2015, Warsaw, Poland, March 23-25, 2015, Proceedings, Part I*, volume 9014 of *Lecture Notes in Computer Science*, pages 375–397. Springer, 2015.
- [Ahr24] Knud Ahrens. SIGNITC: supersingular isogeny graph non-interactive timed commitments. *IACR Cryptol. ePrint Arch.*, page 1225, 2024.
- [AKO⁺22] Divesh Aggarwal, Bhavana Kanukurthi, Sai Lakshmi Bhavana Obbattu, Maciej Obremski, and Sruthi Sekar. Rate one-third non-malleable codes. In Stefano Leonardi and Anupam Gupta, editors, *STOC ’22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*, pages 1364–1377. ACM, 2022.
- [ALP⁺24] Andreea B. Alexandru, Julian Loss, Charalampos Papamanthou, Giorgos Tsimos, and Benedikt Wagner. Sublinear-round broadcast without trusted setup. *IACR Cryptol. ePrint Arch.*, page 770, 2024.
- [Bar01] Boaz Barak. How to go beyond the black-box simulation barrier. In *42nd Annual Symposium on Foundations of Computer Science, FOCS*, pages 106–115, 2001.

- [Bar02] Boaz Barak. Constant-round coin-tossing with a man in the middle or realizing the shared random string model. In *43rd Symposium on Foundations of Computer Science FOCS*, pages 345–355, 2002.
- [BBH⁺19] James Bartusek, Liron Bronfman, Justin Holmgren, Fermi Ma, and Ron D. Rothblum. On the (in)security of kilian-based snargs. In *TCC (2)*, volume 11892 of *Lecture Notes in Computer Science*, pages 522–551. Springer, 2019.
- [BCL⁺20] Marshall Ball, Eshan Chattopadhyay, Jyun-Jie Liao, Tal Malkin, and Li-Yang Tan. Non-malleability against polynomial tampering. In *CRYPTO (3)*, volume 12172 of *Lecture Notes in Computer Science*, pages 97–126. Springer, 2020.
- [BDD⁺21] Carsten Baum, Bernardo David, Rafael Dowsley, Jesper Buus Nielsen, and Sabine Oechsner. TARDIS: A foundation of time-lock puzzles in UC. In *EUROCRYPT (3)*, volume 12698 of *Lecture Notes in Computer Science*, pages 429–459. Springer, 2021.
- [BDG⁺18] Marshall Ball, Dana Dachman-Soled, Siyao Guo, Tal Malkin, and Li-Yang Tan. Non-malleable codes for small-depth circuits. In *59th IEEE Annual Symposium on Foundations of Computer Science, FOCS*, pages 826–837, 2018.
- [BDK⁺19] Marshall Ball, Dana Dachman-Soled, Mukul Kulkarni, Huijia Lin, and Tal Malkin. Non-malleable codes against bounded polynomial time tampering. In *Advances in Cryptology - EUROCRYPT*, pages 501–530, 2019.
- [BDKM16a] Marshall Ball, Dana Dachman-Soled, Mukul Kulkarni, and Tal Malkin. Non-malleable codes for bounded depth, bounded fan-in circuits. In *Advances in Cryptology - EUROCRYPT*, pages 881–908, 2016.
- [BDKM16b] Marshall Ball, Dana Dachman-Soled, Mukul Kulkarni, and Tal Malkin. Non-malleable codes for bounded depth, bounded fan-in circuits. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 881–908. Springer, 2016.
- [BDKM18] Marshall Ball, Dana Dachman-Soled, Mukul Kulkarni, and Tal Malkin. Non-malleable codes from average-case hardness: AC^0 , decision trees, and streaming space-bounded tampering. In *Advances in Cryptology - EUROCRYPT*, pages 618–650, 2018.
- [BDL22] Marshall Ball, Dana Dachman-Soled, and Julian Loss. (nondeterministic) hardness vs. non-malleability. In *CRYPTO (1)*, volume 13507 of *Lecture Notes in Computer Science*, pages 148–177. Springer, 2022.
- [BFM15] Christina Brzuska, Pooya Farshim, and Arno Mittelbach. Random-oracle uninstantiability from indistinguishability obfuscation. In *TCC (2)*, volume 9015 of *Lecture Notes in Computer Science*, pages 428–455. Springer, 2015.
- [BGJ⁺16] Nir Bitansky, Shafi Goldwasser, Abhishek Jain, Omer Paneth, Vinod Vaikuntanathan, and Brent Waters. Time-lock puzzles from randomized encodings. In *ITCS*, 2016.

- [BGW19] Marshall Ball, Siyao Guo, and Daniel Wichs. Non-malleable codes for decision trees. In *Advances in Cryptology - CRYPTO*, pages 413–434, 2019.
- [BHP17] Zvika Brakerski, Shai Halevi, and Antigoni Polychroniadou. Four round secure computation without setup. In *TCC (1)*, volume 10677 of *Lecture Notes in Computer Science*, pages 645–677. Springer, 2017.
- [BL18] Nir Bitansky and Huijia Lin. One-message zero knowledge and non-malleable commitments. In *Theory of Cryptography - 16th International Conference, TCC*, pages 209–234, 2018.
- [BN00] Dan Boneh and Moni Naor. Timed commitments. In *CRYPTO*, volume 1880 of *Lecture Notes in Computer Science*, pages 236–254. Springer, 2000.
- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security*, pages 62–73. ACM, 1993.
- [BSS24] Marshall Ball, Ronen Shaltiel, and Jad Silbak. Non-malleable codes with optimal rate for poly-size circuits. In *EUROCRYPT (4)*, volume 14654 of *Lecture Notes in Computer Science*, pages 33–54. Springer, 2024.
- [CFV19] Sandro Coretti, Antonio Faonio, and Daniele Venturi. Rate-optimizing compilers for continuously non-malleable codes. In *ACNS*, volume 11464 of *Lecture Notes in Computer Science*, pages 3–23. Springer, 2019.
- [CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *STOC*, pages 209–218. ACM, 1998.
- [CJ23] Peter Chvojka and Tibor Jager. Simple, fast, efficient, and tightly-secure non-malleable non-interactive timed commitments. In *Public Key Cryptography (1)*, volume 13940 of *Lecture Notes in Computer Science*, pages 500–529. Springer, 2023.
- [CL17] Eshan Chattopadhyay and Xin Li. Non-malleable codes and extractors for small-depth circuits, and affine functions. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC*, pages 1171–1184, 2017.
- [CLP10] Ran Canetti, Huijia Lin, and Rafael Pass. Adaptive hardness and composable security in the plain model from standard assumptions. In *FOCS*, pages 541–550. IEEE Computer Society, 2010.
- [COSV16] Michele Ciampi, Rafail Ostrovsky, Luisa Siniscalchi, and Ivan Visconti. Concurrent non-malleable commitments (and more) in 3 rounds. In *Advances in Cryptology - CRYPTO*, pages 270–299, 2016.
- [COSV17] Michele Ciampi, Rafail Ostrovsky, Luisa Siniscalchi, and Ivan Visconti. Four-round concurrent non-malleable commitments from one-way functions. In *Advances in Cryptology - CRYPTO*, pages 127–157, 2017.
- [Dam89] Ivan Damgård. A design principle for hash functions. In *Advances in Cryptology - CRYPTO*, volume 435, pages 416–427, 1989.

- [DDN91] Danny Dolev, Cynthia Dwork, and Moni Naor. Non-malleable cryptography (extended abstract). In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing, STOC*, pages 542–552, 1991.
- [DKP21] Dana Dachman-Soled, Ilan Komargodski, and Rafael Pass. Non-malleable codes for bounded parallel-time tampering. In *Advances in Cryptology - CRYPTO*, pages 535–565, 2021.
- [DPW10] Stefan Dziembowski, Krzysztof Pietrzak, and Daniel Wichs. Non-malleable codes. In *Innovations in Computer Science - ICS*, pages 434–452, 2010.
- [Eth] Blockchain data storage. <https://ethereum.org/en/developers/docs/data-availability/blockchain-data-storage-strategies/#storage>. Accessed: 2025-02-11.
- [ETY24] Karim Eldefrawy, Benjamin Terner, and Moti Yung. Composing timed cryptographic protocols: Foundations and applications. *IACR Cryptol. ePrint Arch.*, page 676, 2024.
- [FHMV17] Sebastian Faust, Kristina Hostáková, Pratyay Mukherjee, and Daniele Venturi. Non-malleable codes for space-bounded tampering. In *Advances in Cryptology - CRYPTO*, pages 95–126, 2017.
- [FKPS21] Cody Freitag, Ilan Komargodski, Rafael Pass, and Naomi Sirkin. Non-malleable time-lock puzzles and applications. In *Theory of Cryptography - 19th International Conference, TCC*, pages 447–479, 2021.
- [FMVW16] Sebastian Faust, Pratyay Mukherjee, Daniele Venturi, and Daniel Wichs. Efficient non-malleable codes and key derivation for poly-size tampering circuits. *IEEE Trans. Information Theory*, 62(12):7179–7194, 2016.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Advances in Cryptology - CRYPTO*, pages 537–554, 1999.
- [FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *J. Cryptology*, 26(1):80–101, 2013.
- [FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986.
- [GK03] Shafi Goldwasser and Yael Tauman Kalai. On the (in)security of the fiat-shamir paradigm. In *44th Symposium on Foundations of Computer Science (FOCS 2003)*, pages 102–113, 2003.
- [GKLW21] Rachit Garg, Dakshita Khurana, George Lu, and Brent Waters. Black-box non-interactive non-malleable commitments. In *EUROCRYPT (3)*, volume 12698 of *Lecture Notes in Computer Science*, pages 159–185. Springer, 2021.

- [GKLW23] Rachit Garg, Dakshita Khurana, George Lu, and Brent Waters. On non-uniform security for black-box non-interactive CCA commitments. In *EUROCRYPT (1)*, volume 14004 of *Lecture Notes in Computer Science*, pages 173–204. Springer, 2023.
- [GKW17] Rishab Goyal, Venkata Koppula, and Brent Waters. Lockable obfuscation. In *FOCS*, pages 612–621. IEEE Computer Society, 2017.
- [GLOV12] Vipul Goyal, Chen-Kuei Lee, Rafail Ostrovsky, and Ivan Visconti. Constructing non-malleable commitments: A black-box approach. In *53rd Annual IEEE Symposium on Foundations of Computer Science, FOCS*, pages 51–60, 2012.
- [GLP⁺15] Vipul Goyal, Huijia Lin, Omkant Pandey, Rafael Pass, and Amit Sahai. Round-efficient concurrently composable secure computation via a robust extraction lemma. In *TCC (1)*, volume 9014 of *Lecture Notes in Computer Science*, pages 260–289. Springer, 2015.
- [GMW17] Divya Gupta, Hemanta K. Maji, and Mingyuan Wang. Constant-rate non-malleable codes in the split-state model. *IACR Cryptol. ePrint Arch.*, page 1048, 2017.
- [GMW19] Divya Gupta, Hemanta K. Maji, and Mingyuan Wang. Explicit rate-1 non-malleable codes for local tampering. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 435–466. Springer, 2019.
- [Goy11] Vipul Goyal. Constant round non-malleable protocols using one way functions. In Lance Fortnow and Salil P. Vadhan, editors, *Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC*, pages 695–704, 2011.
- [GPR16] Vipul Goyal, Omkant Pandey, and Silas Richelson. Textbook non-malleable commitments. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC*, pages 1128–1141, 2016.
- [GR19] Vipul Goyal and Silas Richelson. Non-malleable commitments using goldreich-levin list decoding. In *FOCS*, pages 686–699. IEEE Computer Society, 2019.
- [Khu17] Dakshita Khurana. Round optimal concurrent non-malleability from polynomial hardness. In *Theory of Cryptography - 15th International Conference, TCC*, pages 139–171, 2017.
- [Khu21] Dakshita Khurana. Non-interactive distributional indistinguishability (NIDI) and non-malleable commitments. In *EUROCRYPT (3)*, volume 12698 of *Lecture Notes in Computer Science*, pages 186–215. Springer, 2021.
- [Kiy14] Susumu Kiyoshima. Round-efficient black-box construction of composable multi-party computation. In *CRYPTO (2)*, volume 8617 of *Lecture Notes in Computer Science*, pages 351–368. Springer, 2014.
- [KK19] Yael Tauman Kalai and Dakshita Khurana. Non-interactive non-malleability from quantum supremacy. In *Advances in Cryptology - CRYPTO*, pages 552–582, 2019.

- [KLX20] Jonathan Katz, Julian Loss, and Jiayu Xu. On the security of time-lock puzzles and timed commitments. In *TCC (3)*, volume 12552 of *Lecture Notes in Computer Science*, pages 390–413. Springer, 2020.
- [KOS17] Bhavana Kanukurthi, Sai Lakshmi Bhavana Obbattu, and Sruthi Sekar. Four-state non-malleable codes with explicit constant rate. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part II*, volume 10678 of *Lecture Notes in Computer Science*, pages 344–375. Springer, 2017.
- [KOS18] Bhavana Kanukurthi, Sai Lakshmi Bhavana Obbattu, and Sruthi Sekar. Non-malleable randomness encoders and their applications. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 589–617. Springer, 2018.
- [KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. How to prove false statements: Practical attacks on fiat-shamir. In *CRYPTO (6)*, volume 16005 of *Lecture Notes in Computer Science*, pages 3–26. Springer, 2025.
- [KS17] Dakshita Khurana and Amit Sahai. How to achieve non-malleability in one or two rounds. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS*, pages 564–575, 2017.
- [Lam79] Leslie Lamport. Constructing digital signatures from a one-way function. Technical report, 1979.
- [LL12] Feng-Hao Liu and Anna Lysyanskaya. Tamper and leakage resilience in the split-state model. In *Advances in Cryptology - CRYPTO*, pages 517–532, 2012.
- [LP09] Huijia Lin and Rafael Pass. Non-malleability amplification. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC*, pages 189–198, 2009.
- [LP11] Huijia Lin and Rafael Pass. Constant-round non-malleable commitments from any one-way function. In *Proceedings of the 43rd ACM Symposium on Theory of Computing, STOC*, pages 705–714, 2011.
- [LP12] Huijia Lin and Rafael Pass. Black-box constructions of composable protocols without set-up. In *CRYPTO*, volume 7417 of *Lecture Notes in Computer Science*, pages 461–478. Springer, 2012.
- [LPS17] Huijia Lin, Rafael Pass, and Pratik Soni. Two-round and non-interactive concurrent non-malleable commitments from time-lock puzzles. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS*, pages 576–587. IEEE Computer Society, 2017.
- [LPV08] Huijia Lin, Rafael Pass, and Muthuramakrishnan Venkitasubramaniam. Concurrent non-malleable commitments from any one-way function. In *Theory of Cryptography, Fifth Theory of Cryptography Conference, TCC*, pages 571–588, 2008.

- [Mer87] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology - CRYPTO*, pages 369–378, 1987.
- [Mer89] Ralph C. Merkle. A certified digital signature. In *Advances in Cryptology - CRYPTO*, pages 218–238, 1989.
- [MMV11] Mohammad Mahmoody, Tal Moran, and Salil P. Vadhan. Time-lock puzzles in the random oracle model. In *CRYPTO*, volume 6841 of *Lecture Notes in Computer Science*, pages 39–50. Springer, 2011.
- [MT19] Giulio Malavolta and Sri Aravinda Krishnan Thyagarajan. Homomorphic time-lock puzzles and applications. In *Advances in Cryptology - CRYPTO*, pages 620–649, 2019.
- [NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *STOC*, pages 427–437. ACM, 1990.
- [Pas13] Rafael Pass. Unprovable security of perfect NIZK and non-interactive non-malleable commitments. In *TCC*, volume 7785 of *Lecture Notes in Computer Science*, pages 334–354. Springer, 2013.
- [PPV08] Omkant Pandey, Rafael Pass, and Vinod Vaikuntanathan. Adaptive one-way functions and applications. In *Advances in Cryptology - CRYPTO*, pages 57–74, 2008.
- [PR05a] Rafael Pass and Alon Rosen. Concurrent non-malleable commitments. In *46th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 563–572, 2005.
- [PR05b] Rafael Pass and Alon Rosen. New and improved constructions of non-malleable cryptographic protocols. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing, STOC*, pages 533–542, 2005.
- [PW10] Rafael Pass and Hoeteck Wee. Constant-round non-malleable commitments from sub-exponential one-way functions. In *Advances in Cryptology - EUROCRYPT*, 2010.
- [RS91] Charles Rackoff and Daniel R. Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In *CRYPTO*, volume 576 of *Lecture Notes in Computer Science*, pages 433–444. Springer, 1991.
- [RSW96] Ronald L Rivest, Adi Shamir, and David A Wagner. Time-lock puzzles and timed-release crypto. 1996.
- [SS24] Ronen Shaltiel and Jad Silbak. Explicit codes for poly-size circuits and functions that are hard to sample on low entropy distributions. In *STOC*, pages 2028–2038. ACM, 2024.
- [TAF⁺23] Nirvan Tyagi, Arasu Arun, Cody Freitag, Riad Wahby, Joseph Bonneau, and David Mazières. Riggs: Decentralized sealed-bid auctions. In *CCS*, pages 1227–1241. ACM, 2023.
- [TCLM21] Sri Aravinda Krishnan Thyagarajan, Guilhem Castagnos, Fabien Laguillaumie, and Giulio Malavolta. Efficient CCA timed commitments in class groups. In *CCS*, pages 2663–2684. ACM, 2021.

- [Unr07] Dominique Unruh. Random oracles and auxiliary input. In *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, pages 205–223, 2007.
- [Wee10] Hoeteck Wee. Black-box, round-efficient secure computation via non-malleability amplification. In *FOCS*, 2010.

A CCA-Hiding Time-Lock Puzzle in the AI-ROM

We show that a slight modification of the AI-ROM construction of a non-malleable TLP from [FKPS21] satisfies CCA-hiding.

Theorem A.1 (CCA-Hiding Time-Lock Puzzle in the AI-ROM). *Assuming the existence of time-lock puzzles, there exists a CCA-hiding time-lock puzzle in the AI-ROM.*

We note that the associated theorem statement from [FKPS21] (Theorem 4.2) requires sub-exponential security for the underlying TLP to achieve non-malleability (for arbitrary length messages), but our theorem above requires only polynomial security to achieve CCA-hiding. Briefly, this comes from the step in their proof that “flattens” the distinguisher, resulting in an exponential-size but fixed depth distinguisher. This step isn’t needed for CCA-hiding where the distinguisher already has bounded polynomial-depth by definition.

To prove Theorem A.1, we give the TLP construction $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ from [FKPS21] that relies on an underlying TLP $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ and a common random oracle $\mathcal{O}$. The main idea is to use Enc_{tlp} to encode the message m along with a random string r . The encoding algorithm uses $\mathcal{O}(m||r)$ as the randomness for the Enc_{tlp} encoding of m and r . This is reminiscent of the Fujisaki-Okamoto (FO) transform [FO13], and intuitively requires that you must “know” the underlying value m for any encoding you produce. We formalize this in Construction A.2.

Construction A.2. CCA-Hiding TLP in the AI-ROM

Ingredients: Let $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ be a TLP (Definition 4.9) and $\mathcal{O}$ be a common random function (formalized in Section 3.3). We assume without loss of generality that $\text{Enc}_{\text{tlp}}(1^\lambda, \cdot, \cdot)$ uses at most λ bits of randomness.

Construction: We construct a TLP $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ as follows:

- $c \leftarrow \text{Enc}^{\mathcal{O}}(1^\lambda, t, m; r)$:
 1. Sample $r' \leftarrow \mathcal{O}(1^\lambda, m||r)$
 2. Output $c = \text{Enc}_{\text{tlp}}(1^\lambda, t, (m||r); r')$
- $m = \text{Dec}^{\mathcal{O}}(c)$:
 1. Compute $(\hat{m}||r) = \text{Dec}_{\text{tlp}}(c)$.
 2. Output $m := \hat{m}$ if $c = \text{Enc}^{\mathcal{O}}(1^\lambda, t, \hat{m}; r)$, and output $m := \perp$ otherwise.

We refer to Lemmas 4.4 and 4.5 in [FKPS21] for proofs of correctness and efficiency, respectively, of $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ in the AI-ROM. We proceed to prove CCA-hiding holds for $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ via a reduction to (plain) hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$. We note that the following proof is similar to the proof

of functional non-malleability for Lemma 4.10 in [FKPS21]. We give a slightly modified version of the proof for completeness to highlight the difference for CCA-hiding compared to non-malleability.

Lemma A.3 (CCA-Hiding). *Assuming $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ satisfies hiding, $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ is a CCA-hiding TLP in the AI-ROM.*

Proof. Suppose by way of contradiction that $(\text{Enc}^{\mathcal{O}}, \text{Dec}^{\mathcal{O}})$ does not satisfy bounded depth CCA-hiding in the AI-ROM. Then, for every positive monotonic polynomial α , there exists a non-uniform algorithm $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$ that outputs α -gap bounded polynomial depth adversaries for the CCA-hiding game, and polynomials p, q , such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} \mathcal{O} \leftarrow \text{RF} \\ (Z, D) \leftarrow A_\lambda(\mathcal{O}) \\ (m_0, m_1, t) \leftarrow Z_\lambda^{\mathcal{O}} \\ b \leftarrow \{0, 1\} \\ c \leftarrow \text{Enc}^{\mathcal{O}}(1^\lambda, t, m_b) \\ b' \leftarrow D_\lambda^{\mathcal{O}, (\text{Dec}^{\mathcal{O}})^*(\cdot)}(c) \end{array} : \begin{array}{l} b = b' \\ \wedge |m_0| = |m_1| \\ \wedge \alpha(\lambda, |m_0, m_1|) \leq t \leq q(\lambda) \\ \wedge \text{size}(Z), \text{size}(D) \leq q(\lambda) \\ \wedge \text{depth}(D) \leq t/\alpha(\lambda, |m_0, m_1|) \end{array} \right] > \frac{1}{2} + \frac{1}{p(\lambda)}.$$

For given polynomial α and fixed adversary A , we assume without loss of generality that $|m_0| = |m_1|$ and t, Z, D have the correct efficiency required. We define the following sequence of hybrid experiments for every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$.

- $H_{b,0}(\lambda)$: In the first hybrid, we fix a value for the challenge bit b in the CCA-hiding game.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Compute $(m_0, m_1, t) \leftarrow Z_\lambda^{\mathcal{O}}$.
 3. Sample $c \leftarrow \text{Enc}^{\mathcal{O}}(1^\lambda, t, m_b)$.
 4. Compute $b' \leftarrow D_\lambda^{\mathcal{O}, (\text{Dec}^{\mathcal{O}})^*(\cdot)}(c)$.
 5. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,1}(\lambda)$: In this hybrid we swap the random oracle $\mathcal{O}$ with a pre-sampled oracle $\mathcal{P}$. In more detail, let $\text{Exp}^{\mathcal{O}}$ denote the remainder of the experiment consisting of (Z, D) output by $A_\lambda(\mathcal{O})$ and oracle machines Enc, Dec . Let $q'(\lambda)$ be a bound on the number of random oracle queries made by $\text{Exp}^{\mathcal{O}}$ which is polynomially bounded by the efficiency of Z, D, Enc , and Dec . We set $f'(\lambda) = (4 \cdot q'(\lambda) \cdot p(\lambda))^2$, and let Sam be the algorithm from Lemma 3.3 that on input $\text{Exp}^{\mathcal{O}}$ that makes at most $q'(\lambda)$ queries outputs a partial assignment F of $f'(\lambda)$ points.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^{\mathcal{O}})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z_\lambda^{\mathcal{P}}$.
 4. Sample $c \leftarrow \text{Enc}^{\mathcal{P}}(1^\lambda, t, m_b)$.
 5. Compute $b' \leftarrow D_\lambda^{\mathcal{P}, (\text{Dec}^{\mathcal{P}})^*(\cdot)}(c)$.
 6. Output 1 if $b' = b$ and 0 otherwise.

- $H_{b,2}(\lambda)$: In this hybrid, we simulate the decoding oracle calls made by D_λ via the following algorithm Sim that observes the oracle queries made by Exp and has the challenge encoding c and pre-sampled set F hardcoded. On input a tampered encoding $\tilde{c}_i$, Sim outputs $\perp$ if $\tilde{c}_i = c$. Otherwise, for every input of the form $(\hat{m}||\hat{r}) \in \{0,1\}^{|m_b|+\lambda}$ that has either been queried to $\mathcal{P}$ or is in F , Sim checks if $\tilde{c}_i = \text{Enc}^\mathcal{P}(1^\lambda, t, \hat{m}; \hat{r})$. If such an input $(\hat{m}||\hat{r})$ exists, it outputs $\hat{m}$, and otherwise outputs $\perp$.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^\mathcal{O})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z_\lambda^\mathcal{P}$.
 4. Sample $c \leftarrow \text{Enc}^\mathcal{P}(1^\lambda, t, m_b)$
 5. Compute $b' \leftarrow D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}(c)$.
 6. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,3}(\lambda)$: In this hybrid, we use fresh randomness to sample the challenge TLP c .
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^\mathcal{O})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z_\lambda^\mathcal{P}$.
 4. Compute c as follows. Sample $r \leftarrow \{0,1\}^\lambda$ and $c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, (m_b||r))$.
 5. Compute $b' \leftarrow D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}(c)$.
 6. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,4}(\lambda)$: In this hybrid, we replace the message m_b with an unrelated value when computing the challenge TLP c .
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^\mathcal{O})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z_\lambda^\mathcal{P}$.
 4. Compute c as follows. Sample $r \leftarrow \{0,1\}^\lambda$ and $c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, (0^{|m_b|}||r))$.
 5. Compute $b' \leftarrow D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}(c)$.
 6. Output 1 if $b' = b$ and 0 otherwise.

By assumption, it holds that for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned}
 \frac{1}{2} + \frac{1}{p(\lambda)} &< \frac{1}{2} \cdot \Pr[H_{0,0}(\lambda)] + \frac{1}{2} \cdot \Pr[H_{1,0}(\lambda)] \\
 &= \frac{1}{2} \cdot \left(\Pr[H_{0,4}(\lambda)] + \sum_{i=1}^4 (\Pr[H_{0,i-1}(\lambda)] - \Pr[H_{0,i}(\lambda)]) \right) \\
 &\quad + \frac{1}{2} \cdot \left(\Pr[H_{1,4}(\lambda)] + \sum_{i=1}^4 (\Pr[H_{1,i-1}(\lambda)] - \Pr[H_{1,i}(\lambda)]) \right)
 \end{aligned}$$

Observe that $H_{0,4}$ and $H_{1,4}$ are identically distributed since $|m_0| = |m_1|$, so $\Pr[H_{0,4}(\lambda)] + \Pr[H_{1,4}(\lambda)] = 1$ for all $\lambda \in \mathbb{N}$. It follows for infinitely many $\lambda \in \mathbb{N}$,

$$\frac{1}{p(\lambda)} < \frac{1}{2} \sum_{b \in \{0,1\}, i \in [4]} (\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)]).$$

So, there exists a pair of consecutive hybrids such that for some $b \in \{0,1\}$ and $i \in [4]$,

$$\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)] > \frac{1}{4 \cdot p(\lambda)}.$$

In the claims below, we show that this leads to a contradiction. We give the below claims for $b = 0$ without loss of generality.

Claim A.4. *For all $\lambda \in \mathbb{N}$ it holds that $\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)] \leq 1/(4 \cdot p(\lambda))$.*

Proof. This follows from the AI-ROM pre-sampling lemma of Lemma 3.3. Hybrid $H_{0,0}(\lambda)$ corresponds to the case where $\mathcal{O}$ is a random function and $A_\lambda(\mathcal{O})$ outputs a description of the rest of the experiment $\text{Exp}^\mathcal{O}$. Hybrid $H_{0,1}(\lambda)$ corresponds to the case where instead the experiment receives oracle access to $\mathcal{P}$ pre-sampled at F determined by **Sam** given the description of **Exp**. F is fixed at $f'(\lambda) = (3 \cdot q'(\lambda) \cdot p(\lambda))^2$ points, where $q'(\lambda)$ is a bound on the description size and number of oracle queries made by **Exp**. It follows from Lemma 3.3 that the statistical distance between the output bits in both experiments is at most

$$\left(\frac{(q'(\lambda))^2}{2 \cdot f'(\lambda)} \right)^{1/2} = \left(\frac{(q'(\lambda))^2}{2 \cdot (4 \cdot q'(\lambda) \cdot p(\lambda))^2} \right)^{1/2} \leq \frac{1}{4 \cdot p(\lambda)},$$

as required. $\square$

To prove indistinguishability of hybrids $H_{0,1}(\lambda), H_{0,2}(\lambda)$, we rely on the following claim from [FKPS21].

Claim A.5 (Subclaim 4.14 from [FKPS21]). *Assuming hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$, for every non-uniform PPT $Z = \{Z_\lambda\}_{\lambda \in \mathbb{N}}$, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that*

$$\Pr \left[\begin{array}{l} (m^*, c^*, t) \leftarrow Z_\lambda \\ r \leftarrow \{0,1\}^\lambda \end{array} : c^* = \text{Enc}_{\text{tlp}}(1^\lambda, t, m^*; r) \right] \leq \text{negl}(\lambda).$$

Claim A.6. *If $\Pr[H_{0,1}(\lambda)] - \Pr[H_{0,2}(\lambda)] > \frac{1}{4 \cdot p(\lambda)}$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ does not satisfy hiding.*

Proof. We prove this claim by an additional hybrid argument over the number of $\text{Dec}_c^*(\cdot)$ queries made by D_λ , which is at most $\text{size}(D_\lambda) \leq q(\lambda)$. Specifically, we define the following hybrids for each $j \in [0, q(\lambda)]$.

- $H_{0,1,j}(\lambda)$: This hybrid responds to the first j queries that D_λ makes to $\text{Dec}_c^*(\cdot)$ using **Sim**.
 1. Sample $\mathcal{O} \leftarrow \text{RF}$ and $(Z, D) \leftarrow A_\lambda(\mathcal{O})$.
 2. Let $F = \text{Sam}(\text{Exp}^\mathcal{O})$ and $\mathcal{P} \leftarrow \text{RF}[F]$.
 3. Compute $(m_0, m_1, t) \leftarrow Z_\lambda^\mathcal{P}$.
 4. Sample $c \leftarrow \text{Enc}^\mathcal{P}(1^\lambda, t, m_b)$.

5. Let b' be the output of D_λ while responding to the i th query $\tilde{c}_i$ as follows:

- If $i \leq j$, respond with $\tilde{m}_i \leftarrow \text{Sim}(\tilde{c}_i)$.
- If $i > j$, respond with $\tilde{m}_i \leftarrow (\text{Dec}^\mathcal{P})^*_c(\tilde{c}_i)$.

6. Output 1 if $b' = b$ and 0 otherwise.

Note that $H_{0,1,0}(\lambda) \equiv H_{0,1}(\lambda)$ and $H_{0,1,q(\lambda)} \equiv H_{0,2}(\lambda)$, so there must exist an index $j \in [q(\lambda)]$ such that

$$\Pr[H_{0,1,j-1}] - \Pr[H_{0,1,j}(\lambda)] > 1/(4 \cdot q(\lambda) \cdot p(\lambda)).$$

Fix such an index j . We will show that this contradicts the hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$.

Hybrids $H_{0,1,j-1}(\lambda)$ and $H_{0,1,j}(\lambda)$ differ only in the value $\tilde{m}_j$ computed as a response to the decoding oracle query for $\tilde{c}_j$. Let $\tilde{m} \leftarrow (\text{Dec}^\mathcal{P})^*_c(\tilde{c}_j)$ denote the value computed in $H_{0,1,j-1}(\lambda)$ using Dec , and let $\tilde{m}_S \leftarrow \text{Sim}(\tilde{c}_j)$ denote the value computed in $H_{0,1,j}(\lambda)$ using Sim . It follows that

$$\Pr[H_{0,1,j-1}(\lambda)] - \Pr[H_{0,1,j}(\lambda)] = \Pr[\tilde{m}_S \neq \tilde{m}] > 1/(4 \cdot q(\lambda) \cdot p(\lambda)).$$

To analyze $\Pr[\tilde{m}_S \neq \tilde{m}]$, we consider the following two cases:

- If $\tilde{m}_S \neq \perp$, it holds that $\tilde{c}_j = \text{Enc}^\mathcal{P}(1^\lambda, t, \tilde{m}_S; \hat{r})$ for some $\hat{r}$ by construction of Sim , so $\tilde{m}_S = \tilde{m}$ by correctness of $(\text{Enc}^\mathcal{P}, \text{Dec}^\mathcal{P})$. It follows that

$$\Pr[\tilde{m}_S \neq \tilde{m} \mid \tilde{m}_S \neq \perp] = 0.$$

- Otherwise, if $\tilde{m}_S = \perp$, then it must be the case that $\tilde{m} \neq \perp$. By definition of Sim , this means that D_λ computed a value $\tilde{c}_j$ such that $\tilde{m} \neq \perp$ but there is no associated input $(\hat{m} \parallel \hat{r})$ in F or queried to $\mathcal{P}$ such that $\hat{m} = \tilde{m}$ and $\tilde{c}_j = \text{Enc}^\mathcal{P}(1^\lambda, t, \tilde{m}; \hat{r})$. Consider the computation of $\tilde{m} = \text{Dec}^\mathcal{P}(\tilde{c}_j)$. As $\tilde{m} \neq \perp$, it must be the case that $(\tilde{m} \parallel \hat{r}) = \text{Dec}_{\text{tlp}}^\mathcal{P}(\tilde{c}_j)$ for some value $\hat{r}$. But $(\tilde{m} \parallel \hat{r})$ is not in F and hasn't been queried to $\mathcal{P}$ yet, so the value $r' \leftarrow \mathcal{P}(1^\lambda, \tilde{m}_D, \hat{r})$ can be lazily sampled from $\{0, 1\}^\lambda$. It follows that $\tilde{m} \neq \perp$ only if, for fixed values $\tilde{m}_j$, $\hat{r}$, and $\tilde{c}_j$, it holds that $\tilde{c}_j = \text{Enc}_{\text{tlp}}(1^\lambda, t, (\tilde{m}_j \parallel \hat{r}); r')$ for $r' \leftarrow \{0, 1\}^\lambda$. The probability this occurs is at most negligible by Claim A.5 above.

□

To argue indistinguishability of the next two hybrids, we leverage the fact that Sim —and hence $D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}$ —can be computed in bounded polynomial-depth.

Claim A.7. *There exists a polynomial d and a non-uniform machine $M = \{M_\lambda\}_{\lambda \in \mathbb{N}}$ such that, for all $\lambda \in \mathbb{N}$, $M_\lambda(\cdot) \equiv D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}(\cdot)$, $\text{size}(M_\lambda) \in \text{poly}(\lambda, \text{size}(Z_\lambda), \text{size}(D_\lambda), p(\lambda))$, and $\text{depth}(M_\lambda) \leq \text{depth}(D_\lambda) \cdot d(\lambda, |m_0, m_1|)$.*

Proof. Let M_λ be a machine that takes a code c as input and has F hardcoded. $M_\lambda(c)$ runs $D_\lambda(c)$ and responds to its queries to $\mathcal{P}$ and $\text{Sim}(\cdot)$ as follows.

- To respond to each query $\mathcal{P}(1^\lambda, x)$, M_λ maintains a set Q consisting of all queries in F and values queried so far. It checks if $x \in Q$ by checking every input in parallel and returns the associated response if one exists. If no response exists, it samples a uniformly random response $r \leftarrow \{0, 1\}^\lambda$ and stores (x, r) in Q .

- To respond to each query $\text{Sim}(\tilde{c}_i)$, M_λ performs all checks for $(\hat{m}||\hat{r}) \in \{0,1\}^{|m_b|+\lambda}$ needed by Sim in parallel and returns the associated value $\hat{m}$ as $\text{Sim}(\tilde{c}_i)$ would.

M_λ implements oracle calls $\mathcal{P}(x)$ and $\text{Sim}(\tilde{c}_i)$ correctly by construction, so it holds that $M_\lambda(\cdot) \equiv D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}(\cdot)$. It remains to argue that M_λ has the appropriate size and depth.

In terms of size, note that M_λ has F hardcoded and clearly runs in polynomial time in $|F|$, $\text{size}(D_\lambda)$, and Sim , which also runs in polynomial time in $|F|$ and $|\text{Enc}|$. Enc is polynomial-time by construction, and $|F| \in \text{poly}(\lambda, \text{size}(D_\lambda), \text{size}(Z_\lambda), p(\lambda))$ by construction. It follows that $\text{size}(M_\lambda) \in \text{poly}(\lambda, \text{size}(Z_\lambda), \text{size}(D_\lambda), p(\lambda))$, as required.

In terms of depth, note that M_λ runs D_λ and can respond to all parallel queries made by D_λ in parallel. So it suffices to argue that each query to $\mathcal{P}$ or $\text{Sim}(\cdot)$ can be responded to in depth at most $d(\lambda, |m_0, m_1|)$ for some fixed polynomial d . Queries for $\mathcal{P}(x)$ require a lookup in a table for x , which can be computed in constant depth $O(1)$ to check equality and sample a random λ -bit value if needed. Queries to $\text{Sim}(\tilde{c}_i)$ require checking in parallel for every value $(\hat{m}||\hat{r}) \in Q$ with $|\hat{m}| = |m_b|$ if $\tilde{c}_i = \text{Enc}^\mathcal{P}(1^\lambda, t, \hat{m}; \hat{r})$. By the efficiency of Enc_{tlp} and $\mathcal{P}$, this requires fixed size and depth polynomial in $(\lambda, \log t, |m_b|)$ from the efficiency of Enc_{tlp} . Since t is polynomially bounded by assumption, it follows that this is at most $d(\lambda, |m_0, m_1|)$ for some polynomial d . Thus, the depth of M_λ increases by at most $d(\lambda, |m_0, m_1|)$ factor for each round of oracle queries that D_λ makes, so $\text{depth}(M_\lambda) \leq \text{depth}(D_\lambda) \cdot d(\lambda, |m_0, m_1|)$, as required. $\square$

Claim A.8. *If $\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)] > 1/(4 \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ does not satisfy hiding.*

Proof. Hybrids $H_{0,2}(\lambda)$ and $H_{0,3}(\lambda)$ differ in the way that the challenge TLP c is computed. In both cases, c is computed as $\text{Enc}_{\text{tlp}}(1^\lambda, t, (m_b||r); r')$ for some $r' \in \{0,1\}^\lambda$. In $H_{0,2}(\lambda)$, r' is computed as $\mathcal{P}(1^\lambda, m_b||r)$, and in $H_{0,3}(\lambda)$, r' is sampled uniformly at random. The experiments are identically distributed as long as $(m_b||r)$ is not in F (so can be lazily sampled) and the rest of the experiment never queries $\mathcal{P}$ at $(m_b||r)$.

As $r \leftarrow \{0,1\}^\lambda$ is uniformly random and independent of F , the probability that $(m_b||r) \in F$ is at most $|F|/2^\lambda \leq f'(\lambda)/2^\lambda$, which is negligible in λ . It follows that the rest of the experiment must query $(m_b||r)$ with probability at least $1/(5 \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, which we show contradicts the hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$.

Towards constructing an adversary against the hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$, we first show construct a bounded polynomial depth non-uniform adversary $\{B_\lambda\}_{\lambda \in \mathbb{N}}$ that, for any message m , guesses r^* on input $c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, (m||r^*))$ for $r^* \leftarrow \{0,1\}^\lambda$. We define B_λ as follows using M_λ from Claim A.7 above.

- B_λ receives $c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, (m||r^*))$ as input and runs $M_\lambda^\mathcal{P}(c)$. B_λ outputs a random value $\hat{r}$ such that M_λ queries $\mathcal{P}$ at $(m||\hat{r})$.

In terms of correctness, observe that B_λ succeeds when M_λ queries $\mathcal{P}$ at $(m||r^*)$ and happens to guess the correct query. Since M_λ emulates $D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}$, it queries $(m||r^*)$ with probability at least $1/(5 \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. Furthermore, D_λ makes at most $q'(\lambda)$ queries to $\mathcal{P}$, so for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} r^* \leftarrow \{0,1\}^\lambda \\ c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, (m_b||r^*)) \quad : r^* = \hat{r} \\ \hat{r} \leftarrow B_\lambda(c) \end{array} \right] > 1/(5 \cdot p(\lambda) \cdot q'(\lambda)).$$

Using B_λ , we construct an adversary $\{(Z'_\lambda, D'_\lambda)\}_{\lambda \in \mathbb{N}}$ against that hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ as follows.

- Z'_λ computes $(m_0, m_1, t) \leftarrow Z_\lambda$, samples $r_0, r_1 \leftarrow \{0, 1\}^\lambda$, and outputs (m'_0, m'_1, t) for $m'_0 = (m_0 || r_0)$ and $m'_1 = (m_0 || r_1)$.
- $D'_\lambda(c)$ computes $\hat{r} \leftarrow B_\lambda(c)$. It outputs $b' \in \{0, 1\}$ if $\hat{r} = r_{b'}$ and outputs a random bit $b' \leftarrow \{0, 1\}$ otherwise.

For correctness, we need to analyze

$$\Pr \left[\begin{array}{l} b \leftarrow \{0, 1\} \\ (m'_0, m'_1, t) \leftarrow Z'_\lambda \\ c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, m'_b) \\ b' \leftarrow D'_\lambda(c) \end{array} : b' = b \right].$$

Fixing the probability event space, we can rewrite the event that $b' = b$ in the following way by construction of D'_λ :

$$\begin{aligned} \Pr[b' = b] &= \Pr[\hat{r} = r_b] - \Pr[\hat{r} = r_{1-b}] + \frac{1}{2} \cdot \Pr[\hat{r} \notin \{r_0, r_1\}] \\ &= \Pr[\hat{r} = r_b] - \Pr[\hat{r} = r_{1-b}] + \frac{1}{2} \cdot (1 - \Pr[\hat{r} = r_b] - \Pr[\hat{r} = r_{1-b}]) \\ &= \frac{1}{2} + \frac{1}{2} \cdot \Pr[\hat{r} = r_b] - \frac{1}{2} \cdot \Pr[\hat{r} = r_{1-b}]. \end{aligned}$$

Since the inputs to B_λ are appropriately distributed, it follows that $\Pr[\hat{r} = r_b] > 1/(5 \cdot p(\lambda) \cdot q'(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. Additionally, $\Pr[\hat{r} = r_{1-b}] \leq 1/2^\lambda$ since $\hat{r}$ is independent of r_{1-b} . It follows that, for infinitely many $\lambda \in \mathbb{N}$,

$$\Pr[b' = b] > \frac{1}{2} + \frac{1}{5 \cdot p(\lambda) \cdot q'(\lambda)} - \frac{1}{2^{\lambda+1}} \geq \frac{1}{2} + \frac{1}{6 \cdot p(\lambda) \cdot q'(\lambda)}.$$

In terms of running time, note that Z'_λ runs in polynomial time in λ and $\text{size}(Z_\lambda)$. D'_λ simply runs M_λ and otherwise does simply $\text{poly}(\lambda)$ -time operations. It follows from Claim A.7 that $\text{depth}(D'_\lambda) \leq \text{depth}(D_\lambda) \cdot d(\lambda, |m'_0, m'_1|)$ for some fixed polynomial d . So, the reduction to hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ is depth-preserving, so (Z'_λ, D'_λ) is a valid bounded polynomial-depth adversary by Lemma D.2, as required. $\square$

Claim A.9. *If $\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)] > 1/(4 \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ does not satisfy hiding.*

Proof. We construct an adversary $\{(Z'_\lambda, D'_\lambda)\}_{\lambda \in \mathbb{N}}$ against the hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ as follows using the machine M_λ from Claim A.7 above.

- Z'_λ computes $(m_0, m_1, t) \leftarrow Z_\lambda^P$, samples $r \leftarrow \{0, 1\}^\lambda$, and outputs (m'_0, m'_1, t) for $m'_0 = (m_0 || r)$ and $m'_1 = (0^{|m_0|} || r)$.
- $D'_\lambda(c)$ computes $b' \leftarrow M_\lambda^P$ and outputs 1 if $b' = 0$ and 0 otherwise.

In terms of correctness, let $b^* \leftarrow \{0, 1\}$ be the challenge bit for the hiding game against $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$. When $b^* = 0$, the input c to D'_λ is distributed according to c from hybrid $H_{0,3}(\lambda)$, and when $b^* = 1$, the input c is distributed according to c from $H_{0,4}(\lambda)$. Note that D'_λ simply runs $M_\lambda^{\mathcal{P}}$ which is identically distributed to $D_\lambda^{\mathcal{P}, \text{Sim}(\cdot)}(\lambda)$ by Claim A.7. So, when $b^* = 0$, it holds that $b' = b^*$ if $b' = b = 0$ and $H_{0,3}(\lambda)$ holds. Similarly, when $b^* = 1$, it holds that $b' = b^*$ if $b' = 1 = 1 - b$ and $H_{0,4}(\lambda)$ doesn't hold. It follows that (Z'_λ, D'_λ) satisfy the following for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned} \Pr \left[\begin{array}{l} b^* \leftarrow \{0, 1\} \\ (m'_0, m'_1, t) \leftarrow Z'_\lambda \\ c \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, m'_{b^*}) \\ b' \leftarrow D'_\lambda(c) \end{array} : b' = b^* \right] &= \frac{1}{2} \cdot (\Pr[b' = b^* \mid b^* = 0] + \Pr[b' = b^* \mid b^* = 1]) \\ &= \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)]) \\ &> \frac{1}{2} + \frac{1}{8 \cdot p(\lambda)}. \end{aligned}$$

In terms of running time, note that Z'_λ simply runs Z_λ and otherwise runs in time $\text{poly}(\lambda)$ to sample r and form its output. D'_λ simply runs $M_\lambda^{\mathcal{P}}$ which, by Claim A.7, satisfies the necessary conditions for the reduction to be depth-preserving. It follows that the reduction contradicts the hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ by Lemma D.2 as required. $\square$

This completes the proof of Lemma A.3. $\square$

B CCA-Hiding Time-Lock Puzzle in the Plain Model

We show that the non-malleable code against bounded polynomial-depth tampering of [DKP21] (which was shown to be a functional non-malleable time-lock puzzle in [FKPS21]) also satisfies CCA-hiding.

Theorem B.1 (CCA-Hiding Time-Lock Puzzle in the Plain Model). *Assuming the existence of time-lock puzzles, keyless multi-collision-resistant hash functions, injective one-way functions, and non-interactive witness indistinguishable proofs for NP, all with sub-exponential security, there exists a CCA-hiding time-lock puzzle.*

To prove the above theorem, we give the TLP construction from [DKP21] and associated proofs in Section B.2. Before doing so, we formally define the additional ingredients needed in Section B.1.

B.1 Ingredients

The construction of [DKP21] relies on time-lock puzzles, one-time signatures, “tuned” one-message ZK arguments, and extractable and tag-based non-malleable non-interactive commitments.

Time-lock puzzles. We require $2^{\lambda+|m|}$ -secure TLPs (Definition 4.9) for messages of length $|m|$. This can be constructed from any sub-exponentially secure TLP by using security parameter $(\lambda + |m|)^c$ for appropriate constant c when encoding messages of length $|m|$. For simplicity, we denote such a TLP by simply using $\lambda + m$ as the security parameter and providing $\text{poly}(\lambda + |m|)$ bits of randomness for encodings.

One-time signatures. We require a one-time signature that with only polynomial security. We note that one-time signatures can be constructed for arbitrary-length messages from (keyed) CRH via Lamport’s construction [Lam79]. We formally define the required one-time signature below.

Definition B.2 (One-Time Signature). A one-time signature consists of three algorithms $(\text{Gen}_{\text{sig}}, \text{Sign}_{\text{sig}}, \text{Ver}_{\text{sig}})$ with the following syntax and efficiency:

- $(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda)$: A probabilistic polynomial-time algorithm that on input a security parameter 1^λ outputs a verification key $\text{vk} \in \{0, 1\}^*$ and a signing key $\text{sk} \in \{0, 1\}^*$.
- $\sigma = \text{Sign}_{\text{sig}}(\text{sk}, m)$: A deterministic polynomial-time algorithm that on input a signing key $\text{sk} \in \{0, 1\}^*$ and a message $m \in \{0, 1\}^*$ outputs a signature $\sigma \in \{0, 1\}^*$.
- $b = \text{Ver}_{\text{sig}}(\text{vk}, m, \sigma)$: A deterministic polynomial-time algorithm that on input a verification key $\text{vk} \in \{0, 1\}^*$, a message $m \in \{0, 1\}^*$, and a signature $\sigma \in \{0, 1\}^*$ outputs a bit $b \in \{0, 1\}$.

We require that $(\text{Gen}_{\text{sig}}, \text{Sign}_{\text{sig}}, \text{Ver}_{\text{sig}})$ satisfy the following:

- **Correctness:** For all $\lambda \in \mathbb{N}$ and $m \in \{0, 1\}^*$, it holds that

$$\Pr \left[(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda) : 1 = \text{Ver}_{\text{sig}}(\text{vk}, m, \text{Sign}_{\text{sig}}(\text{sk}, m)) \right] = 1.$$

- **One-Time Strong Unforgeability:** For every non-uniform PPT adversary $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$ that makes at most one signing oracle query for a message m resulting in signature σ , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda) \\ (m', \sigma') \leftarrow A_\lambda^{\text{Sign}_{\text{sig}}(\text{sk}, \cdot)}(\text{vk}) \end{array} : \begin{array}{l} 1 = \text{Ver}_{\text{sig}}(\text{vk}, m', \sigma') \\ \wedge (m, \sigma) \neq (m', \sigma') \end{array} \right] \leq \text{negl}(\lambda).$$

Tuned one-message zero-knowledge arguments. We require a “tuned” one-message ZK arguments for a sequence of relations $R = \{R_\lambda\}_{\lambda \in \mathbb{N}}$ with instances consisting of two parts $x = (x_1, x_2)$. At a high level, a one-message argument is tuned for x_1 if: (1) it is weakly sound for x_1 , meaning an adversary can only cheat on false statements x where x_1 comes from a bounded-size set, and (2) there is a polynomial-time ZK simulator for every x given inefficient pre-preprocessing on x_1 . We require that the bounded set for weak soundness is polynomial in the size of the associated adversary, and that the inefficient pre-processing for the ZK simulator can be computed in sub-exponential size and fixed polynomial depth and fools sub-exponential size distinguishers. Bitansky and Lin [BL18] construct such a one-message zero-knowledge argument for any efficiently verifiable R from sub-exponentially secure keyless multi-collision-resistant hash functions, injective one-way functions, and non-interactive witness-indistinguishable proofs.

Definition B.3 (Tuned One-Message ZK Arguments). Let $\eta_{\text{sim}}, \eta_{\text{zk}} \in (0, 1)$ be constants. An $(\eta_{\text{sim}}, \eta_{\text{zk}})$ -sub-exponentially secure tuned one-message zero-knowledge argument for a sequence of relations $\{R_\lambda\}_{\lambda \in \mathbb{N}}$ with two-part instances $x = (x_1, x_2)$ consists of a pair of algorithms $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ with the following syntax and efficiency:

- $\pi \leftarrow \text{Prove}_{\text{zk}}(1^\lambda, x, w)$: A probabilistic polynomial-time algorithm that on input a security parameter 1^λ , an instance $x \in \{0, 1\}^*$, and a witness $w \in \{0, 1\}^*$ outputs a proof $\pi \in \{0, 1\}^*$.

- $b = \text{Ver}_{\text{zk}}(1^\lambda, x, \pi)$: A deterministic polynomial-time algorithm that on input a security parameter 1^λ , an instance $x \in \{0, 1\}^*$, and a proof $\pi \in \{0, 1\}^*$ outputs a bit $b \in \{0, 1\}$.

We require that $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ satisfy the following properties:

- **Completeness:** For all $\lambda \in \mathbb{N}$ and $(x, w) \in R_\lambda$, it holds that

$$1 = \text{Ver}_{\text{zk}}(1^\lambda, x, \text{Prove}_{\text{zk}}(1^\lambda, x, w)).$$

- **Weak soundness:** For any non-uniform PPT adversary $A = \{A_\lambda\}_{\lambda \in \mathbb{N}}$, there exists a sequence of sets $S = \{S_\lambda\}_{\lambda \in \mathbb{N}}$ of size $\text{poly}(\lambda, \text{size}(A_\lambda))$ and a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[(x = (x_1, x_2), \pi) \leftarrow A_\lambda : \begin{array}{l} 1 = \text{Ver}_{\text{zk}}(1^\lambda, x, \pi) \\ \wedge x \notin R_\lambda \\ \wedge x_1 \notin S_\lambda \end{array} \right] \leq \text{negl}(\lambda).$$

- **Subexponential tuned zero-knowledge:** There exist a uniform simulator Sim_{zk} such that for any non-uniform $\text{poly}(2^{\lambda^{\eta_{\text{zk}}}})$ -size distinguisher D , there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ and $(x, w) \in R_\lambda$, it holds that

$$\Pr \left[\begin{array}{l} b \leftarrow \{0, 1\} \\ \pi_0 \leftarrow \text{Prove}_{\text{zk}}(1^\lambda, x, w) \\ \pi_1 \leftarrow \text{Sim}_{\text{zk}}(1^\lambda, x) \\ b' \leftarrow D(\pi_b) \end{array} : b' = b \right] \leq \frac{1}{2} + \text{negl}(2^{\lambda^{\eta_{\text{zk}}}}).$$

Furthermore, it holds that Sim_{zk} can be computed in two parts: first, a $\text{poly}(2^{\lambda^{\eta_{\text{sim}}}})$ -time and $\text{poly}(\lambda, |x_1|)$ -depth algorithm $\text{Sim}_{\text{zk}}^{(1)}(1^\lambda, x_1)$ outputs a trapdoor td of size $\text{poly}(\lambda, |x_1|)$, and then a polynomial-time algorithm $\text{Sim}_{\text{zk}}^{(2)}(\text{td}, x)$ outputs a simulated proof π .

Extractable and tag-based non-malleable non-interactive commitments. Next, we require a quasi-polynomial extractable and polynomially-secure tag-based non-malleable non-interactive commitments. We additionally require that the tag-space is of size 2^λ . Bitansky and Lin [BL18] construct such commitments from the same assumptions as the tuned one-message ZK arguments above as well as sub-exponentially secure TLPs. We assume without loss of generality that the commitment uses λ bits of randomness, and we note that any non-interactive commitment has an additional “trivial” decoding algorithm that computes a message m underlying a commitment c_{com} in $\text{poly}(2^{\lambda+|m|})$ -time and fixed polynomial depth by iterating over all possible pairs of message and randomness values.

Definition B.4 (Extractable and Tag-Based Non-Malleable Non-Interactive Commitment). Let $c_{\text{ext}} > 1$ be a constant. A c_{ext} -quasi-polynomial extractable and tag-based non-malleable non-interactive commitment consists of a pair of algorithms $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ with the following syntax and efficiency:

- $c_{\text{com}} \leftarrow \text{Enc}_{\text{com}}(1^\lambda, \text{tag}, m)$: A probabilistic polynomial-time algorithm that on input a security parameter 1^λ , a tag $\text{tag} \in \{0, 1\}^\lambda$, and a message $m \in \{0, 1\}^*$, outputs a commitment c_{com} . We implicitly assume that c_{com} specifies 1^λ , tag , and $|m|$.

- $m = \text{Dec}_{\text{com}}(c_{\text{com}})$: A deterministic $\text{poly}(2^{(\log \lambda)^{c_{\text{ext}}}})$ -time algorithm that on input a commitment $c_{\text{com}} \in \{0, 1\}^*$ outputs a message $m \in \{0, 1\}^*$.

We require that $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ is an encoding scheme and satisfies plain non-malleability of Definition 4.7 against pairs of non-uniform polynomial-time adversaries with respect to a tag-based special decoding algorithm

$$\text{Dec}_{c, \text{tag}}^*(\tilde{c}) := \begin{cases} \perp & \text{if tag} = \tilde{\text{tag}} \text{ from } \tilde{c} \text{ or} \\ & \tilde{c} \text{ is not for same } 1^\lambda, |m| \text{ as } c, \text{ or} \\ \text{Dec}_{\text{com}}(\tilde{c}_{\text{com}}) & \text{otherwise.} \end{cases}$$

B.2 Construction and Proof of CCA-Hiding

The construction relies on an underlying TLP $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$, an extractable and tag-based non-malleable non-interactive commitment $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$, a tuned one-message ZK argument $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$, and a one-time signature $(\text{Gen}_{\text{sig}}, \text{Ver}_{\text{sig}}, \text{Sign}_{\text{sig}})$ as above. At a high level, the construction of [DKP21] follows a variant of the Naor-Yung paradigm [NY90] adapted to the fully non-interactive setting. That is, encoding a message m consists of two separate encodings of m together with a ZK proof of consistency using Prove_{zk} . The first encoding of m uses the underlying TLP Enc_{tlp} , which gives a way to decode the overall TLP. The second encoding of m uses the commitment Enc_{com} , which gives a non-malleability guarantee for the overall TLP. Specifically, the commitment is non-malleable with respect to a tag consisting of a signing verification key, and the two encodings and consistency proof are signed relative to this key. This is formalized in Construction B.5.

Construction B.5. CCA-Hiding TLP in the Plain Model

Ingredients: Let $(\text{Gen}_{\text{sig}}, \text{Sign}_{\text{sig}}, \text{Ver}_{\text{sig}})$ be a one-time signature (Definition B.2), $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ be a TLP (Definition 4.9), $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ be an extractable and tag-based non-malleable non-interactive commitment (Definition B.4), $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ be a tuned one-message ZK argument (Definition B.3) for the sequence of relations $\{R_\lambda\}_{\lambda \in \mathbb{N}}$ such that

$$R_\lambda = \left\{ \begin{array}{l} (x = (\text{vk}, c_{\text{tlp}}, c_{\text{com}}), \\ w = (m, r_{\text{tlp}}, r_{\text{com}})) \end{array} : \begin{array}{l} c_{\text{tlp}} \text{ is for } \lambda, t, |m| \text{ and} \\ c_{\text{tlp}} = \text{Enc}_{\text{tlp}}(1^{\lambda+|m|}, t, m; r_{\text{tlp}}) \\ \wedge c_{\text{com}} = \text{Enc}_{\text{com}}(1^\lambda, m; r_{\text{com}}) \end{array} \right\}$$

and is tuned with respect to the input c_{tlp} .

Construction: We construct a TLP (Enc, Dec) as follows:

- $c \leftarrow \text{Enc}(1^\lambda, t, m)$:
 1. Sample $(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda)$.
 2. Compute $c_{\text{tlp}} \leftarrow \text{Enc}_{\text{tlp}}(1^{\lambda+|m|}, t, m; r_{\text{tlp}})$ for $r_{\text{tlp}} \leftarrow \{0, 1\}^{\text{poly}(\lambda+|m|)}$.
 3. Compute $c_{\text{com}} \leftarrow \text{Enc}_{\text{com}}(1^\lambda, \text{vk}, m; r_{\text{com}})$ for $r_{\text{com}} \leftarrow \{0, 1\}^\lambda$.
 4. Compute a proof $\pi \leftarrow \text{Prove}(1^\lambda, (\text{vk}, c_{\text{tlp}}, c_{\text{com}}), (m, r_{\text{tlp}}, r_{\text{com}}))$.
 5. Compute $\sigma = \text{Sign}_{\text{sig}}(\text{sk}, (c_{\text{tlp}}, c_{\text{com}}, \pi))$.
 6. Output $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$.

- $m = \text{Dec}(c)$:
 1. Parse $c := (\mathbf{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ for security parameter λ .
 2. If $\text{Ver}_{\text{sig}}(\mathbf{vk}, (c_{\text{tlp}}, c_{\text{com}}, \pi), \sigma) = 0$ or $\text{Ver}_{\text{zk}}(1^\lambda, (\mathbf{vk}, c_{\text{tlp}}, c_{\text{com}}), \pi) = 0$, output $\perp$.
 3. Otherwise, output $m = \text{Dec}_{\text{tlp}}(c_{\text{tlp}})$.

Correctness and efficiency follow directly from the correctness and efficiency of the underlying primitives. For the proof of CCA-hiding, we primarily follow the proof of functional non-malleability of this construction from [FKPS21] (Lemma A.3, which in turn is based on Claim 6.2 in [DKP21]), and note the differences when relevant.

Lemma B.6. (*CCA-Hiding*) *Assuming the security of the above building blocks (Enc, Dec) is a CCA-hiding TLP.*

Proof. Suppose by way of contradiction that (Enc, Dec) does not satisfy bounded polynomial-depth CCA-hiding. Namely, for every positive monotonic polynomial α , there exists an α -gap polynomial sized bounded depth adversary $(Z, D) = \{(Z_\lambda, D_\lambda)\}_{\lambda \in \mathbb{N}}$ for the CCA-hiding game and a polynomial p , such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (m_0, m_1, t) \leftarrow Z_\lambda \\ b \leftarrow \{0, 1\} \\ c \leftarrow \text{Enc}(1^\lambda, t, m_b) \\ b' \leftarrow D_\lambda^{\text{Dec}_c^*(\cdot)}(c) \end{array} : \begin{array}{l} |m_0| = |m_1| \\ \wedge b = b' \end{array} \right] > \frac{1}{2} + \frac{1}{p(\lambda)}.$$

For given polynomial α and fixed adversary (Z, D) , we assume without loss of generality that $|m_0| = |m_1|$. We define the following sequence of hybrid experiments for every $\lambda \in \mathbb{N}$ and $b \in \{0, 1\}$.

- $H_{b,0}(\lambda)$: In the first hybrid, we simply fix a value of b in the above CCA-hiding experiment.
 1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
 2. Sample $c \leftarrow \text{Enc}(1^\lambda, t, m_b)$.
 3. Compute $b' \leftarrow D_\lambda^{\text{Dec}_c^*(\cdot)}(c)$.
 4. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,1}(\lambda)$: In this hybrid, we respond to decoding oracle queries via a modified algorithm $\text{Dec}_{c, \mathbf{vk}}^*(\cdot)$ that has the challenge verification key $\mathbf{vk}$ hardcoded and rejects queries that don't change $\mathbf{vk}$.
 1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
 2. Sample $c := (\mathbf{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma) \leftarrow \text{Enc}(1^\lambda, t, m_b)$.
 3. Let b' be the output of $D_\lambda(\cdot)$ give oracle access to $\text{Dec}_{c, \mathbf{vk}}^*(\cdot)$ that on input $\tilde{c} = (\tilde{\mathbf{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ outputs $\tilde{m}$ defined as follows:
 - If $\tilde{\mathbf{vk}} = \mathbf{vk}$, set $\tilde{m} := \perp$.
 - Otherwise, set $\tilde{m} := \text{Dec}(\tilde{c})$.
 4. Output 1 if $b' = b$ and 0 otherwise.

- $H_{b,2}(\lambda)$: In this hybrid, we compute the proof π for the challenge TLP c using the ZK simulator Sim_{zk} guaranteed to exist for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$.

1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
2. Set $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ as follows:
 - Sample $(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda)$.
 - Compute the TLP $c_{\text{tlp}} \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, m_b)$.
 - Compute the commitment $c_{\text{com}} \leftarrow \text{Enc}_{\text{com}}(1^\lambda, \text{vk}, m_b)$.
 - Compute a simulated proof $\pi \leftarrow \text{Sim}_{\text{zk}}(1^\lambda, (\text{vk}, c_{\text{tlp}}, c_{\text{com}}))$ for the relation $\mathcal{R}_\lambda$.
 - Compute the signature $\sigma \leftarrow \text{Sign}_{\text{sig}}(\text{sk}, (c_{\text{tlp}}, c_{\text{com}}, \pi))$.
3. Let b' be the output of $D_\lambda(\cdot)$ give oracle access to $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$ that on input $\tilde{c} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ outputs $\tilde{m}$ defined as follows:
 - If $\tilde{\text{vk}} = \text{vk}$, set $\tilde{m} := \perp$.
 - Otherwise, set $\tilde{m} := \text{Dec}(\tilde{c})$.
4. Output 1 if $b' = b$ and 0 otherwise.

- $H_{b,3}(\lambda)$ In this hybrid, we commit to $0^{|m_b|}$ instead of m_b when computing c_{com} .

1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
2. Set $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ as follows:
 - Sample $(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda)$.
 - Compute the TLP $c_{\text{tlp}} \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, m_b)$.
 - Compute the commitment $c_{\text{com}} \leftarrow \text{Enc}_{\text{com}}(1^\lambda, \text{vk}, 0^{|m_b|})$.
 - Compute a simulated proof $\pi \leftarrow \text{Sim}_{\text{zk}}(1^\lambda, (\text{vk}, c_{\text{tlp}}, c_{\text{com}}))$ for the relation $\mathcal{R}_\lambda$.
 - Compute the signature $\sigma \leftarrow \text{Sign}_{\text{sig}}(\text{sk}, (c_{\text{tlp}}, c_{\text{com}}, \pi))$.
3. Let b' be the output of $D_\lambda(\cdot)$ give oracle access to $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$ that on input $\tilde{c} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ outputs $\tilde{m}$ defined as follows:
 - If $\tilde{\text{vk}} = \text{vk}$, set $\tilde{m} := \perp$.
 - Otherwise, set $\tilde{m} := \text{Dec}(\tilde{c})$.
4. Output 1 if $b' = b$ and 0 otherwise.

- $H_{b,4}(\lambda)$: In this hybrid, we respond to the decoding oracle queries via a modified algorithm $\text{Dec}_{x, \text{vk}, S}^*(\cdot)$ that additionally has a set S hardcoded. Let $n = n(\lambda)$ be a polynomial bounding the number of decoding queries by D_λ . We define the set S as follows:

- For each $\lambda \in \mathbb{N}$ and $i \in [n]$, let $B^{(i)}$ be the algorithm that computes $(m_0, m_1, t) \leftarrow Z_\lambda$, $c \leftarrow \text{Enc}(1^\lambda, t, m_b)$, and runs $D_\lambda^{\text{Dec}_{c, \text{vk}}^*(\cdot)}$ while computing responses for the first $i - 1$ decoding queries. Let $\tilde{c} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ be the i th decoding query (if one exists). $B^{(i)}$ outputs the associated instance and proof $((\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}), \tilde{\pi})$ for R_λ .
- Let $S^{(i)}$ be the set of underlying time-lock puzzles for $B^{(i)}$ that is guaranteed to exist by weak soundness of $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$.

- We define the set S to consists of time lock puzzles $\hat{c}_{\text{tlp}}$ and associated underlying messages $\hat{m}$ where $\hat{c}_{\text{tlp}}$ is in $S^{(i)}$ for some $i \in [n]$. Namely,

$$S = \{(\hat{c}_{\text{tlp}}, \hat{m}) : \exists i \in [n], \hat{c}_{\text{tlp}} \in S^{(i)} \wedge \hat{m} = \text{Dec}_{\text{tlp}}(\hat{c}_{\text{tlp}})\}.$$

The decoding algorithm $\text{Dec}_{x, \text{vk}, S}^*(\cdot)$ either uses S to decode or otherwise decodes from the underlying commitment rather than the underlying TLP.

1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
 2. Set $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ as follows:
 - Sample $(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda)$.
 - Compute the TLP $c_{\text{tlp}} \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, m_b)$.
 - Compute the commitment $c_{\text{com}} \leftarrow \text{Enc}_{\text{com}}(1^\lambda, \text{vk}, 0^{|m_b|})$.
 - Compute a simulated proof $\pi \leftarrow \text{Sim}_{\text{zk}}(1^\lambda, (\text{vk}, c_{\text{tlp}}, c_{\text{com}}))$ for the relation $\mathcal{R}_\lambda$.
 - Compute the signature $\sigma \leftarrow \text{Sign}_{\text{sig}}(\text{sk}, (c_{\text{tlp}}, c_{\text{com}}, \pi))$.
 3. Let b' be the output of $D_\lambda(\cdot)$ give oracle access to $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$ that on input $\tilde{c} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ outputs $\tilde{m}$ defined as follows:
 - If $\tilde{\text{vk}} = \text{vk}$, set $\tilde{m} := \perp$.
 - If $\text{Ver}_{\text{sig}}(\tilde{\text{vk}}, (\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}), \tilde{\sigma}) = 0$ or $\text{Ver}_{\text{zk}}(1^\lambda, (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}), \tilde{\pi}) = 0$, set $\tilde{m} := \perp$.
 - If $(\tilde{c}_{\text{tlp}}, \tilde{m}) \in S$ for some $\tilde{m}$, set $\tilde{m} := \tilde{m}$.
 - Otherwise, set $\tilde{m} := \text{Dec}_{\text{com}}(\tilde{c}_{\text{com}})$.
 4. Output 1 if $b' = b$ and 0 otherwise.
- $H_{b,5}(\lambda)$: In this hybrid, we replace the value m_b with $0^{|m_b|}$ when computing c_{tlp} .
 1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
 2. Set $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ as follows:
 - Sample $(\text{vk}, \text{sk}) \leftarrow \text{Gen}_{\text{sig}}(1^\lambda)$.
 - Compute the TLP $c_{\text{tlp}} \leftarrow \text{Enc}_{\text{tlp}}(1^\lambda, t, 0^{|m_b|})$.
 - Compute the commitment $c_{\text{com}} \leftarrow \text{Enc}_{\text{com}}(1^\lambda, \text{vk}, 0^{|m_b|})$.
 - Compute a simulated proof $\pi \leftarrow \text{Sim}_{\text{zk}}(1^\lambda, (\text{vk}, c_{\text{tlp}}, c_{\text{com}}))$ for the relation $\mathcal{R}_\lambda$.
 - Compute the signature $\sigma \leftarrow \text{Sign}_{\text{sig}}(\text{sk}, (c_{\text{tlp}}, c_{\text{com}}, \pi))$.
 3. Let b' be the output of $D_\lambda(\cdot)$ give oracle access to $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$ that on input $\tilde{c} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ outputs $\tilde{m}$ defined as follows:
 - If $\tilde{\text{vk}} = \text{vk}$, set $\tilde{m} := \perp$.
 - If $\text{Ver}_{\text{sig}}(\tilde{\text{vk}}, (\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}), \tilde{\sigma}) = 0$ or $\text{Ver}_{\text{zk}}(1^\lambda, (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}), \tilde{\pi}) = 0$, set $\tilde{m} := \perp$.
 - If $(\tilde{c}_{\text{tlp}}, \tilde{m}) \in S$ for some $\tilde{m}$, set $\tilde{m} := \tilde{m}$.
 - Otherwise, set $\tilde{m} := \text{Dec}_{\text{com}}(\tilde{c}_{\text{com}})$.
 4. Output 1 if $b' = b$ and 0 otherwise.

By assumption, it holds that for infinitely many $\lambda \in \mathbb{N}$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{p(\lambda)} &< \frac{1}{2} \cdot \Pr[H_{0,0}(\lambda)] + \frac{1}{2} \cdot \Pr[H_{1,0}(\lambda)] \\ &= \frac{1}{2} \cdot \left(\Pr[H_{0,5}(\lambda)] + \sum_{i=1}^5 (\Pr[H_{0,i-1}(\lambda)] - \Pr[H_{0,i}(\lambda)]) \right) \\ &\quad + \frac{1}{2} \cdot \left(\Pr[H_{1,5}(\lambda)] + \sum_{i=1}^5 (\Pr[H_{1,i-1}(\lambda)] - \Pr[H_{1,i}(\lambda)]) \right) \end{aligned}$$

Observe that $H_{0,5}$ and $H_{1,5}$ are identically distributed since $|m_0| = |m_1|$, so $\Pr[H_{0,5}(\lambda)] + \Pr[H_{1,5}(\lambda)] = 1$ for all $\lambda \in \mathbb{N}$. It follows for infinitely many $\lambda \in \mathbb{N}$,

$$\frac{1}{p(\lambda)} < \frac{1}{2} \sum_{b \in \{0,1\}, i \in [5]} (\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)]).$$

So, there exists a pair of consecutive hybrids such that for some $b \in \{0,1\}$ and $i \in [5]$,

$$\Pr[H_{b,i-1}(\lambda)] - \Pr[H_{b,i}(\lambda)] > \frac{1}{5 \cdot p(\lambda)}.$$

In the claims below, we show that this leads to a contradiction of our assumptions on the underlying ingredients. We give the below claims for $b = 0$ without loss of generality.

Claim B.7. *If $\Pr[H_{0,0}(\lambda)] - \Pr[H_{0,1}(\lambda)] > 1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Gen}_{\text{sig}}, \text{Sign}_{\text{sig}}, \text{Ver}_{\text{sig}})$ does not satisfy one-time strong unforgeability against non-uniform polynomial-time adversaries.*

Proof. We construct an adversary $\{A_\lambda\}_{\lambda \in \mathbb{N}}$ against the one-time strong unforgeability of $(\text{Gen}_{\text{sig}}, \text{Sign}_{\text{sig}}, \text{Ver}_{\text{sig}})$ as follows.

- On input a verification key vk , A_λ computes $(m_0, m_1, t) \leftarrow Z_\lambda$ and $c_{\text{tlp}}, c_{\text{com}}, \pi$ as in $\text{Enc}(1^\lambda, t, m_0)$. It computes $\sigma = \text{Sign}_{\text{sig}}(\text{sk}, (c_{\text{tlp}}, c_{\text{com}}, \pi))$ using one signing oracle call and uses challenge TLP $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$. A_λ runs $D_\lambda(c)$, which makes oracle queries to $\text{Dec}_c^*(\cdot)$. On input an oracle query $\tilde{c} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$, A_λ checks if $\tilde{\text{vk}} = \text{vk}$ but $(\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma}) \neq (c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$. If this is the case and $\text{Ver}_{\text{sig}}(\text{vk}, (\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}), \tilde{\sigma}) = 1$, it outputs $((\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}), \tilde{\sigma})$. Otherwise, it responds with $\tilde{m} = \text{Dec}_c^*(\tilde{c})$ and continues running D_λ until it makes no more queries. If A_λ hasn't output any value yet, it outputs $\perp$.

In terms of running time, note that A_λ makes a single signing oracle query and otherwise runs parts of $H_{0,0}(\lambda)$, which all run in polynomial-time by the efficiency of Z_λ , D_λ , Enc , and Dec . It follows that A_λ is a valid adversary.

In terms of correctness, we note that A_λ outputs a non- $\perp$ value whenever some query $\tilde{c}$ that D_λ makes to $\text{Dec}_c^*(\cdot)$ has $\tilde{\text{vk}} = \text{vk}$, $(\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma}) \neq (c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$, and $\text{Ver}_{\text{sig}}(\text{vk}, (\tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}), \tilde{\sigma}) = 1$. This corresponds to a valid forgery, so A_λ succeeds if such an event ever occurs. Furthermore, if such an event doesn't occur, the two hybrids are identically distributed since if $\tilde{\text{vk}} = \text{vk}$ and $\tilde{c} = \tilde{c}$, $\text{Dec}_c^*(\tilde{c})$ already outputs $\perp$. It follows that A_λ succeeds with probability at least $1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, so $(\text{Gen}_{\text{sig}}, \text{Sign}_{\text{sig}}, \text{Ver}_{\text{sig}})$ does not satisfy one-time strong unforgeability. $\square$

Claim B.8. *If $\Pr[H_{0,1}(\lambda)] - \Pr[H_{0,2}(\lambda)] > 1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then Sim_{zk} is not a valid ZK simulator for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ against non-uniform polynomial-time adversaries.*

Proof. The proof follows via a reduction to the correctness guarantee of Sim_{zk} as a ZK simulator for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ similar to Claim A.4 of [FKPS21]. At a high level, the adversary in the reduction emulates the rest of the common experiment between hybrids $H_{0,1}(\lambda)$ and $H_{0,2}(\lambda)$, which runs in polynomial-time by the efficiency of Z_λ , Enc , and Dec . The hybrids only differ in the way that the proof π is generated, which implies that the associated adversary succeeds with distinguishing advantage at least $1/(12p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, which contradicts the assumption that Sim_{zk} is a valid ZK simulator for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$. $\square$

Claim B.9. *If $\Pr[H_{0,2}(\lambda)] - \Pr[H_{0,3}(\lambda)] > 1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ is not hiding against non-uniform polynomial-time adversaries.*

Proof. The proof follows via a reduction to the hiding of $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ similar to Claim A.5 of [FKPS21]. At a high level, the adversary in the reduction emulates the rest of the common experiment between hybrids $H_{0,1}(\lambda)$ and $H_{0,2}(\lambda)$ to derive a contradiction. Naively, this requires super-polynomial security since Sim_{zk} runs in super-polynomial time. However, the reduction can leverage the fact that Sim_{zk} can be computed in polynomial-time given inefficient pre-processing on c_{tlp} . As such, the reduction can fix a sufficient random value r_{tlp} used to generate c_{tlp} and then fix an associated trapdoor td . Given td , computing $\text{Sim}_{\text{zk}}(1^\lambda, (\text{vk}, c_{\text{tlp}}, c_{\text{com}}))$ and the rest of the experiment only requires polynomial-time. $\square$

Claim B.10. *If $\Pr[H_{0,3}(\lambda)] - \Pr[H_{0,4}(\lambda)] > 1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then either $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ is not weakly sound, Sim_{zk} is not a valid ZK simulator for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$, or $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ is not tag-based non-malleable.*

Proof. We prove this claim by an additional hybrid argument over the number of $\text{Dec}_{c, \text{vk}}^*(\cdot)$ queries made by D_λ , which is at most a polynomial $n = n(\lambda)$. Specifically, we define the following hybrids for each $j \in [0, n]$.

- $H_{0,3,j}(\lambda)$: This hybrid responds to the first j queries that D_λ makes to $\text{Dec}_{c, \text{vk}}^*(\cdot)$ using $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$ instead.
 1. Compute $(m_0, m_1, t) \leftarrow Z_\lambda$.
 2. Set $c := (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ as in $H_{0,3}(\lambda)$.
 3. Let b' be the output of D_λ while responding to the i th query $\tilde{c}_i$ as follows:
 - If $i \leq n - j$, respond with $\tilde{m}_i = \text{Dec}_{c, \text{vk}}^*(\tilde{c}_i)$.
 - If $i > n - j$, respond with $\tilde{m}_i = \text{Dec}_{c, \text{vk}, S}^*(\tilde{c}_i)$.
 4. Output 1 if $b' = b$ and 0 otherwise.

Note that $H_{0,3,0}(\lambda) \equiv H_{0,3}(\lambda)$ and $H_{0,3,n} \equiv H_{0,4}(\lambda)$, so there must exist an index $j \in [n]$ such that

$$\Pr[H_{0,3,j-1}] - \Pr[H_{0,3,j}(\lambda)] > 1/(5 \cdot n \cdot p(\lambda)).$$

Fix such an index j . We show that this leads to a contradiction for one of the assumptions above.

Hybrids $H_{0,3,j-1}(\lambda)$ and $H_{0,3,j}(\lambda)$ differ only in the value $\tilde{m}_{n-j}$ computed as a response to the decoding oracle query for $\tilde{c}_{n-j}$. Let $\tilde{m} = \text{Dec}_{c, \text{vk}}^*(\tilde{c}_{n-j})$ denote the value computed in $H_{0,3,j-1}(\lambda)$

just using Dec_{tlp} , and let $\tilde{m}_S = \text{Dec}_{c, \text{vk}, S}^*(\tilde{c}_{n-j})$ denote the value computed in $H_{0,3,j}(\lambda)$ using S and Dec_{com} . It follows that, for infinitely many $\lambda \in \mathbb{N}$,

$$\Pr[H_{0,3,j-1}(\lambda)] - \Pr[H_{0,3,j}(\lambda)] = \Pr[\tilde{m} \neq \tilde{m}_S] > 1/(5 \cdot n \cdot p(\lambda)).$$

The proof that these queries can't differ with more than negligible probability is similar to the proof of Claim A.6 in [FKPS21] and reduces to the various assumptions above. In both settings, we are now only arguing about a single query (or technically a round of non-adaptive queries in the case of [FKPS21]). In our case, however, we have to simulate $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$ for the queries $i > n - j$, which may require computing $\text{Dec}_{\text{com}}(\cdot)$. For this reason, we need to assume the underlying commitment satisfies CCA security rather than just (functional) non-malleability.

Our goal is to derive a contradiction whenever $\Pr[\tilde{m} \neq \tilde{m}_S]$. We emphasize that the query $\tilde{c}_{n-j}$ we are interested in is generated according to hybrid $H_{0,3}$, but we define the set S according to an honestly generated TLP $c \leftarrow \text{Enc}(1^\lambda, t, m_0)$ as in hybrid $H_{0,1}$ (this is to ensure $|S|$ is polynomially bounded). As such, we first analyze the probability the responses differ using $\text{Dec}_{c, \text{vk}}^*$ vs. $\text{Dec}_{c, \text{vk}, S}^*$ when the $(n - j)$ th query is sampled according to hybrid $H_{0,1}$, and then show that the responses can't differ too much as we switch to hybrid $H_{0,2}$ and then finally $H_{0,3}$, as desired. To formalize this, let $\tilde{m}^k$ and $\tilde{m}_S^k$ denote the $(n - j)$ th response using $\text{Dec}_{c, \text{vk}}^*(\tilde{c}_{n-j})$ and $\text{Dec}_{c, \text{vk}, S}^*(\tilde{c}_{n-j})$, respectively, when c and $\tilde{c}_{n-j}$ are sampled according to hybrid $H_{0,k}$ for $k \in [3]$. We define the quantity

$$P_k(\lambda) := \Pr[\tilde{m}^k \neq \tilde{m}_S^k]$$

Recall that our goal is to show that $P_3(\lambda)$ can't be too large. In the following sub-claims, we decompose $P_3(\lambda)$ as the sum of $P_1(\lambda)$, $(P_2(\lambda) - P_1(\lambda))$, and $(P_3(\lambda) - P_2(\lambda))$. It follows that one of these quantities must be at least $1/(15 \cdot n \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, which we show leads to a contradiction of our assumptions.

Subclaim B.11. *If $P_1(\lambda) > 1/(15 \cdot n \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ does not satisfy weak soundness for non-uniform polynomial-time adversaries.*

Proof. This proof follows via a reduction to the weak soundness of $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ similar Subclaim A.7 of [FKPS21]. At a high level, we defined the adversary $B^{(n-j)}$ and associated set $S^{(n-j)}$ in hybrid $H_{0,4}$ as guaranteed by the weak soundness property of $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$. The output of $B^{(n-j)}$ exactly corresponds to the values $(\text{vk}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ associated with query $\tilde{c}_{n-j}$ by construction. Namely, $B^{(n-j)}$ computes the challenge TLP $c \leftarrow \text{Enc}(1^\lambda, t, m_0)$ as in $H_{0,1}$ and then runs D_λ until its $(n - j)$ th query and responds to the first $n - j - 1$ queries using $\text{Dec}_{c, \text{vk}}^*(\cdot)$.

Note that if the responses differ $\tilde{m}^1 \neq \tilde{m}_S^1$, it must be the case that the ZK proof accepts and $\tilde{c}_{\text{tlp}} \notin S^{(n-j)}$. In this case, we have that $\tilde{m}^1 = \text{Dec}_{\text{tlp}}(\tilde{c}_{n-j})$ and $\tilde{m}_S^1 = \text{Dec}_{\text{com}}(\tilde{c}_{n-j})$. But if the underlying values for Dec_{tlp} and Dec_{com} differ, that implies that $(\text{vk}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}) \notin R_\lambda$, so

$$\Pr \left[\begin{array}{l} \text{Ver}_{\text{zk}}(1^\lambda, (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}), \tilde{\pi}) = 1 \\ \wedge \tilde{c}_{\text{tlp}} \notin S^{(n-j)} \\ \wedge (\text{vk}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}) \notin R_\lambda \end{array} \right] \geq \Pr[\tilde{m}^1 \neq \tilde{m}_S^1] > 1/(15 \cdot n \cdot p(\lambda)).$$

This contradicts the weak soundness guarantee of $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$, as required. $\square$

Subclaim B.12. *If $P_1(\lambda) - P_2(\lambda) > 1/(15 \cdot n \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then Sim_{zk} is not a valid ZK simulator for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ against non-uniform sub-exponential-time distinguishers.*

Proof. This proof follows via a reduction to the correctness guarantee of Sim_{zk} as a simulator for $(\text{Prove}_{\text{zk}}, \text{Ver}_{\text{zk}})$ similar to Subclaim A.8 of [FKPS21]. At a high level, we argue that the algorithm that checks the predicate for whether or not the $(n - j)$ th responses are equal, cannot differ too much between hybrids $H_{0,1}$ and $H_{0,2}$. Indeed, the only difference between these hybrids is how the proof π is generated for the challenge TLP c . In $H_{0,1}$, π is computed honestly using Prove_{zk} , and in $H_{0,2}$, π is simulated using Sim_{zk} . It follows that if Sim_{zk} is distinguishable from real proofs with advantage $|P_1(\lambda) - P_2(\lambda)|/2 > 1/(30 \cdot n \cdot p(\lambda))$.

In terms of running time, the remainder of the experiment consists of running Enc , Dec , and D_λ until D_λ makes its $(n - j)$ th query $\tilde{c}_{n-j} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$. To compute the associated responses and check if they're equal or not, we potentially have to compute Dec_{tlp} , Dec_{com} , and check if $\tilde{c}_{\text{tlp}} \in S$. Everything above runs in polynomial-time, except for computing Dec_{com} , which requires sub-exponential time. It follows that the Sim_{zk} is not valid ZK simulator against sub-exponential-size distinguishers, as required. $\square$

Subclaim B.13. *If $P_2(\lambda) - P_3(\lambda) > 1/(15 \cdot n \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ is not tag-based non-malleable against non-uniform polynomial-time adversaries.*

Proof. This proof follows via a reduction to the non-malleability of $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ similar to Subclaim A.9 of [FKPS21]. Recall that tag-based non-malleability of $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ is equivalent to hiding against adversaries that make a single $\text{Dec}_{\text{com}}(\cdot)$ query for a commitment with a different tag. So, at a high level, we reduce to such an algorithm that checks the predicate for whether or not the $(n - j)$ th responses are equal using $\text{Dec}_{c, \text{vk}}^*(\cdot)$ vs. $\text{Dec}_{c, \text{vk}, S}^*(\cdot)$. The difference in this reduction between values produced by hybrids $H_{0,2}$ and $H_{0,3}$ is how the commitment value c_{com} is computed for the challenge TLP c . In $H_{0,2}$, c_{com} is computed as a commitment to m_0 , and in $H_{0,3}$, it is a commitment for $0^{|m_0|}$. It follows that the reduction distinguishes these two cases with advantage $|P_2(\lambda) - P_3(\lambda)|/2 > 1/(30 \cdot n \cdot p(\lambda))$.

In terms of running time, we observe that the reduction runs the common parts in experiments $H_{0,2}$ and $H_{0,3}$. In order to run Sim_{zk} efficiently, we observe that we can non-uniformly fix underlying TLP randomness r_{tlp} resulting in a fixed value c_{tlp} in the reduction for the challenge TLP c . The rest of the reduction is efficient and runs in polynomial-time up until D_λ makes its $(n - j)$ th query $\tilde{c}_{n-j} = (\tilde{\text{vk}}, \tilde{c}_{\text{tlp}}, \tilde{c}_{\text{com}}, \tilde{\pi}, \tilde{\sigma})$ and we need to compute $\text{Dec}_{c, \text{vk}, S}^*(\tilde{c}_{n-j})$. Whenever the output of $\text{Dec}_{c, \text{vk}, S}^*$ is non- $\perp$, it must be the case that the tag $\tilde{\text{vk}} \neq \text{vk}$ by construction. So, we can compute $\text{Dec}_{\text{com}}(\tilde{c}_{\text{com}})$ using a single decoding oracle call to $\text{Dec}_{\text{com}}(\cdot)$. It follows that this reduction violates the tag-based non-malleability of $(\text{Enc}_{\text{com}}, \text{Dec}_{\text{com}})$ as required. $\square$

This completes the proof of Claim B.10. $\square$

Claim B.14. *If $\Pr[H_{0,4}(\lambda)] - \Pr[H_{0,5}(\lambda)] > 1/(5p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ does not satisfy hiding for $2^{\lambda+|m_0|}$ -size bounded polynomial-depth adversaries.*

Proof. This proof follows via a reduction to the hiding of $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$ similar to Claim A.11 of [FKPS21]. We construct adversaries $(Z', D') = \{(Z'_\lambda, D'_\lambda)\}_{\lambda \in \mathbb{N}}$ for the $2^{\lambda+|m_0|}$ -size bounded polynomial-depth hiding game with respect to $(\text{Enc}_{\text{tlp}}, \text{Dec}_{\text{tlp}})$.

- Z'_λ computes $(m_0, m_1, t) \leftarrow Z_\lambda$ and outputs $(m_0, 0^{|m_0|}, t)$.

- On input a TLP c_{tlp} for either m_0 or $0^{|m_0|}$, D'_λ computes $c = (\text{vk}, c_{\text{tlp}}, c_{\text{com}}, \pi, \sigma)$ by computing vk , c_{com} , π , and σ as in $H_{0,4}(\lambda)$. It then runs $D_\lambda(c)$ and computes the responses for the oracle calls to $\text{Dec}_{c,\text{vk},S}(\cdot)$ explicitly. However, for queries $\tilde{c}$ that require computing $\text{Dec}_{\text{com}}(\tilde{c}_{\text{com}})$, D'_λ instead extracts the underlying value $\tilde{m}$ by trying all combinations of messages and randomness in parallel. Finally, D'_λ outputs the bit b' output by D_λ .

In terms of correctness, let b^* be the challenge bit in the hiding game. When $b^* = 0$, the TLP c_{tlp} is for the message m_0 so the game is identically distributed to $H_{0,4}(\lambda)$. When $b^* = 1$, the TLP c_{tlp} is for $0^{|m_0|}$ so the game is identically distributed to $H_{0,5}(\lambda)$. It follows that (Z'_λ, D'_λ) have distinguishing advantage $(\Pr[H_{0,4}(\lambda)] - \Pr[H_{0,5}(\lambda)]) / 2 > 1 / (10 \cdot p(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. observe that when the TLP hiding experiment chooses the message m_0 .

In terms of running time, we argue that the reduction above is depth-preserving as in Definition D.1 and results in a $\text{poly}(2^{\lambda+|m_0|})$ -size adversary, so the resulting adversary is a valid $2^{\lambda+|m_0|}$ -size bounded polynomial-depth adversary by Lemma D.2.

- First, note that Z'_λ simply runs Z_λ , so runs in polynomial-time. Furthermore, Z'_λ outputs the same time bound t as Z_λ , as required. It also outputs the same length messages, so the additional outputs are bounded as required.
- To bound the size of D'_λ , note that D'_λ runs in polynomial-time aside from computing Sim_{zk} and extracting $\tilde{m}$ from $\tilde{c}_{\text{com}}$ when computing $\text{Dec}_{c,\text{vk},S}^*(\tilde{c})$. Sim_{zk} runs in sub-exponential time in λ by assumption. Next, D'_λ uses a brute-force extraction procedure to compute $\tilde{m}$, which runs in time $2^{\lambda+|m_0|} \cdot \text{poly}(\lambda)$. It follows that $\text{size}(D'_\lambda) \in \text{poly}(2^{\lambda+|m_0|})$, as required.
- It remains to bound the depth of D'_λ . For this, we observe that Sim_{zk} can be computed in fixed polynomial depth by assumption. $|S|$ depends on Z and D , but we can do all checks for $(\tilde{c}_{\text{tlp}}, \hat{m}) \in S$ in parallel, requiring only fixed polynomial depth. Next, the brute-force extraction procedure for computing $\tilde{m}$ for $\text{Dec}_{\text{com}}(\tilde{c}_{\text{com}})$ can be computed in fixed polynomial depth by checking all messages and randomness values in parallel. All other operations have fixed polynomial running time in λ and $|m_0|$ by the efficiency of the underlying primitives, independent of Z and D . It follows that $\text{depth}(D'_\lambda) \leq \text{depth}(D_\lambda) \cdot d(\lambda, |m_0|, m_1)$.

It follows that the reduction is depth-preserving, as required. □

This completes the proof of lemma B.6 □

C Overview of Existing Constructions

In this section, we provide a brief overview for the most relevant encodings from [DKP21, FKPS21, BSS24, GKLW23] and emphasize the rate achieved in each approach.

- The non-malleable TLP of Dachman-Soled, Komargodski, and Pass [DKP21] (formalized as a TLP in [FKPS21]) follows the Naor-Yung paradigm [NY90] implicit also in [DDN91, LPS17, BL18]. They achieve non-malleability by using an appropriate zero-knowledge proof (from [BL18]) to ensure consistency between an underlying time-lock puzzle and a non-interactive non-malleable commitment to the message m . Their construction inherits bad rate from both the proof of consistency and the underlying encodings.

- The non-malleable TLP of Freitag, Komargodski, Pass, and Sirkin [FKPS21] relies only on a (plain) TLP in the AI-ROM. At a high level, they use the AI-ROM to instantiate the Fujisaki-Okamoto (FO) transform [FO13] for TLPs. They require that the underlying time-lock puzzle is $2^{|m|}$ -secure, so they have to set the associated security parameter $\lambda' \geq \max(\lambda, |m|)$. However, known time-lock puzzles of [RSW96, BGJ⁺16] incur polynomial overheads in their security parameter (e.g., for [RSW96], this follows from subexponential-time attacks on factoring), resulting in an overall scheme with inverse-polynomial rate.
- The non-malleable code of Ball, Shaltiel, and Silbak [BSS24] achieves rate-1 for bounded polynomial-size tampering. At a high level, their construction encodes a small seed s that is then expanded via a PRG to mask the message m , resulting in rate-1 code. Their particular encoding for the seed and PRG results in slow encoding (so their construction is not a TLP) and can only achieve inverse-polynomial security.
- The non-interactive commitment by Garg, Khurana, Lu, and Waters [GKLW23] goes through a notion of tagged non-malleability for commitments. In their main tag-amplification step, they also encode a seed s which is then expanded to mask the message m . Their final construction turns out to be rate-1, even though they don't explicitly make note of this. Their construction makes use of a hinting PRG for this step (which exist from either the CDH or LWE assumptions) as opposed to NIWIs from the construction of [BL18].

D Depth-Preserving Reductions

In this section, we give a helpful lemma for arguing about size- and depth-preserving reductions in this work. We formalize the notion of a depth- or size-preserving reduction as follows. At a high level, these are polynomial-time reductions for pairs of adversaries (Z, D) where Z specifies a time bound t that preserve either the size or depth, respectively, of the second adversary D up to some fixed polynomial.

Definition D.1 (Depth- and Size-Preserving Reductions). Consider a reduction that, given any pair of adversaries (Z, D) with advantage ϵ in a security game such that Z_λ outputs values (z_λ, t_λ) for every $\lambda \in \mathbb{N}$, produces another pair of adversaries (Z', D') with advantage ϵ' in another security game such that Z'_λ outputs values (z'_λ, t'_λ) with the same time bound for every $\lambda \in \mathbb{N}$. We say that such a reduction is β -size *depth-preserving* if there exists a polynomial d such that for any (Z, D) , the resulting adversary (Z', D') satisfies:

- $\text{size}(Z'_\lambda), \text{size}(D'_\lambda) \in \text{poly}(\lambda, \text{size}(Z_\lambda), \text{size}(D_\lambda), 1/\epsilon(\lambda), 1/\epsilon'(\lambda))$,
- $\text{size}(D'_\lambda) \in \text{poly}(\lambda, \beta(\lambda), \text{size}(Z_\lambda), \text{size}(D_\lambda), 1/\epsilon(\lambda), 1/\epsilon'(\lambda))$
- $|z'_\lambda| \leq d(\lambda, |z_\lambda|)$, and
- $\text{depth}(D'_\lambda) \leq \text{depth}(D_\lambda) \cdot d(\lambda, |z_\lambda|)$.

Furthermore, we say that a reduction is *size-preserving* if (Z', D') satisfies:

- $\text{size}(Z'_\lambda) \in \text{poly}(\lambda, \text{size}(Z_\lambda), \text{size}(D_\lambda), 1/\epsilon(\lambda), 1/\epsilon'(\lambda))$,
- $|z'_\lambda| \leq d(\lambda, |z_\lambda|)$, and

- $\text{size}(D'_\lambda) \leq \text{size}(D_\lambda) \cdot d(\lambda, |z_\lambda|)$.

We next formalize a helpful claim that if a reduction is depth-preserving for bounded polynomial depth adversaries, then the resulting adversary is also bounded polynomial depth. We note that a similar argument holds size-preserving reductions for bounded polynomial size adversaries. Furthermore, such a claim holds trivially for (non-bounded) polynomial size adversaries as long as the reduction is polynomially efficient.

Lemma D.2 (Depth-Preserving Reductions Preserve Bounded Polynomial Depth Hiding). *Let (Enc, Dec) and $(\text{Enc}', \text{Dec}')$ be encoding schemes. If $(\text{Enc}', \text{Dec}')$ satisfies β -size bounded polynomial depth hiding and there is a depth-preserving reduction from the bounded polynomial depth hiding of (Enc, Dec) to the β -size bounded polynomial depth hiding of $(\text{Enc}', \text{Dec}')$, then (Enc, Dec) satisfies bounded polynomial depth hiding.*

Proof. Suppose by way of contradiction that (Enc, Dec) does not satisfy bounded polynomial depth hiding. So, for every positive, monotonic polynomial α^* , there exists a pair of non-uniform adversaries (Z^*, D^*) and polynomials p^*, q^* such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (m_0, m_1, t) \leftarrow Z_\lambda^* \\ b \leftarrow \{0, 1\} \\ c \leftarrow \text{Enc}(1^\lambda, \text{aux}, m_b) \\ b' \leftarrow D_\lambda^*(c) \end{array} : \begin{array}{l} b' = b \wedge |m_0| = |m_1| = n \\ \wedge \alpha^*(\lambda, 2n) \leq t \leq q^*(\lambda) \\ \wedge \text{size}(D_\lambda) \leq q^*(\lambda) \\ \wedge \text{depth}(D_\lambda) \leq t/\alpha^*(\lambda, 2n) \end{array} \right] > 1/2 + 1/p^*(\lambda).$$

We want to reach a contradiction by showing that $(\text{Enc}', \text{Dec}')$ does not satisfy bounded polynomial depth hiding.

Let α' be any positive, monotonic polynomial and let $n = |m_0|$. Consider the polynomial $\alpha(\lambda, 2n) = \alpha'(\lambda, d(\lambda, 2n)) \cdot d(\lambda, 2n)$ and consider the associated pair of non-uniform adversaries (Z, D) for the hiding of (Enc, Dec) with distinguishing advantage $1/p$ for polynomial p that are guaranteed to exist for α by assumption. We consider the associated hiding adversary (Z', D') for $(\text{Enc}', \text{Dec}')$ with distinguishing advantage $1/p'$ for some polynomial p' guaranteed from the reduction in the lemma statement. We want to argue that there exists a polynomial q' such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (m'_0, m'_1, t) \leftarrow Z'_\lambda \\ b \leftarrow \{0, 1\} \\ c \leftarrow \text{Enc}'(1^\lambda, \text{aux}, m'_b) \\ b' \leftarrow D'_\lambda(c) \end{array} : \begin{array}{l} b' = b \wedge |m'_0| = |m'_1| = n' \\ \wedge \alpha'(\lambda, 2n') \leq t \leq q'(\lambda) \\ \wedge \text{size}(D'_\lambda) \leq q'(\lambda, \beta(\lambda)) \\ \wedge \text{depth}(D'_\lambda) \leq t/\alpha'(\lambda, 2n') \end{array} \right] > 1/2 + 1/p'(\lambda).$$

The success probability is already guaranteed by the reduction. So, it remains to argue that each of the efficiency properties hold.

The time bound t is the same value as output by Z_λ by assumption, and we are guaranteed that $\text{depth}(D_\lambda) \leq t/\alpha(\lambda, 2n)$. Since $\text{depth}(D'_\lambda) \leq \text{depth}(D_\lambda) \cdot d(\lambda, 2n)$, it follows that

$$\begin{aligned} \text{depth}(D'_\lambda) &\leq d(\lambda, 2n) \cdot \text{depth}(D_\lambda) \\ &\leq d(\lambda, 2n) \cdot t/(\alpha'(\lambda, d(\lambda, 2n)) \cdot d(\lambda, 2n)) \\ &= t/\alpha'(\lambda, d(\lambda, 2n)) \\ &\leq t/\alpha'(\lambda, 2n'), \end{aligned}$$

as required. Note that the last line follows from monotonicity of α' since $2n' \leq d(\lambda, 2n)$. Next, we note that the reduction guarantees Z' has size polynomially bounded in λ , $\text{size}(Z)$, $\text{size}(D)$, p , p' . Similarly, D' has size that is polynomially bounded in λ , $\beta(\lambda)$, $\text{size}(Z)$, $\text{size}(D)$, p , p' . So, there exists a polynomial q' (that may depend on the reduction and all of these parameters) that bounds the size of Z' and D' , as required. $\square$